fwap -- Full-Waveform Acoustic Processing

Release 0.4.0

fwap contributors

Apr 25, 2026

CONTENTS

1 Quick start **3**
- 1.1 Installation . . . 3
- 1.2 Documentation builds . . . 3
- 1.3 Run the demos . . . 4
- 1.4 Programmatic use . . . 4

2 Chapter-to-module map **7**
- 2.1 Supporting modules . . . 8
- 2.2 Recommended companion references . . . 9

3 API reference **11**
- 3.1 fwap.coherence . . . 11
- 3.2 fwap.picker . . . 14
- 3.3 fwap.wavesep . . . 29
- 3.4 fwap.tomography . . . 36
- 3.5 fwap.dispersion . . . 44
- 3.6 fwap.dip . . . 53
- 3.7 fwap.anisotropy . . . 58
- 3.8 fwap.attenuation . . . 65
- 3.9 fwap.rockphysics . . . 68
- 3.10 fwap.geomechanics . . . 80
- 3.11 fwap.cylindrical . . . 87
- 3.12 fwap.lwd . . . 89
- 3.13 fwap.synthetic . . . 96
- 3.14 fwap.io . . . 102
- 3.15 fwap.plotting . . . 109

4 fwap roadmap **111**
- 4.1 Released (for reference) . . . 111
- 4.2 Open items . . . 113
- 4.3 Non-goals . . . 116

5 Changelog **117**

6 Indices and tables **127**

Python Module Index **129**

Index **131**

Python implementation of the algorithms described in

> Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing.* Editions Technip, Paris, 136 pp. ISBN 978-2-7108-0664-6. (Originally published in French as *Traitement des diagraphies acoustiques.*)

The book picks four borehole-acoustic problems and works each one through from a raw multichannel waveform to a log curve a petrophysicist can actually use. This package provides a modern NumPy/SciPy implementation of the four chapter algorithms plus an extension layer that covers the post-1994 borehole-acoustic literature:

- cross-dipole Alford rotation, with a petrophysical labelling that exposes the max-horizontal-stress azimuth, the shear-wave splitting time, and a flexural-fracture indicator; **plus** the VTI Thomsen-γ shear-anisotropy parameter from the combined dipole shear (C_{44}) and Stoneley (C_{66} via the White / Norris tube-wave inversion) measurement (`fwap.anisotropy`);
- Q attenuation from the array sonic (`fwap.attenuation`);
- elastic moduli, Reuss / Voigt / Hill mixing, Gassmann fluid substitution, the Stoneley slowness-based permeability indicator, the amplitude-based fracture indicator, the Hornby et al. (1989) reflection-coefficient fracture-aperture inversion, and a slow-formation Vs estimator from the low-frequency Stoneley phase velocity (`fwap.rockphysics`);
- a geomechanics layer on top of `ElasticModuli`: Rickman brittleness / fracability index, Eaton uniaxial-strain closure stress, Lacy sandstone UCS, Bratli–Risnes sand-stability flag, density-log overburden integration, and a one-call `geomechanics_indices()` bundle (`fwap.geomechanics`);
- a Rayleigh-speed and physics-grounded flexural dispersion law (`fwap.cylindrical`);
- a stress-vs-intrinsic anisotropy classifier from a fast / slow cross-dipole flexural-dispersion-curve crossover (Sinha & Kostek 1996; `classify_flexural_anisotropy()`), plus a dispersion-corrected STC for the pseudo-Rayleigh / guided trapped mode (`dispersive_pseudo_rayleigh_stc()`);
- a phenomenological LWD (logging-while-drilling) layer (`fwap.lwd`): a steel-collar `Mode` factory, a slowness-band notch filter for collar rejection, and a quadrupole-source ring synthesizer + m=2 receiver-side stacker (Tang & Cheng 2004 sect. 2.4–2.5);
- a picker → log-curve bridge `track_to_log_curves()` that converts a track of `DepthPicks` into the `{mnemonic: ndarray}` dict the LAS / DLIS writers consume directly;
- log-format I/O for LAS, DLIS and SEG-Y (`fwap.io`).

CHAPTER

ONE

QUICK START

1.1 Installation

```
pip install -e .
```

The package requires Python >= 3.9. Core dependencies are NumPy, SciPy, Matplotlib (Matplotlib only for the demos and CLI), and the log-format libraries `lasio` (LAS), `dlisio` + `dliswriter` (DLIS), and `segyio` (SEG-Y).

1.2 Documentation builds

Pre-built PDF snapshots of this manual live in `docs/`:

- `docs/fwap.pdf` – all-in-one reference, ~140 pages. GitHub's blob viewer lazy-loads PDFs and only renders the first few pages inline; download the file (right-click *Download* on the GitHub blob page, or use the `raw` URL) to read all of it.
- `docs/fwap-quickstart.pdf` – Quick-start section only.
- `docs/fwap-chapter-map.pdf` – Chapter-to-module map.
- `docs/fwap-roadmap.pdf` – Roadmap.
- `docs/fwap-changelog.pdf` – Changelog.

The four per-section PDFs are short enough (3-9 pages each) that GitHub's blob viewer renders them inline; the all-in-one PDF is the reference for offline reading.

To regenerate the HTML or PDFs locally:

```
pip install -e .[docs]
sphinx-build -b html docs docs/_build/html       # HTML

# PDFs: needs a TeX Live install (xelatex + xindy + poppler-utils):
sphinx-build -b latex docs docs/_build/latex
make -C docs/_build/latex                        # builds all PDFs
```

`make` produces five PDFs in `docs/_build/latex/`: the all-in-one `fwap.pdf` plus four per-section PDFs configured via `latex_documents` in `docs/conf.py`. The LaTeX builder is pinned to `xelatex` in `docs/conf.py` because some of the docstrings carry Unicode math glyphs that `pdflatex` cannot typeset.

1.3 Run the demos

```
python -m fwap
```

or equivalently `fwap all`. This writes a diagnostic figure per algorithm to `figures/`.

To run a specific demo:

```
fwap stc            # Part 1: STC + rule-based picker
fwap pseudorayleigh # Part 1: 4-mode picker incl. pseudo-Rayleigh
fwap wavesep        # Part 2: f-k filter + SVD / Karhunen-Loeve
fwap taup           # Part 2: tau-p / slant-stack / linear Radon
fwap intercept      # Part 3: Coppens & Mari intercept-time inversion
fwap dipole         # Part 3: dipole flexural dispersion
fwap dip            # Part 4: dip / azimuth from azimuthal array
fwap alford         # Extension: cross-dipole Alford rotation
fwap attenuation    # Extension: Q by centroid shift and spectral ratio
fwap lwd            # Extension: LWD collar rejection + quadrupole stack
fwap las            # Extension: LAS I/O round-trip
fwap dlis           # Extension: DLIS I/O round-trip
fwap segy           # Extension: SEG-Y I/O round-trip
```

1.4 Programmatic use

```
from fwap import (
    ArrayGeometry, monopole_formation_modes, synthesize_gather,
    stc, pick_modes,
)

geom = ArrayGeometry.schlumberger_array_sonic()
data = synthesize_gather(geom, monopole_formation_modes())
surface = stc(data, dt=geom.dt, offsets=geom.offsets,
              window_length=4.0e-4)
picks = pick_modes(surface)    # {"P": ..., "S": ..., "Stoneley": ...}
```

Each `ModePick` carries the picked slowness, arrival time, semblance coherence, **and per-mode amplitude** (the RMS of the per-trace stack contribution at the picked cell), so the two halves of the book's Workflow-1 deliverable – amplitude *and* coherence logs – come straight off the picker:

```
for name, p in picks.items():
    print(f"{name}: slow={p.slowness * 1e6 / 3.281:.2f} us/ft  "
          f"coh={p.coherence:.3f}  amp={p.amplitude:.3f}")
```

Cross-mode consistency QC flags depths where the picks aren't internally consistent (Vp/Vs out of band, or canonical mode time ordering violated):

```
from fwap import quality_control_picks

qc = quality_control_picks(picks, depth=1000.0)
if qc.flagged:
    print(f"flagged at {qc.depth} m: {qc.reasons}")
```

Rock-physics moduli from the recovered logs:

```
from fwap import elastic_moduli

m = elastic_moduli(vp=4500.0, vs=2500.0, rho=2400.0)
print(m.young / 1e9, "GPa")      # Young's modulus
print(m.poisson)                 # Poisson's ratio
```

Geomechanics indices on top of `ElasticModuli` for the Workflow-3 deliverables (Rickman brittleness / fracability, Eaton closure stress, Lacy sandstone UCS, Bratli-Risnes sand-stability flag):

```
from fwap import geomechanics_indices, overburden_stress

sigma_v = overburden_stress(depth, density)             # density log
geo = geomechanics_indices(
    m, sigma_v_pa=sigma_v, pore_pressure_pa=10.0e6,
)
print(geo.brittleness, geo.fracability, geo.ucs / 1e6, "MPa",
      geo.closure_stress / 1e6, "MPa", geo.sand_stability)
```

Picker -> log-curve bridge converts a per-depth pick track straight into the `{mnemonic: ndarray}` dict the LAS / DLIS writers consume:

```
from fwap import (
    track_modes, track_to_log_curves, write_las,
)

track = track_modes(stc_results, depths)
depths_arr, curves = track_to_log_curves(track)
write_las("output.las", depths_arr, curves)
# curves carries DTP / DTS / DTST / COHP / COHS / COHST / AMP* /
# VPVS in the standard fwap mnemonics; slowness in us/ft.
```

VTI Thomsen-γ from the combined dipole shear (C_{44}) and Stoneley low-frequency tube-wave inversion (C_{66}):

```
from fwap import thomsen_gamma_from_logs

res = thomsen_gamma_from_logs(
    slowness_dipole=dts_curve,          # s/m
    slowness_stoneley=dtst_curve,       # s/m
    rho=density_log,                    # kg/m^3
    rho_fluid=1000.0, v_fluid=1500.0,
)
# res.c44, res.c66 in Pa; res.gamma is the Thomsen shear-anisotropy
# parameter (0 for isotropic, positive for typical VTI shales).
```

Slow-formation Vs from the low-frequency Stoneley phase velocity, the primary sonic-only V_S route when the formation has no S head wave on a monopole gather and pseudo-Rayleigh does not exist:

```
from fwap import vs_from_stoneley_slow_formation

vs = vs_from_stoneley_slow_formation(
    slowness_stoneley=dtst_curve, rho=density_log,
    rho_fluid=1000.0, v_fluid=1500.0,
)
```

LWD (logging-while-drilling) phenomenological layer – collar contamination synthesis + slowness-band rejection:

```
from fwap import (
    monopole_formation_modes, synthesize_lwd_gather,
    notch_slowness_band, DEFAULT_COLLAR_SLOWNESS_S_PER_M,
    stc, pick_modes,
)

# Synthesize a monopole gather contaminated by the LWD collar.
data = synthesize_lwd_gather(
    geom, monopole_formation_modes(),
    collar_slowness=DEFAULT_COLLAR_SLOWNESS_S_PER_M,
)
# Notch a +/- 15 % band around the known collar slowness.
c = DEFAULT_COLLAR_SLOWNESS_S_PER_M
cleaned = notch_slowness_band(
    data, dt=geom.dt, offsets=geom.offsets,
    slow_min=c * 0.85, slow_max=c * 1.15,
)
# Run STC + pick_modes on the cleaned record as usual.
```

CHAPTER

TWO

CHAPTER-TO-MODULE MAP

Each algorithm module of `fwap` implements one Part of Mari, Coppens, Gavin & Wicquart (1994). Several additional modules implement downstream workflows and log-format I/O that are not in the book.

Book part	Topic	Module(s)
Part 1	AI picking of P / S / pseudo-Rayleigh / Stoneley (slowness windows + STC + Viterbi continuity + wavelet-shape and onset-polarity expert rules)	*fwap.coherence*, *fwap.picker*
Part 2	Wave separation: f-k, tau-p / slant-stack, and SVD / Karhunen-Loeve	*fwap.wavesep*
Part 3	Intercept-time inversion (with altered-zone thickness + velocity-contrast pair) and dipole-flexural processing	*fwap.tomography*, *fwap.dispersion*
Part 4	Dip measurement based on acoustic data	*fwap.dip*
(extension)	Cross-dipole Alford rotation + petrophysical labelling (max-H stress azimuth, splitting time, fracture indicator); VTI Thomsen-gamma from dipole-derived C_44 and Stoneley-derived C_66 (White / Norris tube-wave inversion)	*fwap.anisotropy*
(extension)	Q from array sonic	*fwap.attenuation*
(extension)	Elastic moduli from Vp, Vs, rho; Reuss / Voigt / Hill mixing; Gassmann fluid substitution; Stoneley slowness-based permeability indicator + amplitude-based fracture indicator + Hornby (1989) reflection-coefficient aperture inversion; slow-formation Vs from low-frequency Stoneley phase velocity	*fwap.rockphysics*
(extension)	Geomechanics indices on top of *ElasticModuli*: Rickman brittleness / fracability, Eaton uniaxial-strain closure stress, Lacy sandstone UCS, Bratli-Risnes sand-stability flag, density-log overburden integration, one-call `geomechanics_indices()` bundle	*fwap.geomechanics*
(extension)	Cylindrical-borehole surface-wave speeds and a physics-grounded flexural dispersion law	*fwap.cylindrical*
(extension)	LWD phenomenological layer: collar Mode factory, slowness-band notch for collar rejection, quadrupole-source ring synthesis, m=2 receiver- side stacker, LWD-tuned picker priors (Tang & Cheng 2004 sect. 2.4-2.5)	*fwap.lwd*
(extension)	Dispersion-curve / dispersive-STC additions: dispersive STC for the pseudo-Rayleigh / guided trapped mode; stress-vs-intrinsic anisotropy classifier from a fast / slow flexural dispersion-curve crossover (Sinha & Kostek 1996)	*fwap.dispersion*
(extension)	Picker -> log-curve bridge: `track_to_log_curves()` converts a per-depth pick track into a LAS/DLIS-ready `{mnemonic: ndarray}` dict	*fwap.picker*
(extension)	LAS / DLIS / SEG-Y log-format I/O	*fwap.io*

2.1 Supporting modules

- *fwap.synthetic* – canonical P/S/Stoneley/pseudo-Rayleigh test gathers, plus the phenomenological flexural and pseudo- Rayleigh dispersion laws used by the demos and tests.
- `fwap.demos` – one worked example per chapter (plus one per extension).
- `fwap.cli` – the `fwap` command-line demo runner.

2.2 Recommended companion references

- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes.* CRC Press. (Forward / mode-theory companion to Mari 1994.)
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier. (Inverse / processing companion; underpins the Stoneley-permeability, Stoneley-fracture, Thomsen-γ and LWD layers.)
- Mari, J.-L., Glangeaud, F., & Coppens, F. (1999). *Signal Processing for Geologists and Geophysicists.* Editions Technip, Paris. ISBN 2-7108-0752-1.
- Mari, J.-L., & Vergniault, C. (2018). *Well Seismic Surveying and Acoustic Logging.* EDP Open.
- Coppens, F., & Mari, J.-L. (1995). Application of the intercept time method to full waveform acoustic data. *First Break* 13(1), 11-20.
- Hornby, B. E., Johnson, D. L., Winkler, K. W., & Plumb, R. A. (1989). Fracture evaluation using reflected Stoneley-wave arrivals. *Geophysics* 54(10), 1274-1288.
- Rickman, R., Mullen, M. J., Petre, J. E., Grieser, W. V., & Kundert, D. (2008). A practical use of shale petrophysics for stimulation design optimization. *SPE 115258*.
- Sinha, B. K., & Kostek, S. (1996). Stress-induced azimuthal anisotropy in borehole flexural waves. *Geophysics* 61(6), 1899-1907.

CHAPTER

THREE

API REFERENCE

Every module below is regenerated from its source docstrings. Start from *Chapter-to-module map* to see how the modules map onto the 1994 book.

`fwap.coherence`	Slowness-time coherence (STC / semblance).
`fwap.picker`	Rule-based P / S / pseudo-Rayleigh / Stoneley picker on STC coherence surfaces.
`fwap.wavesep`	Wave separation -- f-k, tau-p, and SVD / Karhunen-Loeve projection.
`fwap.tomography`	Intercept-time (delay) inversion for borehole acoustic logs.
`fwap.dispersion`	Dipole flexural processing: dispersion estimation and dispersive STC.
`fwap.dip`	Dip and azimuth estimation from a borehole ring array.
`fwap.anisotropy`	Cross-dipole Alford rotation for shear-wave azimuthal anisotropy.
`fwap.attenuation`	Attenuation (quality factor Q) estimation from array-sonic gathers.
`fwap.rockphysics`	Elastic moduli from sonic velocities.
`fwap.geomechanics`	Geomechanics indices derived from elastic moduli.
`fwap.cylindrical`	Elastic surface-wave speeds and cylindrical-mode dispersion models.
`fwap.lwd`	Logging-While-Drilling (LWD) acoustic processing -- phenomenological layer.
`fwap.synthetic`	Synthetic multi-receiver borehole gathers.
`fwap.io`	I/O wrappers for the formats sonic data arrives in.
`fwap.plotting`	Plotting helpers for the fwap demos and for interactive use.

3.1 fwap.coherence

Slowness-time coherence (STC / semblance).

Implements the coherence surface that the rule-based picker in `fwap.picker` consumes. STC is the workhorse of Part 1 of the book (automatic multi-wave log production).

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 1. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Kimball, C. V., & Marzetta, T. L. (1984). Semblance processing of borehole acoustic array data. *Geophysics* 49(3), 274-281.
- Neidell, N. S., & Taner, M. T. (1971). Semblance and other coherency measures for multichannel data. *Geophysics* 36(3), 482-497.

Functions

`find_peaks`(result[, threshold, ...])	Local-maxima picker on an STC coherence surface.
`semblance`(window)	Semblance of co-aligned windowed traces.
`stc`(data, dt, offsets[, slowness_range, ...])	Slowness-time coherence map (Kimball & Marzetta, 1984).

Classes

`STCResult`(slowness, time, coherence, ...[, ...])	Slowness-time coherence surface returned by `stc()`.

class `fwap.coherence.`**`STCResult`**(*slowness*, *time*, *coherence*, *window_length*, *amplitude=None*)

Bases: `object`

Slowness-time coherence surface returned by `stc()`.

Parameters

- **slowness** (*ndarray*)
- **time** (*ndarray*)
- **coherence** (*ndarray*)
- **window_length** (*float*)
- **amplitude** (*ndarray | None*)

slowness

Trial slownesses (s/m), increasing.

Type
ndarray, shape (n_slowness,)

time

Start time of each STC window relative to the start of the record (s).

Type
ndarray, shape (n_time,)

coherence

Semblance value in `[0, 1]`; `NaN` for windows whose total energy falls below the `min_energy_fraction` floor passed to `stc()`.

Type
ndarray, shape (n_slowness, n_time)

window_length

STC time-window length actually used (s). Kept on the result so downstream picker code (see *fwap.picker.pick_modes()* and *fwap.picker.track_modes()*) can derive a sensible default time scale.

Type
float

amplitude

Per-cell stack amplitude (same units as the input data), defined as the RMS over the STC time window of the per-trace stack `(sum_i x_i_aligned) / N_traces`. For a single coherent mode of amplitude `A` on every trace this equals the RMS of `A` (so a unit-amplitude sine on every trace gives `amplitude = 1/sqrt(2)`). `NaN` at the same bins where `coherence` is `NaN`. `None` on results from STC variants that have not yet been extended to track amplitude (e.g. the dispersion-corrected STC of *fwap.dispersion.dispersive_stc()*).

Type
ndarray or None, shape (n_slowness, n_time)

amplitude: ndarray | None = None

coherence: ndarray

slowness: ndarray

time: ndarray

window_length: float

`fwap.coherence.`**find_peaks**(*result*, *threshold=0.5*, *min_separation_s=0.0001*, *min_separation_slow=1.5e-05*)

Local-maxima picker on an STC coherence surface.

Uses `scipy.ndimage.maximum_filter` to find each cell equal to its neighbourhood maximum in a single vectorised pass.

Returns
peaks – Rows are `[slowness, time, coherence]` – or `[slowness, time, coherence, amplitude]` when the STC result carries an `amplitude` map – sorted by descending coherence.

Return type
ndarray, shape (n_peaks, n_cols)

Parameters

- **result** (`STCResult`)
- **threshold** (*float*)
- **min_separation_s** (*float*)
- **min_separation_slow** (*float*)

`fwap.coherence.`**semblance**(*window*)

Semblance of co-aligned windowed traces.

Parameters
window (*ndarray, shape (N, L)*) – `N` traces of length `L`, already time-aligned on the trial slowness.

Returns
Semblance in `[0, 1]` (1 = perfectly coherent). Returns `NaN` when the denominator is zero (an all-zero window).

Return type
float

Notes

Semblance (Neidell & Taner, 1971, *Geophysics* 36(3), 482-497) is the ratio of stack power to sum-of-traces power:

rho = (sum_t (sum_i x_i(t))2)**
N * sum_t sum_i x_i(t)**2

which lies in [0, 1] with 1 for perfectly coherent traces. Callers needing an energy floor should filter windows upstream – the per-window cost of that check in the inner loop is identical, and the appropriate threshold depends on the caller.

`fwap.coherence.`**`stc`**(*data*, *dt*, *offsets*, *slowness_range=(5e-05, 0.0005)*, *n_slowness=181*, *window_length=0.0004*, *time_step=1*, *min_energy_fraction=1e-08*)

Slowness-time coherence map (Kimball & Marzetta, 1984).

Trace alignment is done in the frequency domain (a fractional-sample phase shift per slowness) so that slowness resolution is a true function of the slowness grid, not quantised by `dt / dx`. One rFFT / iRFFT pair per slowness plus a single `np.lib.stride_tricks.sliding_window_view` stack gives a fully vectorised implementation.

Parameters

- **`data`** (*`ndarray, shape (n_rec, n_samples)`*)
- **`dt`** (*`float`*) – Sampling interval (s).
- **`offsets`** (*`ndarray, shape (n_rec,)`*) – Source-to-receiver offsets (m).
- **`slowness_range`** (*`(float, float)`*) – Lower and upper slowness to scan (s/m).
- **`n_slowness`** (*`int`*) – Number of slowness samples in the grid.
- **`window_length`** (*`float`*) – STC time-window length (s).
- **`time_step`** (*`int`*) – Stride of window starts in samples.
- **`min_energy_fraction`** (*`float`*) – Windows with energy below this fraction of the gather RMS are returned as `NaN` in the coherence map.

Return type
STCResult

3.2 fwap.picker

Rule-based P / S / pseudo-Rayleigh / Stoneley picker on STC coherence surfaces.

Implements the knowledge-based picker described in Part 1 of the book (an "AI approach for the picking of waves on full-waveform acoustic data"): expert rules on slowness range, coherence threshold, mode ordering and depth-to-depth continuity are codified into a deterministic selection over the candidate peaks returned by *`fwap.coherence.find_peaks()`*. Two further book-listed expert rules – **wavelet shape** and **onset polarity** – are exposed as post-pick filters via *`filter_picks_by_shape()`* (single-depth) and *`filter_track_by_shape()`* (multi-depth track) so callers can opt in without altering the core slowness / coherence / continuity selection. Default priors cover the four monopole arrivals enumerated by Mari et al. – the P head-wave, the S head-wave, the pseudo-Rayleigh / guided trapped mode (when present in fast formations), and the low-frequency Stoneley tube wave.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 1. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Aron, J., Chang, S. K., Codazzi, D., Dworak, R., Hsu, K., Lau, T., Minerbo, G., & Yogeswaren, E. (1994). Real-time sonic logging while drilling in hard and soft rocks. *SEG Technical Program Expanded Abstracts*, 13, 1-4.
- Serra, O. (1984). *Fundamentals of Well-Log Interpretation: 1. The Acquisition of Logging Data.* Elsevier (for the slowness windows used in `DEFAULT_PRIORS`).

Functions

`filter_picks_by_shape`(picks, data, dt, ...)	Drop picks whose stacked waveform fails the polarity / shape rules.
`filter_track_by_shape`(track_picks, datas, ...)	Apply `filter_picks_by_shape()` per-depth across a track.
`onset_polarity`(stack)	Sign of the largest-absolute sample in a stacked waveform.
`pick_modes`(stc_result[, priors, threshold, ...])	Label P / S / PseudoRayleigh / Stoneley modes on an STC surface via physical rules.
`quality_control_picks`(picks, *[, depth, ...])	Cross-mode consistency QC at one depth.
`quality_control_track`(track, *[, vp_vs_min, ...])	Apply `quality_control_picks()` per-depth across a track.
`track_modes`(stc_results, depths[, priors, ...])	Per-depth picking with a depth-aware continuity regulariser.
`track_to_log_curves`(track, *[, modes, ...])	Convert a per-depth pick track into LAS/DLIS-ready log curves.
`viterbi_pick`(stc_results, depths[, priors, ...])	Joint-log-likelihood (Viterbi) mode picker across a depth sweep.
`viterbi_pick_joint`(stc_results, depths[, ...])	Fully-joint 3-mode Viterbi picker.
`viterbi_posterior_marginals`(stc_results, depths)	Joint 3-mode forward-backward: MAP picks plus per-mode posterior marginals.
`wavelet_shape_score`(stack, dt, f0)	Absolute Pearson correlation of a stacked window vs a Ricker(f0).

Classes

`DepthPicks`(depth, picks)	
`ModePick`(name, slowness, time, coherence[, ...])	A single mode pick at one depth.
`PickQualityFlags`(depth, vp_vs, ...)	Per-depth cross-mode consistency QC for a multi-mode pick set.
`PosteriorPick`(slownesses, times, coherences, ...)	Per-mode, per-depth posterior marginal from `viterbi_posterior_marginals()`.

class `fwap.picker.`**`DepthPicks`**(*depth: 'float', picks: 'dict[str, ModePick]'=<factory>*)

Bases: `object`

Parameters

- **depth** (*float*)
- **picks** (*dict[str, ModePick]*)

depth: float

picks: dict[str, *ModePick*]

class fwap.picker.**ModePick**(*name*, *slowness*, *time*, *coherence*, *amplitude=None*)

Bases: object

A single mode pick at one depth.

amplitude is the per-trace stack amplitude at the picked (slowness, time) cell of the STC surface (see *fwap.coherence.STCResult.amplitude* for the exact definition). It is the second leg – alongside coherence – of the per-mode amplitude/coherence log pair that Mari et al. (1994), Part 1 list as the rule-based picker's deliverable. None when the upstream STC variant did not populate STCResult.amplitude (currently only some legacy paths).

Parameters

- **name** (*str*)
- **slowness** (*float*)
- **time** (*float*)
- **coherence** (*float*)
- **amplitude** (*float | None*)

amplitude: float | None = None

coherence: float

name: str

slowness: float

time: float

class fwap.picker.**PickQualityFlags**(*depth*, *vp_vs*, *vp_vs_in_band*, *time_order_ok*, *flagged*, *reasons*)

Bases: object

Per-depth cross-mode consistency QC for a multi-mode pick set.

Returned by *quality_control_picks()* (and the multi-depth *quality_control_track()*). Covers the *cross-consistency between modes* layer of the book's QC philosophy (Mari et al. 1994, Part 1, closing paragraph) – the *log continuity* layer is enforced inside the Viterbi pickers.

Parameters

- **depth** (*float*)
- **vp_vs** (*float | None*)
- **vp_vs_in_band** (*bool*)
- **time_order_ok** (*bool*)
- **flagged** (*bool*)
- **reasons** (*tuple[str, ...]*)

depth

Depth (m) the QC was computed at.

Type
float

vp_vs

Vp/Vs ratio derived from the P and S picks (= `s_S / s_P`). `None` when either pick is missing or when `s_P` is zero.

Type
float or None

vp_vs_in_band

True when `vp_vs` lies inside the configured `[vp_vs_min, vp_vs_max]` physical band, *or* when no Vp/Vs could be computed (a missing Vp/Vs is not flagged as inconsistent – it just isn't checked).

Type
bool

time_order_ok

True when the picked arrival times respect the canonical ordering `t_P <= t_S <= t_PseudoRayleigh <= t_Stoneley`, skipping modes that weren't picked. Useful when the upstream picker was run with a soft time-order constraint (`viterbi_pick(time_order_slack > 0)` / `viterbi_pick_joint(soft_time_order=...)`) where the ordering can be deliberately violated.

Type
bool

flagged

True when any check failed at this depth.

Type
bool

reasons

Human-readable per-check failure descriptions; empty when `flagged` is False.

Type
tuple of str

depth: float

flagged: bool

reasons: tuple[str, ...]

time_order_ok: bool

vp_vs: float | None

vp_vs_in_band: bool

class `fwap.picker.`**PosteriorPick**(*slownesses*, *times*, *coherences*, *probabilities*, *p_absent*)

Bases: `object`

Per-mode, per-depth posterior marginal from *`viterbi_posterior_marginals()`*.

Unlike *`ModePick`*, which carries a single best-estimate slowness and its coherence, a `PosteriorPick` describes the full posterior probability distribution over candidate picks (including the probability that the mode is absent at this depth).

Parameters

- **slownesses** (*ndarray*)
- **times** (*ndarray*)

- **coherences** (*ndarray*)
- **probabilities** (*ndarray*)
- **p_absent** (*float*)

slownesses

Slownesses of the in-window candidates for this mode at this depth.

Type
ndarray, shape (n_candidates,)

times

Arrival times of the candidates.

Type
ndarray, shape (n_candidates,)

coherences

Coherence values of the candidates (the per-cell emission before time / absence bonuses).

Type
ndarray, shape (n_candidates,)

probabilities

Posterior probability that this mode is picked at the corresponding candidate, summed over all triples containing it. `probabilities.sum() + p_absent == 1.0`.

Type
ndarray, shape (n_candidates,)

p_absent

Posterior probability that the mode is absent at this depth.

Type
float

coherences: ndarray

p_absent: float

probabilities: ndarray

slownesses: ndarray

times: ndarray

`fwap.picker.`**filter_picks_by_shape**(*picks*, *data*, *dt*, *offsets*, ** (Keyword-only parameters separator (PEP 3102))*, *priors=None*, *window_length=0.0004*, *analysis_factor=2.0*)

Drop picks whose stacked waveform fails the polarity / shape rules.

Implements two of the book's expert-rule layers (Mari et al. 1994, Part 1) as a post-pick filter, applied to the dict of per-mode picks returned by `pick_modes()`. Picks whose mode in `priors` declares neither rule are passed through unchanged.

Two opt-in rules per mode (read from each mode's prior dict):

- `polarity` (`+1` / `-1` / `0`): expected sign of the stacked window's largest-absolute sample. `0` (the default) disables the gate.
- `shape_match_min` (float in `[0, 1]`): minimum absolute Pearson correlation between the stacked window and a Ricker template at the prior's `f0` (Hz). `0.0` disables the gate. When enabled, the prior **must** also carry an `f0` key.

Parameters

- **picks** (*dict from str to ModePick*) – Per-mode picks at one depth, e.g. the output of *pick_modes()*.
- **data** (*ndarray, shape (n_rec, n_samples)*) – The same gather the picks were derived from. Used to re-stack the moveout-aligned window at each pick's `(slowness, time)`.
- **dt** (*float*) – Sampling interval (s).
- **offsets** (*ndarray, shape (n_rec,)*) – Source-to-receiver offsets (m).
- **priors** (*dict, optional*) – Per-mode prior windows, with optional `polarity`, `shape_match_min` and `f0` keys driving the gates. Defaults to `DEFAULT_PRIORS`, which declares neither gate so the default-priors call is a no-op pass-through.
- **window_length** (*float, default 4e-4*) – STC time-window length (s) used to produce the picks. The polarity / shape gates analyse a window centred on the STC window's midpoint – `pick.time + window_length / 2` – because `pick.time` is a window-start time and a window started there can capture a sidelobe rather than the main wavelet lobe.
- **analysis_factor** (*float, default 2.0*) – Width of the analysis window as a multiple of `window_length`. The default `2.0` gives the polarity / shape gates one STC-window of context on each side of the STC window's centre, which is enough to bracket a Ricker's main lobe plus its near sidelobes for the typical sonic prior frequencies (3-15 kHz).

Returns
A new dict containing only the picks that passed every enabled gate. The original `picks` dict is not mutated.

Return type
dict from str to ModePick

`fwap.picker.`**`filter_track_by_shape`**(*track_picks*, *datas*, *dt*, *offsets*, ***, *priors=None*, *window_length=0.0004*, *analysis_factor=2.0*)

Apply *filter_picks_by_shape()* per-depth across a track.

The multi-depth analogue of *filter_picks_by_shape()*: the same polarity / shape gates run once per depth against the matching per-depth gather. `track_picks` and `datas` must have the same length.

Parameters

- **track_picks** (*sequence of DepthPicks*) – Output of *track_modes()*, *viterbi_pick()*, or *viterbi_pick_joint()*.
- **datas** (*sequence of ndarray, shape (n_rec, n_samples)*) – One gather per depth, in the same order as `track_picks`.
- **dt** (as in *filter_picks_by_shape()*.)
- **offsets** (as in *filter_picks_by_shape()*.)
- **priors** (as in *filter_picks_by_shape()*.)
- **window_length** (as in *filter_picks_by_shape()*.)
- **analysis_factor** (*float*)

Returns
Filtered track. `DepthPicks` instances are new; the `picks` dicts are filtered copies of the originals.

Return type
list of *DepthPicks*

`fwap.picker.onset_polarity(stack)`

Sign of the largest-absolute sample in a stacked waveform.

Returns `+1` when the dominant excursion is positive, `-1` when negative, and `0` for an all-zero input. The book's "onset polarity" expert rule (Mari et al. 1994, Part 1) gates picks against an expected first-motion sign convention; the dominant excursion of an STC-window-stacked pulse-like wavelet is the main lobe at the prior centre frequency, so its sign is the natural per-pick polarity readout.

3.2.1 Caveat – pulse-like wavelets only

This heuristic is only meaningful for *pulse-like* wavelets where one excursion clearly dominates the others (Ricker P / Ricker S, in the canonical 1994 monopole gather). For multi-cycle wavetrains – Gabor / Stoneley – the dominant excursion can be either a peak or a trough depending on the sub-sample alignment of the analysis window, so the polarity readout is not stable. In practice the polarity gate in `DEFAULT_PRIORS` should only be enabled on the impulsive modes (P, S); leave Stoneley and PseudoRayleigh at `polarity=0` (the default = "ignore").

Parameters

stack (*ndarray*)

Return type

int

`fwap.picker.pick_modes(stc_result, priors=None, threshold=0.4, *, selection_rule='scored', time_penalty=0.1, time_scale=None)`

Label P / S / PseudoRayleigh / Stoneley modes on an STC surface via physical rules.

The `selection_rule` keyword controls how ties within a prior window are broken; see `_best_candidate()` for the three strategies. PseudoRayleigh is silently absent on gathers that don't carry a guided-mode arrival – its prior window is empty on a 3-mode (P + S + Stoneley) synthetic.

Parameters

- **stc_result** (`STCResult`) – Slowness-time coherence surface from `stc()`.
- **priors** (*dict, optional*) – Per-mode prior windows. Defaults to `DEFAULT_PRIORS`.
- **threshold** (*float*) – Coherence threshold for the peak picker.
- **selection_rule** (*str*) – See `_best_candidate()`.
- **time_penalty** (*float*) – Weight of the time penalty in the `'scored'` rule; ignored otherwise.
- **time_scale** (*float, optional*) – Time normaliser for the `'scored'` rule. Defaults to `stc_result.window_length`.

Return type

dict[str, ModePick]

Notes

The mode-ordering rule is still strict (P -> S -> Stoneley, each required to be no earlier in time than the previous). In altered zones the S head-wave can appear before the formation P re-emerges (see Aron et al., 1994, *SEG Expanded Abstracts*); for those cases a joint log-likelihood across modes is more robust than the greedy rule used here. This is flagged as future work.

`fwap.picker.quality_control_picks(picks, *, depth=None, vp_vs_min=1.4, vp_vs_max=2.6, require_time_order=True)`

Cross-mode consistency QC at one depth.

Two checks:

- **Vp/Vs in physical band.** Computed as `s_S / s_P` (which equals Vp/Vs since slowness is the reciprocal of velocity). Flagged when outside `[vp_vs_min, vp_vs_max]`. Skipped – and reported as `vp_vs_in_band=True` – when either P or S is missing.
- **Canonical time ordering.** Flagged when the picked arrival times do not satisfy `t_P <= t_S <= t_PseudoRayleigh <= t_Stoneley` over the modes that were picked. Disable by passing `require_time_order=False`.

The function only **flags** – it never modifies the picks. The caller decides what to do with flagged depths (drop, mark in plots, hand to a human for review).

Parameters

- **picks** (*dict from str to ModePick, or* `DepthPicks`) – Per-mode picks at one depth. When a *`DepthPicks`* is passed, its `depth` field is used unless overridden by the explicit `depth` keyword.
- **depth** (*float, optional*) – Override depth value. Required when `picks` is a plain dict; ignored otherwise unless given.
- **vp_vs_min** (*float*) – Inclusive Vp/Vs band. Defaults span the canonical sedimentary-rock range from gas-charged sandstones (~1.4) to clay-rich shales / fluid-saturated carbonates (~2.6).
- **vp_vs_max** (*float*) – Inclusive Vp/Vs band. Defaults span the canonical sedimentary-rock range from gas-charged sandstones (~1.4) to clay-rich shales / fluid-saturated carbonates (~2.6).
- **require_time_order** (*bool, default True*) – Disable to skip the canonical-ordering check entirely.

Return type
PickQualityFlags

`fwap.picker.`**`quality_control_track`**(*track*, ***, *vp_vs_min=1.4*, *vp_vs_max=2.6*, *require_time_order=True*)

Apply *`quality_control_picks()`* per-depth across a track.

Parameters

- **track** (*sequence of* `DepthPicks`) – Output of *`track_modes()`*, *`viterbi_pick()`*, *`viterbi_pick_joint()`*, or any other multi-depth picker.
- **vp_vs_min** (*as in*) – *`quality_control_picks()`*.
- **vp_vs_max** (*as in*) – *`quality_control_picks()`*.
- **require_time_order** (*as in*) – *`quality_control_picks()`*.

Returns
One entry per depth, in the order of `track`.

Return type
list of *PickQualityFlags*

`fwap.picker.`**`track_modes`**(*stc_results*, *depths*, *priors=None*, *threshold=0.4*, *max_slow_jump=0.00016404199475065617*, *continuity_max_gap=None*, *continuity_tol_growth=0.5*, *continuity_tol_cap_factor=3.0*, ***, *selection_rule='scored'*, *time_penalty=0.1*, *time_scale=None*)

Per-depth picking with a depth-aware continuity regulariser.

The continuity constraint stores both the last successful pick's slowness and the depth at which it was picked per mode. The effective jump tolerance grows with the depth gap since the last pick, so a mode missed in a disturbed

zone can be re-acquired at a slightly different slowness a few depths later without the tracker treating that as a violation. Beyond `continuity_max_gap` the constraint is dropped entirely, preventing the tracker from remaining locked onto a stale slowness across extended data gaps.

Parameters

- **stc_results** (*sequence of STCResult*) – One STC surface per depth.
- **depths** (*ndarray, shape (n_depth,)*) – Tool depth (m) for each STC surface, same length as `stc_results`.
- **priors** (*dict, optional*) – Per-mode prior windows. Defaults to `DEFAULT_PRIORS`.
- **threshold** (*float*) – Coherence threshold for the peak picker.
- **max_slow_jump** (*float*) – Slowness jump tolerance (s/m) between adjacent depths with zero gap growth. Scaled up by `continuity_tol_growth` for each depth-unit of gap since the last successful pick on the same mode. Default: 50 us/ft.
- **continuity_max_gap** (*float, optional*) – Depth gap (m) beyond which the continuity constraint is dropped entirely for a given mode. Defaults to five times the median depth spacing of `depths` (or infinity for a single depth).
- **continuity_tol_growth** (*float, default 0.5*) – Fractional growth of `max_slow_jump` per unit depth gap. Set to 0 for a gap-independent absolute tolerance.
- **continuity_tol_cap_factor** (*float, default 3.0*) – Hard cap on the effective tolerance, expressed as a multiple of `max_slow_jump`. Prevents runaway widening of the continuity window when a caller sets a large `continuity_max_gap`. The cap is disabled by passing `float("inf")`.
- **selection_rule** (*Literal['max_coherence', 'scored']*) – Passed through to `_best_candidate()`; see its docs.
- **time_penalty** (*float*) – Passed through to `_best_candidate()`; see its docs.
- **time_scale** (*float | None*) – Passed through to `_best_candidate()`; see its docs.

Return type
list of *DepthPicks*

`fwap.picker.`**`track_to_log_curves`**(*track*, **, modes=None, include_amplitude=True, include_coherence=True, include_vp_vs=True, include_time=False, null_value=nan*)

Convert a per-depth pick track into LAS/DLIS-ready log curves.

Bridges the picker output (*`track_modes()`*, *`viterbi_pick()`*, *`viterbi_pick_joint()`*) and the I/O writers (*`fwap.io.write_las()`*, *`fwap.io.write_dlis()`*) by building one fixed-length `(n_depth,)` array per (mode, attribute) pair, keyed by the standard fwap mnemonics. Slownesses are converted to **us/ft** (the borehole-acoustic unit used by the LAS/DLIS unit table); coherences and amplitudes are kept dimensionless / in their input units.

The Workflow-1 deliverable per Mari et al. (1994), Part 1 is a set of continuous Vp / Vs / Stoneley slowness curves with matching coherence (and amplitude) tracks. This function is the last mile: it produces the dict that `fwap.io.write_las(path, depth, curves)()` and `fwap.io.write_dlis(path, depth, curves)()` consume directly.

3.2.2 Mnemonic conventions

Canonical mode -> suffix mapping:
`P` -> `P`, `S` -> `S`, `Stoneley` -> `ST`, `PseudoRayleigh` -> `PR`. Modes outside this set use `mode_name.upper()` as the suffix.

Per mode, the columns produced are:

Mnemonic	Quantity	Unit
DT*	slowness	us/ft
COH*	coherence	(-)
AMP*	per-cell amplitude	(-)
TIM*	pick time	s

plus a single `VPVS` (= `s_S / s_P` = `Vp / Vs`) when both P and S are picked.

param track
Output of *`track_modes()`*, *`viterbi_pick()`*, *`viterbi_pick_joint()`*, or any other multi-depth picker.

type track
sequence of DepthPicks

param modes
Restrict output to these mode names. Defaults to every mode that appears anywhere in `track` (preserving first-seen order).

type modes
sequence of str, optional

param include_amplitude
Emit `AMP*` columns. Skipped per mode if no pick of that mode carries an amplitude.

type include_amplitude
bool, default True

param include_coherence
Emit `COH*` columns.

type include_coherence
bool, default True

param include_vp_vs
Emit a `VPVS` column when both `P` and `S` columns exist in the output.

type include_vp_vs
bool, default True

param include_time
Emit `TIM*` columns (pick time in seconds). Off by default because pick times are intermediate diagnostics rather than published log curves.

type include_time
bool, default False

param null_value
Fill value at depths where a mode was not picked. `NaN` is the LAS / DLIS native null marker; pass a numeric sentinel like `-999.25` if a downstream consumer requires that.

type null_value
float, default `NaN`

returns

- **depths** (*ndarray, shape (n_depth,)*) – Depth axis pulled from `DepthPicks.depth`, in the same unit the picker was called with (typically metres).

- **curves** (*dict[str, ndarray]*) – Mnemonic -> `(n_depth,)` array. All arrays are aligned on `depths`. Suitable to pass to `fwap.io.write_las()` or `fwap.io.write_dlis()` directly.

Examples

```
>>> from fwap import (
...     track_modes, track_to_log_curves, write_las,
... )
>>> track = track_modes(stc_results, depths)
>>> depths, curves = track_to_log_curves(track)
>>> write_las("output.las", depths, curves)
```

Parameters

- **track** (*Sequence[DepthPicks]*)
- **modes** (*Sequence[str] | None*)
- **include_amplitude** (*bool*)
- **include_coherence** (*bool*)
- **include_vp_vs** (*bool*)
- **include_time** (*bool*)
- **null_value** (*float*)

Return type
tuple[*ndarray*, dict[str, *ndarray*]]

`fwap.picker.`**`viterbi_pick`**(*stc_results*, *depths*, *priors=None*, *threshold=0.4*, *slow_jump_sigma=6.561679790026247e-05*, *time_order_slack=0.0*, *time_prior_weight=500.0*, *absence_cost=3.0*)

Joint-log-likelihood (Viterbi) mode picker across a depth sweep.

Replacement for the greedy `track_modes()` that addresses the two known failure modes of the per-depth rule-based pipeline:

- S-head-wave arriving before the formation P in altered zones, which the strict time-ordering rule of `pick_modes()` refuses to pick at all (see the note at `pick_modes()`).
- Missed picks inside noisy depth intervals propagating into the continuity tolerance of `track_modes()` and eventually letting a noise peak be reacquired far from the true mode.

For each mode, Viterbi over depths maximises

$$\sum_d \log \rho_d(s,t) \;-\; \sum_d \frac{(s_d - s_{d-1})^2}{2\sigma^2}$$

where ρ_d is the STC coherence at the candidate cell and σ is `slow_jump_sigma`. A "mode absent at this depth" state is available at cost `absence_cost` so long gaps do not force spurious picks.

Modes are processed in their `priors[*]["order"]` order (default P, S, Stoneley). Time ordering between modes is enforced on each depth: candidates for mode *k+1* must arrive no earlier than the Viterbi-picked time of mode *k* at the same depth, minus `time_order_slack` (positive slack permits the S-before-P case flagged above).

Parameters

- **stc_results** (*sequence of STCResult*) – One STC surface per depth.

- **depths** (*ndarray, shape (n_depth,)*) – Tool depth (m) for each STC surface.
- **priors** (*dict, optional*) – Per-mode prior windows. Defaults to `DEFAULT_PRIORS`.
- **threshold** (*float, default 0.4*) – Coherence floor passed through to *fwap.coherence.find_peaks()*.
- **slow_jump_sigma** (*float, default 20 us/ft*) – Gaussian scale (s/m) of the per-mode depth-to-depth slowness jump penalty.
- **time_order_slack** (*float, default 0.0*) – Allowed amount (s) that mode *k+1* may arrive before mode *k* at the same depth. Zero enforces strict ordering; a small positive value permits the S-before-P-in-altered-zones case.
- **time_prior_weight** (*float, default 500.0*) – Per-mode preference for earlier arrivals within each depth's candidate pool, in log-probability units per second. With the default a 1 ms later arrival is penalised by ~0.5 log-probability units, enough to break ties between nearby-coherence peaks (e.g. a P-window candidate at the P slowness vs one at the S slowness) but not so much that a strong late peak is overridden by a weak early one.
- **absence_cost** (*float, default 3.0*) – Cost (in log-probability units, i.e. ~``-log(coherence)``) of declaring a mode absent at a given depth. 3.0 is roughly equivalent to a minimum-coherence threshold of $e^{-3} \approx 0.05$.

Returns
One *DepthPicks* per depth. A mode absent at a given depth simply does not appear in the `picks` dict.

Return type
list of *DepthPicks*

Notes

The current implementation runs a separate Viterbi pass per mode, with the previous mode's time-path fed into the emission score of the next mode (sequential-within-depth, joint-across-depth). A fully joint picker over all three modes would be possible but would blow up the state space cubically for no meaningful accuracy gain on typical sonic data.

References

- Viterbi, A. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory* 13(2), 260-269.
- Aron, J., et al. (1994). Real-time sonic logging while drilling (flagged S-before-P altered-zone case).

`fwap.picker.`**`viterbi_pick_joint`**(*stc_results, depths, priors=None, threshold=0.4, slow_jump_sigma=6.561679790026247e-05, time_order_slack=0.0, time_prior_weight=500.0, absence_cost=3.0, top_k_per_mode=None, soft_time_order=None, max_triples_per_depth=2000*)

Fully-joint 3-mode Viterbi picker.

State at each depth is a triple `(c_P, c_S, c_St)` of per-mode candidate indices (with an "absent" option per mode), subject to the within-depth time-ordering constraint `t_P <= t_S <= t_Stoneley` (strict by default, soft if `soft_time_order` is set). Viterbi DP runs over `(depth, triple)`; the result is the globally optimal per-mode path across the full sweep.

The trellis is hardcoded for the (P, S, Stoneley) triple; the pseudo-Rayleigh / guided mode in `DEFAULT_PRIORS` is *not* handled here, because adding a fourth mode squares the trellis width. Use *track_modes()* or *viterbi_pick()* for full 4-mode picking; both run per-mode Viterbi and scale linearly in the number of modes.

3.2.3 Differences vs *viterbi_pick()*

viterbi_pick() runs Viterbi on each mode independently and feeds the best path's picked time as a soft constraint into the next mode. Joint Viterbi optimises over the triple as a single unit, so coupling effects – e.g. a depth where the best-by- coherence P pick would force an impossible S in the next depth, or where S-before-P in an altered zone is the jointly optimal answer – are handled exactly instead of through a sequential relaxation. On clean data the two produce identical picks; the difference appears on noisy or altered-zone intervals.

3.2.4 Cost

Per-depth triple enumeration is `(n_P + 1) * (n_S + 1) * (n_St + 1)` before ordering filter, typically reducing to ~500-1000 valid triples (fewer with `top_k_per_mode` set). Transition cost for the trellis at each depth step is `n_prev * n_curr` – bounded by `max_triples_per_depth`. On realistic 30-depth sweeps with ~15 peaks per mode, total runtime is under half a second.

3.2.5 Complexity

Time is `O(n_depth * T^2)` where `T` is the per-depth triple count; memory is `O(n_depth * T)`. `T` grows as the *product* of per-mode candidate counts before the time-ordering filter, so very peaky STC surfaces can blow up the trellis quickly. Use `top_k_per_mode` (typical: 5-10) to cap that product, or raise the coherence `threshold` to thin the candidate pool; the safety check at `max_triples_per_depth` stops a runaway expansion with a descriptive error instead of an out-of-memory crash.

param stc_results
type stc_results
sequence of STCResult

param depths
type depths
ndarray, shape (n_depth,)

param priors
Per-mode prior windows. Defaults to `DEFAULT_PRIORS`.

type priors
dict, optional

param threshold
param slow_jump_sigma
param time_order_slack
param time_prior_weight

:param : :param absence_cost: See *viterbi_pick()*; same semantics. :param top_k_per_mode: If set, keep only the K most-coherent candidates per mode

per depth before triple enumeration. Bounds the trellis size (and runtime) in the presence of very peaky STC surfaces without hitting `max_triples_per_depth`. `None` (default) keeps every candidate that passed the prior window + coherence-threshold filter.

type top_k_per_mode
int, optional

param soft_time_order
If set to a positive value `lambda`, the strict `t_P <= t_S <= t_Stoneley` constraint is replaced with a soft penalty `lambda * violation_magnitude` added to the emission. Useful in altered zones where S legitimately arrives before P and the strict constraint would kill the entire triple. `None` (default) keeps the hard constraint.

type soft_time_order
float, optional

param max_triples_per_depth
Safety cap on the per-depth triple count. Hitting this limit raises `ValueError`; set `top_k_per_mode` or tighten the coherence `threshold` / prior windows.

type max_triples_per_depth
int, default 2000

rtype
list of DepthPicks

References

- Viterbi, A. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory* 13(2), 260-269.

Parameters

- **stc_results** (*Sequence[STCResult]*)
- **depths** (*ndarray*)
- **priors** (*dict[str, dict[str, float]] | None*)
- **threshold** (*float*)
- **slow_jump_sigma** (*float*)
- **time_order_slack** (*float*)
- **time_prior_weight** (*float*)
- **absence_cost** (*float*)
- **top_k_per_mode** (*int | None*)
- **soft_time_order** (*float | None*)
- **max_triples_per_depth** (*int*)

Return type
list[DepthPicks]

`fwap.picker.`**`viterbi_posterior_marginals`**(*stc_results*, *depths*, *priors=None*, *threshold=0.4*, *slow_jump_sigma=6.561679790026247e-05*, *time_order_slack=0.0*, *time_prior_weight=500.0*, *absence_cost=3.0*, *top_k_per_mode=None*, *soft_time_order=None*, *max_triples_per_depth=2000*)

Joint 3-mode forward-backward: MAP picks plus per-mode posterior marginals.

Runs exactly the same trellis as `viterbi_pick_joint()`, but in addition to the max-sum forward pass it also computes the log-sum-exp forward and backward messages. Marginalising the posterior over the (depth, triple) lattice yields, at every depth, the probability that each mode is picked at each of its candidate slownesses – plus the probability that the mode is absent.

Useful for:

- **Uncertainty quantification**: the MAP pick's slowness is accompanied by a distribution, not just a coherence value.

- **Ambiguous picks**: if two candidates have similar posterior probability, the MAP answer is not the whole story; a caller can flag such depths for manual QC.
- **Absence-probability mask**: `1 - p_absent` is a cleaner mask than the raw MAP coherence when used to weight downstream products.

Parameters

- **stc_results** (*Sequence[STCResult]*)
- **depths** (*ndarray*)
- **priors** (*dict[str, dict[str, float]] | None*)
- **threshold** (*float*)
- **slow_jump_sigma** (*float*)
- **time_order_slack** (*float*)
- **time_prior_weight** (*float*)
- **absence_cost** (*float*)
- **top_k_per_mode** (*int | None*)
- **soft_time_order** (*float | None*)
- **max_triples_per_depth** (*int*)

Return type

tuple[list[DepthPicks], list[dict[str, PosteriorPick]]]

:param : :param time_order_slack: :param time_prior_weight: :param absence_cost: :param : :param top_k_per_mode: Identical to `viterbi_pick_joint()`. :param soft_time_order: Identical to `viterbi_pick_joint()`. :param max_triples_per_depth: Identical to `viterbi_pick_joint()`.

Returns

- **map_picks** (*list of DepthPicks*) – Same as `viterbi_pick_joint()` on this input (the MAP / Viterbi path through the trellis).
- **posteriors** (*list of dict[str, PosteriorPick]*) – One dict per depth, keyed by mode name. Each entry carries the per-candidate posterior probability vector and the probability that the mode is absent at that depth.

Parameters

- **stc_results** (*Sequence[STCResult]*)
- **depths** (*ndarray*)
- **priors** (*dict[str, dict[str, float]] | None*)
- **threshold** (*float*)
- **slow_jump_sigma** (*float*)
- **time_order_slack** (*float*)
- **time_prior_weight** (*float*)
- **absence_cost** (*float*)
- **top_k_per_mode** (*int | None*)
- **soft_time_order** (*float | None*)

- **max_triples_per_depth** (*int*)

Return type
tuple[list[DepthPicks], list[dict[str, PosteriorPick]]]

References

- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE* 77(2), 257-286 (Algorithm 2, forward-backward).

`fwap.picker.`**`wavelet_shape_score`**(*stack*, *dt*, *f0*)

Absolute Pearson correlation of a stacked window vs a Ricker(f0).

Returns a score in `[0, 1]`: `1.0` for a stacked window that is exactly a Ricker at the prior centre frequency `f0` (modulo amplitude and time shift), `0.0` for an uncorrelated waveform. The Ricker template is centred on the location of `stack`'s largest-absolute sample so the correlation is invariant to sub-window jitter of the picked arrival time.

The score is **polarity-blind** – it returns the absolute correlation – so it can be combined orthogonally with *onset_polarity()* for a separate sign check.

Parameters

- **stack** (*ndarray*)
- **dt** (*float*)
- **f0** (*float*)

Return type
float

3.3 fwap.wavesep

Wave separation – f-k, tau-p, and SVD / Karhunen-Loeve projection.

Implements the multichannel velocity-filter family described in Part 2 of the book ("Wave separation in acoustic well logging"). Three complementary tools are provided:

- *fk_filter()* – frequency-wavenumber pass-band
- *tau_p_forward()*, *tau_p_inverse()*, *tau_p_filter()* – linear Radon (slant-stack)

  pass-band
- *svd_project()*, *sequential_kl_separation()* – moveout-corrected SVD / K-L

The book lists *both* f-k and tau-p as the textbook domains for multichannel velocity filtering ("working in the frequency-wavenumber (f-k) or tau-p domain"). f-k is unitary and exact-round-trip but requires uniform offset spacing; tau-p accepts arbitrary offsets and keeps a sharper localisation of linear-moveout events at the cost of needing the rho filter (**|f|** in the temporal frequency domain) for an accurate inverse.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 2. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Mari, J.-L., & Glangeaud, F. (1990). *Wave Separation.* Editions Technip, Paris.
- Freire, S. L. M., & Ulrych, T. J. (1988). Application of singular value decomposition to vertical seismic profiling. *Geophysics* 53(6), 778-785.

- Yilmaz, O. (2001). *Seismic Data Processing*, 2nd ed., Section 1.2 (f-k domain conventions) and Section 6.3 (tau-p / linear Radon). SEG, Tulsa.
- Beylkin, G. (1987). Discrete radon transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 35(2), 162-172 (rho-filter inverse).

Functions

`apply_moveout`(data, dt, offsets, slowness[, ...])	Flatten a gather on a trial slowness using frequency-domain fractional shifts.
`fk_filter`(data, dt, dx, slow_min, slow_max)	f-k (apparent-slowness) bandpass filter.
`fk_forward`(data, dt, dx)	Forward 2-D FFT of a regularly-sampled gather into the f-k domain.
`fk_inverse`(spec, n_samples)	Inverse 2-D FFT back to the time-offset domain.
`sequential_kl_separation`(data, dt, offsets, ...)	Peel off coherent modes one slowness at a time.
`svd_project`(data, dt, offsets, slowness[, ...])	SVD / Karhunen-Loeve separation.
`tau_p_adjoint`(panel, dt, offsets, slownesses)	Adjoint of `tau_p_forward()` (smeared back-projection).
`tau_p_filter`(data, dt, offsets, slow_min, ...)	Apparent-slowness band-pass filter via tau-p forward / mask / inverse.
`tau_p_forward`(data, dt, offsets, slownesses)	Forward tau-p (slant-stack / linear Radon) transform.
`tau_p_inverse`(panel, dt, offsets, slownesses, *)	Inverse tau-p (slant-stack / linear Radon) transform.
`unapply_moveout`(data, dt, offsets, slowness)	Inverse of `apply_moveout` -- restore the original move-out.

`fwap.wavesep.`**`apply_moveout`**(*data*, *dt*, *offsets*, *slowness*, *reference=0*)

Flatten a gather on a trial slowness using frequency-domain fractional shifts.

3.3.1 Sign convention

Trace `i` carries a mode with arrival at `t_i = offset_i * s` (plus a constant). To flatten the gather at the reference offset we need to *advance* trace `i` by `(offset_i - offset_ref) * s` (earlier-arriving traces are left alone; later-arriving traces are shifted earlier). In FFT convention $X(f) = \sum x(n) e^{-2\pi i f n}$, a time advance by `tau` corresponds to multiplication by $e^{+2\pi i f \tau}$ (see `fwap._common._phase_shift()`).

Parameters

- **`data`** (*`ndarray`*)
- **`dt`** (*`float`*)
- **`offsets`** (*`ndarray`*)
- **`slowness`** (*`float`*)
- **`reference`** (*`int`*)

Return type

ndarray

`fwap.wavesep.`**`fk_filter`**(*data*, *dt*, *dx*, *slow_min*, *slow_max*, *positive_only=True*, *taper_width=0.1*)

f-k (apparent-slowness) bandpass filter.

Keeps only energy whose apparent slowness `s = -k/f` lies in `[slow_min, slow_max]`. When `positive_only=False` the same passband in `|s|` is kept on both sides of the `k` axis (up-going and down-going), with the cosine taper applied symmetrically.

3.3.2 Sign convention

Under the NumPy FFT convention `X(f) = sum_n x(n) e^{-2 pi i f n}` a forward-propagating wave of physical slowness `s > 0` – that is, a source record `x(t, i) = src(t - i * dx * s)` – produces energy at wavenumber `k = -f * s`. The physical apparent slowness recovered from the f-k plane is therefore `S = -k / f`. See Yilmaz (2001), *Seismic Data Processing*, Section 1.2.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **dx** (*float*)
- **slow_min** (*float*)
- **slow_max** (*float*)
- **positive_only** (*bool*)
- **taper_width** (*float*)

Return type
ndarray

`fwap.wavesep.`**`fk_forward`**(*data*, *dt*, *dx*)

Forward 2-D FFT of a regularly-sampled gather into the f-k domain.

Parameters

- **data** (*ndarray, shape (n_rec, n_samples)*) – Real-valued gather, receivers uniformly spaced by `dx`.
- **dt** (*float*) – Time sampling interval (s).
- **dx** (*float*) – Receiver spacing (m).

Returns

- **spec** (*ndarray, shape (n_rec, n_samples // 2 + 1), complex*) – 2-D spectrum. Axis 0 is wavenumber (`k`), fft-shifted so that `k` increases monotonically from most negative to most positive. Axis 1 is temporal frequency (`f`), non-negative rFFT layout with `f[0] = 0` and `f[-1] = Nyquist`.
- **f** (*ndarray, shape (n_samples // 2 + 1,)*) – Frequency axis (Hz).
- **k** (*ndarray, shape (n_rec,)*) – Wavenumber axis (1/m), fft-shifted to match `spec`.

Return type
tuple[*ndarray*, *ndarray*, *ndarray*]

`fwap.wavesep.`**`fk_inverse`**(*spec*, *n_samples*)

Inverse 2-D FFT back to the time-offset domain.

The inverse of *`fk_forward()`*: consumes a spectrum in the fft-shifted (wavenumber, rFFT-frequency) layout and returns the real-valued time-offset gather.

Parameters

- **spec** (*ndarray, shape (n_rec, n_samples // 2 + 1), complex*) – Spectrum in the layout returned by *`fk_forward()`*.
- **n_samples** (*int*) – Original number of time samples; required because the rFFT output does not by itself distinguish an even- from an odd- length input.

Return type
ndarray, shape (*n_rec*, *n_samples*)

`fwap.wavesep.`**`sequential_kl_separation`**(*data*, *dt*, *offsets*, *slownesses*, *rank=1*, *reference=0*, *, *n_keep=None*)

Peel off coherent modes one slowness at a time.

`rank` is forwarded to `svd_project()`; `n_keep` is the deprecated alias kept for backward compatibility.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **slownesses** (*Sequence[float]*)
- **rank** (*int*)
- **reference** (*int*)
- **n_keep** (*int | None*)

Return type
tuple[list[*ndarray*], *ndarray*]

`fwap.wavesep.`**`svd_project`**(*data*, *dt*, *offsets*, *slowness*, *rank=1*, *reference=0*, *, *n_keep=None*)

SVD / Karhunen-Loeve separation.

Keeps the rank-`rank` coherent part of `data` that travels at the given apparent slowness; returns `(coherent, residual)`.

The `rank` parameter is the number of singular components kept (the "K" in K-L decomposition); for borehole sonic data a single component captures the dominant coherent arrival.

`n_keep` is accepted as a deprecated alias for `rank` to keep older call sites working. Pass it as a keyword if you must; `rank` takes precedence when both are supplied.

Because `apply_moveout` now correctly flattens the gather, the largest singular vector of the moveout-corrected matrix genuinely represents the coherent arrival at that slowness (Freire & Ulrych, 1988, *Geophysics* 53(6), 778-785).

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **slowness** (*float*)
- **rank** (*int*)
- **reference** (*int*)
- **n_keep** (*int | None*)

Return type
tuple[*ndarray*, *ndarray*]

`fwap.wavesep.`**`tau_p_adjoint`**(*panel*, *dt*, *offsets*, *slownesses*)

Adjoint of `tau_p_forward()` (smeared back-projection).

Sums each slowness slice of the panel back along its corresponding moveout line in (t, x):

```
\hat d(t, x_i) = \frac{1}{N_{slowness}} \sum_p
                 panel(t - p \cdot x_i, p)
```

The $1/N_{slowness}$ factor is a Riemann-sum approximation of a slowness integral; it keeps the adjoint output's order of magnitude comparable to the input data rather than scaling with the slowness-grid density.

The adjoint is *not* the inverse of `tau_p_forward()` – Radon is non-unitary, and adjoint-of-forward smears each event by the rectangular-aperture sinc kernel. Use it as the back-end of a slowness-band filter (`tau_p_filter()`), where the smearing is acceptable; for an exact round-trip use `tau_p_inverse()` instead.

Parameters

- **panel** (*ndarray, shape (n_slowness, n_samples)*)
- **dt** (*float*)
- **offsets** (*ndarray, shape (n_rec,)*)
- **slownesses** (*ndarray, shape (n_slowness,)*)

Return type
ndarray, shape (*n_rec*, *n_samples*)

`fwap.wavesep.`**`tau_p_filter`**(*data*, *dt*, *offsets*, *slow_min*, *slow_max*, **, n_slowness=181*, *slowness_pad_factor=1.5*, *taper_width=0.1*)

Apparent-slowness band-pass filter via tau-p forward / mask / inverse.

Mirror of `fk_filter()` in the linear-Radon domain: the multichannel record is forward-transformed to (tau, p), a cosine-tapered mask keeps slownesses in `[slow_min, slow_max]` (with a wider grid around it for clean edge behaviour), and `tau_p_adjoint()` projects back to (t, x).

3.3.3 Differences vs `fk_filter()`

- tau-p does **not** require uniform offsets; `fk_filter` does.
- tau-p uses the adjoint as the back-end (the LSQR-inverse from `tau_p_inverse()` does not commute with masking, so using it here would amplify the masked panel rather than band-pass it). Out-of-band rejection is set by the slowness- grid aperture and the cosine taper rather than by a strict orthogonality, and amplitudes are smeared by the slant-stack point-spread function (event amplitude is preserved up to a modest aperture-dependent constant, *not* exactly).

param data
Real-valued gather.

type data
ndarray, shape (n_rec, n_samples)

param dt
Time sampling interval (s).

type dt
float

param offsets
Source-to-receiver offsets (m). Need not be uniform.

type offsets
ndarray, shape (n_rec,)

param slow_min
Pass-band edges in apparent slowness (s/m). Must satisfy `0 < slow_min < slow_max`.

type slow_min
float

param slow_max
Pass-band edges in apparent slowness (s/m). Must satisfy `0 < slow_min < slow_max`.

type slow_max
float

param n_slowness
Number of slowness samples in the tau-p grid.

type n_slowness
int, default 181

param slowness_pad_factor
Extra slowness coverage outside the pass-band, expressed as a multiple of the pass-band width. The slowness grid spans `[slow_min - pad, slow_max + pad]` with `pad = slowness_pad_factor * (slow_max - slow_min)`, clipped at zero on the lower side. Wider padding gives a better-conditioned linear Radon frame at the cost of a denser slowness grid.

type slowness_pad_factor
float, default 1.5

param taper_width
Width of the half-cosine ramp on each pass-band edge, expressed as a fraction of the pass-band width. `0.0` gives a hard rectangular mask.

type taper_width
float, default 0.1

rtype
ndarray, shape (n_rec, n_samples)

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **slow_min** (*float*)
- **slow_max** (*float*)
- **n_slowness** (*int*)
- **slowness_pad_factor** (*float*)
- **taper_width** (*float*)

Return type
ndarray

`fwap.wavesep.`**`tau_p_forward`**(*data*, *dt*, *offsets*, *slownesses*)

Forward tau-p (slant-stack / linear Radon) transform.

For each candidate apparent slowness p, sums the trace amplitudes along the linear-moveout line $t = \tau + p \cdot x_i$ (with $x_i = \text{offsets}[i] - \text{offsets}[0]$):

```
panel(\tau, p) = \sum_i d(\tau + p \cdot x_i, x_i)
```

Implemented in the frequency domain via per-trace fractional time shifts of `fwap._common._phase_shift()`, which keeps the transform exact for non-integer `p * dx / dt` and avoids the sample-snapping error of a naive time-domain stack.

Unlike *fk_forward()*, the offsets need *not* be uniformly spaced.

Parameters

- **data** (*ndarray, shape (n_rec, n_samples)*) – Real-valued gather.
- **dt** (*float*) – Time sampling interval (s).
- **offsets** (*ndarray, shape (n_rec,)*) – Source-to-receiver offsets (m). Need not be uniform; the slant-stack uses `offsets - offsets[0]` as the lever arm.
- **slownesses** (*ndarray, shape (n_slowness,)*) – Apparent-slowness grid (s/m). For wave separation pick `n_slowness` >= 2 * n_rec to keep the linear Radon frame well conditioned, and span a band centred on the moveout you want to isolate.

Returns

`panel[k, j]` is the slant-stacked amplitude at slowness `slownesses[k]` and intercept time `j * dt`.

Return type

ndarray, shape (n_slowness, *n_samples*)

`fwap.wavesep.`**`tau_p_inverse`**(*panel*, *dt*, *offsets*, *slownesses*, *, *ridge=1e-10*)

Inverse tau-p (slant-stack / linear Radon) transform.

Solves the per-frequency least-squares problem

$$\hat{d}(f,\cdot) = \arg\min_{d'} \|A_f\, d' - panel(f,\cdot)\|_2^2$$

where $A_f[p,i] = \exp(2\pi i f\, s_p\, x_i)$ is the forward stacking matrix at frequency f. Because each temporal frequency is independent in tau-p, the inverse problem decouples into `n_f` small linear systems of size `n_slowness x n_rec`, each solved exactly via the Moore-Penrose pseudoinverse. With `n_slowness > n_rec` (the standard over-determined case for a sonic array) and the panel coming from *tau_p_forward()* the round-trip is identity to floating-point precision.

Parameters

- **panel** (*ndarray, shape (n_slowness, n_samples)*) – tau-p panel from *tau_p_forward()* (or any same-shape array, e.g. a slowness-band-masked panel for use as a band-pass filter back-end).
- **dt** (*float*) – Time sampling interval (s).
- **offsets** (*ndarray, shape (n_rec,)*) – Source-to-receiver offsets (m). Must match the offsets used in the forward transform.
- **slownesses** (*ndarray, shape (n_slowness,)*) – Slowness grid used in the forward transform.
- **ridge** (*float, default 1e-10*) – Tikhonov regularisation added to the per-frequency normal- equation diagonal. Prevents catastrophic blow-up when the slowness grid is too coarse to resolve all `n_rec` traces at a given frequency (which happens at very low frequencies where the rows of A_f are nearly identical). Default is small enough to be invisible on well-conditioned cases but large enough to keep low-f bins finite.

Returns

Reconstructed gather.

Return type
ndarray, shape (*n_rec*, *n_samples*)

`fwap.wavesep.`**`unapply_moveout`**(*data*, *dt*, *offsets*, *slowness*, *reference=0*)

Inverse of `apply_moveout` – restore the original moveout.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **slowness** (*float*)
- **reference** (*int*)

Return type
ndarray

3.4 fwap.tomography

Intercept-time (delay) inversion for borehole acoustic logs.

Implements the algorithmic spine of Part 3 of the book: the borehole adaptation of refraction-seismic intercept-time analysis. The least- squares system jointly solves for a "virgin" formation slowness log and per-depth source/receiver delays, giving access to the near-borehole altered zone.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 3. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Coppens, F., & Mari, J.-L. (1995). Application of the intercept time method to full waveform acoustic data. *First Break* 13(1), 11-20.
- Coppens, F., & Mari, J.-L. (1995). Imagerie par refraction en diagraphie acoustique. *Revue de l'Institut Francais du Petrole* 50(2), 143.
- Tarantola, A. (2005). *Inverse Problem Theory and Methods for Model Parameter Estimation*, Chapter 3. SIAM.
- Aster, R. C., Borchers, B., & Thurber, C. H. (2018). *Parameter Estimation and Inverse Problems*, 3rd ed., Chapter 4. Elsevier.

Functions

altered_zone_estimate(delay, slowness_virgin, *)	Joint `(thickness, velocity-contrast)` altered-zone estimate.
assemble_observations_from_picks(...)	Convert an `(n_depth, n_rec)` first-arrival pick matrix into the ragged observation form consumed by *solve_intercept_time()* `(method="midpoint")`.
build_design_matrix(travel_times, offsets, ...)	Midpoint design matrix.
build_design_matrix_segmented(travel_times, ...)	Segmented (1-D tomography) design matrix.
delay_to_altered_zone_thickness(delay, ...)	Convert an intercept-time delay to an altered-zone thickness.

continues on next page

Table 7 – continued from previous page

`delay_to_altered_zone_velocity_contrast`(...)	Convert an intercept-time delay to an altered-zone slowness contrast.
`solve_intercept_time`(travel_times, offsets, ...)	Least-squares intercept-time inversion with regularisation and posterior standard errors.

Classes

`AlteredZoneEstimate`(thickness, ...)	Joint altered-zone characterisation produced by `altered_zone_estimate()`.
`InterceptTimeResult`(depths, slowness, ...[, ...])	Output of `solve_intercept_time()`.

class `fwap.tomography.`**`AlteredZoneEstimate`**(*thickness*, *slowness_altered*, *slowness_contrast*)

Bases: `object`

Joint altered-zone characterisation produced by `altered_zone_estimate()`.

Each attribute is a per-depth array (or a scalar if the input delay was scalar). The triple is internally consistent under $\Delta t = 2h\Delta s$ at every depth.

Parameters

- **thickness** (*ndarray*)
- **slowness_altered** (*ndarray*)
- **slowness_contrast** (*ndarray*)

thickness

Altered-zone thickness (m).

Type
ndarray

slowness_altered

Absolute altered-zone slowness (s/m).

Type
ndarray

slowness_contrast

Altered-zone minus virgin slowness (s/m).

Type
ndarray

slowness_altered: ndarray

slowness_contrast: ndarray

thickness: ndarray

class `fwap.tomography.`**`InterceptTimeResult`**(*depths*, *slowness*, *delay_src*, *delay_rec*, *rms_residual*, *sigma_slowness*, *sigma_delay_src*, *sigma_delay_rec*, *method='midpoint'*)

Bases: `object`

Output of `solve_intercept_time()`.

Parameters

- **depths** (*ndarray*)
- **slowness** (*ndarray*)
- **delay_src** (*ndarray*)
- **delay_rec** (*ndarray*)
- **rms_residual** (*float*)
- **sigma_slowness** (*ndarray*)
- **sigma_delay_src** (*ndarray*)
- **sigma_delay_rec** (*ndarray*)
- **method** (*Literal['midpoint', 'segmented']*)

depths, slowness, delay_src, delay_rec

Depth axis and the three unknown columns.

rms_residual

RMS of `G @ m - d` on the observation rows only (i.e. not counting regularisation rows).

Type
float

sigma_slowness, sigma_delay_src, sigma_delay_rec

Marginal 1-sigma posterior standard errors, computed from `sigma_d**2 * diag((G_aug.T @ G_aug)^-1)`, where `sigma_d = rms_residual`. A principled Bayesian treatment would carry independent prior variances and a full covariance (see Tarantola, 2005, sect. 3.2, or Aster, Borchers & Thurber, 2018, chap. 4).

method

Design matrix style used (`"midpoint"` or `"segmented"`).

Type
Literal['midpoint', 'segmented']

delay_rec: ndarray

delay_src: ndarray

depths: ndarray

method: Literal['midpoint', 'segmented'] = 'midpoint'

rms_residual: float

sigma_delay_rec: ndarray

sigma_delay_src: ndarray

sigma_slowness: ndarray

slowness: ndarray

`fwap.tomography.altered_zone_estimate`(*delay*, *slowness_virgin*, *, *thickness=None*, *slowness_altered=None*)

Joint `(thickness, velocity-contrast)` altered-zone estimate.

The Workflow-2 deliverable per Mari et al. (1994), Part 3 is the altered-zone *thickness* and *velocity contrast* together; the refraction-geometry relation $\Delta t = 2h\,(s_{\text{altered}} - s_{\text{virgin}})$ is one equation in two unknowns, so to deliver the pair the caller must pin one of them and the routine derives the other.

Pass exactly one of `thickness` (a scalar or per-depth array altered-zone thickness in metres) or `slowness_altered` (a scalar absolute altered-zone slowness in s/m).

Parameters

- **delay** (*ndarray or float*) – Per-depth delay time (s) – typically *InterceptTimeResult.delay_src* or *InterceptTimeResult.delay_rec*.
- **slowness_virgin** (*float*) – Undisturbed-formation slowness (s/m).
- **thickness** (*float or ndarray, optional*) – Anchor: assumed altered-zone thickness (m).
- **slowness_altered** (*float, optional*) – Anchor: assumed absolute altered-zone slowness (s/m). Must be strictly greater than `slowness_virgin`.

Returns

`thickness`, `slowness_altered` and `slowness_contrast` per depth; consistent under `delay = 2 * thickness * slowness_contrast`.

Return type

AlteredZoneEstimate

Raises

ValueError – If both anchors or neither anchor is supplied; if the supplied `thickness` has any non-positive entry; or if `slowness_altered <= slowness_virgin`.

`fwap.tomography.assemble_observations_from_picks`(*tool_depths*, *offsets*, *first_arrivals*)

Convert an `(n_depth, n_rec)` first-arrival pick matrix into the ragged observation form consumed by *solve_intercept_time()* (`method="midpoint"`).

Observations whose mapped receiver cell falls outside the depth grid are dropped rather than clipped; this keeps the assembled design matrix well-conditioned at the edges of the logged interval.

Parameters

- **tool_depths** (*ndarray, shape (n_depth,)*) – Tool (source) depth for each shot (m). Must have at least two entries and be approximately uniformly spaced – `dz` is taken as the mean of `np.diff(tool_depths)`.
- **offsets** (*ndarray, shape (n_rec,)*) – Source-to-receiver offsets (m), positive downhole.
- **first_arrivals** (*ndarray, shape (n_depth, n_rec)*) – First-break travel times (s), one row per tool depth.

Returns

- **travel_times** (*ndarray, shape (n_obs,)*) – Selected first-arrival picks.
- **offsets_out** (*ndarray, shape (n_obs,)*) – Offset for each observation.
- **src_idx, rec_idx** (*ndarray, shape (n_obs,) of int*) – Indices into `depth_axis` for the source and receiver endpoints of each observation; `rec_idx` is computed from `round((tool_depth + offset - z0) / dz)`.
- **n_depth** (*int*)

- **depth_axis** (*ndarray, shape (n_depth,)*) – Copy of `tool_depths`, returned as the cell-centre axis.

Return type
tuple[*ndarray*, *ndarray*, *ndarray*, *ndarray*, int, *ndarray*]

`fwap.tomography.`**`build_design_matrix`**(*travel_times*, *offsets*, *src_depth_idx*, *rec_depth_idx*, *n_depth*)

Midpoint design matrix.

Unknown vector `m = [ S(z_0..N-1) | d_src(z_0..N-1) | d_rec(z_0..N-1) ]`. Each observation row assigns the full offset length to the single midpoint cell between source and receiver, and picks one delay from the source and one from the receiver block.

This is fast and requires only the pick indices, but biases the slowness estimate when offsets are not small compared to the cell thickness `dz`; see *`build_design_matrix_segmented()`* for the ray-path form.

Parameters

- **`travel_times`** (*`ndarray, shape (n_obs,)`*) – Picked travel times (s).
- **`offsets`** (*`ndarray, shape (n_obs,)`*) – Source-to-receiver offset (m) for each observation.
- **`src_depth_idx`** (*`ndarray, shape (n_obs,)`*) – Integer indices into the depth cell grid for the source and receiver endpoints.
- **`rec_depth_idx`** (*`ndarray, shape (n_obs,)`*) – Integer indices into the depth cell grid for the source and receiver endpoints.
- **`n_depth`** (*`int`*) – Number of depth cells.

Returns

- **G** (*ndarray, shape (n_obs, 3 * n_depth)*)
- **d** (*ndarray, shape (n_obs,)*) – Copy of `travel_times`.

Return type
tuple[*ndarray*, *ndarray*]

`fwap.tomography.`**`build_design_matrix_segmented`**(*travel_times*, *src_depths*, *rec_depths*, *cell_depths*, ***, *chunk_size=None*)

Segmented (1-D tomography) design matrix.

Unknown vector layout (identical to *`build_design_matrix()`*):

```
m = [ S(z_0..N-1) | d_src(z_0..N-1) | d_rec(z_0..N-1) ]
```

For each observation the ray path from `src_depths[i]` to `rec_depths[i]` is split across every depth cell it traverses; each cell contributes `s_cell * path_length`. Source and receiver delay columns are indicator columns at the nearest cell to each endpoint.

This is the correct form when offsets are not small compared to the cell thickness `dz`. Source and receiver delays are still picked at the nearest cell (the delay stack is separate from the slowness stack – cf. Coppens & Mari, 1995, *First Break* 13(1), 14-16).

Parameters

- **`travel_times`** (*`ndarray, shape (n_obs,)`*) – Picked travel-times (s).
- **`src_depths`** (*`ndarray, shape (n_obs,)`*) – Source and receiver depths (m). Ordering within a pair is irrelevant; the shallower endpoint is taken as the start of the ray.

- **rec_depths** (*ndarray, shape (n_obs,)*) – Source and receiver depths (m). Ordering within a pair is irrelevant; the shallower endpoint is taken as the start of the ray.
- **cell_depths** (*ndarray, shape (n_cells,)*) – Cell centres (m). Must be strictly increasing and uniformly spaced to within 1e-6 relative tolerance on the first difference.
- **chunk_size** (*int, optional*) – Number of observations processed per broadcast chunk. Defaults to a cache-friendly size chosen from `_CHUNK_TARGET_BYTES` and `n_cells`.

Returns

- **G** (*ndarray, shape (n_obs, 3 * n_cells)*)
- **d** (*ndarray, shape (n_obs,)*) – Copy of `travel_times`.

Raises

ValueError – If any input has an incompatible shape, if the cell grid is not strictly increasing, or if the grid is not uniformly spaced.

Return type

tuple[*ndarray*, *ndarray*]

Notes

- A depth exactly on a cell boundary is assigned to the **higher** of the two adjacent cells.
- Rays whose endpoints fall outside the cell grid contribute zero to the slowness block for the out-of-grid portion; their delay indicators are clipped to the nearest in-grid cell.

`fwap.tomography.`**delay_to_altered_zone_thickness**(*delay*, *slowness_virgin*, *slowness_altered*)

Convert an intercept-time delay to an altered-zone thickness.

First-order borehole refraction geometry: the ray leaves the source, refracts into the damaged zone at slowness `s_altered`, travels along the borehole wall, and refracts back out at the receiver. Relative to the bulk-formation (`s_virgin`) reference, the extra traveltime spent inside the altered layer is

$$\Delta t = 2\,h\,(s_{\text{altered}} - s_{\text{virgin}})$$

for a single-sided altered-zone thickness `h` (metres). Solving for `h`:

$$h = \frac{\Delta t}{2\,(s_{\text{altered}} - s_{\text{virgin}})}$$

This is the standard first-order conversion behind the "damaged- halo thickness" log described in Coppens & Mari (1995), section 3. It assumes (a) the altered zone is thin compared to the source- receiver offset, (b) the altered-zone slowness is well-estimated from an ancillary log (e.g. a shallow-reading resistivity interpretation or an assumed fractional velocity reduction), and (c) the delay column is one-sided (either `delay_src` or `delay_rec` from *solve_intercept_time()*, not their sum).

3.4.1 Single-delay under-determination

Each per-depth delay Δt is a single equation in two unknowns (h, s_{altered}). This routine pins s_{altered} and solves for h; the dual direction – pin h, solve for the slowness contrast – is *delay_to_altered_zone_velocity_contrast()*. The book (Mari et al. 1994, Part 3) frames the workflow output as the full `(thickness, velocity-contrast)` pair; use *altered_zone_estimate()* to obtain both at once given either anchor.

param delay

Per-depth delay time (s), typically *InterceptTimeResult.delay_src* or *InterceptTimeResult.delay_rec*.

type delay
ndarray or float

param slowness_virgin
Undisturbed-formation slowness (s/m).

type slowness_virgin
float

param slowness_altered
Damaged-zone slowness (s/m). Must be strictly greater than `slowness_virgin` (a damaged zone is slower, by definition).

type slowness_altered
float

returns
Altered-zone thickness (m), same shape as `delay`. Values are clipped to be non-negative – negative delays (which can arise from inversion noise in the virgin zones) map to a zero thickness.

rtype
ndarray or float

raises ValueError
If `slowness_altered <= slowness_virgin`.

References

Coppens, F., & Mari, J.-L. (1995). Application of the intercept time method to full waveform acoustic data. *First Break* 13(1), 11-20 (section 3: altered-zone characterisation).

Parameters

- **delay** (*ndarray*)
- **slowness_virgin** (*float*)
- **slowness_altered** (*float*)

Return type
ndarray

`fwap.tomography.`**`delay_to_altered_zone_velocity_contrast`**(*delay*, *thickness*)

Convert an intercept-time delay to an altered-zone slowness contrast.

Algebraic dual of `delay_to_altered_zone_thickness()`: pins the altered-zone thickness h and solves for the slowness contrast $\Delta s = s_{\text{altered}} - s_{\text{virgin}}$ from the same first-order refraction relation

$$\Delta t = 2\,h\,\Delta s \quad \Longrightarrow \quad \Delta s = \frac{\Delta t}{2h}.$$

The absolute altered-zone slowness is recovered by adding back the virgin slowness: $s_{\text{altered}} = s_{\text{virgin}} + \Delta s$.

Parameters

- **delay** (*ndarray or float*) – Per-depth delay time (s).
- **thickness** (*float or ndarray*) – Assumed altered-zone thickness (m). A scalar pins the same thickness at every depth; a same-shape array allows a depth-varying anchor (e.g. driven by an image-log halo thickness). Must be strictly positive at every depth.

Returns
Slowness contrast (s/m), same shape as `delay`. Values are clipped to be non-negative – negative delays (inversion noise in the virgin zones) map to a zero contrast.

Return type
ndarray

Raises
ValueError – If any `thickness` value is non-positive.

`fwap.tomography.`**`solve_intercept_time`**(*travel_times*, *offsets*, *src_depth_idx*, *rec_depth_idx*, *n_depth*, *depth_axis=None*, *mean_delay_zero=True*, *smooth_s=0.0*, *smooth_src=0.0*, *smooth_rec=0.0*, *delay_l2=0.0*, *method='midpoint'*)

Least-squares intercept-time inversion with regularisation and posterior standard errors.

Parameters

- **method** (*{"midpoint", "segmented"}*) – If `"midpoint"`, the whole offset is assigned to the midpoint cell – fast but biased at long offsets. If `"segmented"`, the offset is split across every cell the ray traverses; in that case `src_depth_idx` and `rec_depth_idx` are reinterpreted as *depths* in metres (they are still named `_idx` for API continuity).
- **smooth_s** (*float, default 0.0*) – Per-block first-difference regularisation weights. Setting any of these > 0 penalises abrupt depth-to-depth changes in the corresponding unknown (formation slowness, source delay, or receiver delay). Default 0 gives an unregularised least-squares fit.
- **smooth_src** (*float, default 0.0*) – Per-block first-difference regularisation weights. Setting any of these > 0 penalises abrupt depth-to-depth changes in the corresponding unknown (formation slowness, source delay, or receiver delay). Default 0 gives an unregularised least-squares fit.
- **smooth_rec** (*float, default 0.0*) – Per-block first-difference regularisation weights. Setting any of these > 0 penalises abrupt depth-to-depth changes in the corresponding unknown (formation slowness, source delay, or receiver delay). Default 0 gives an unregularised least-squares fit.
- **mean_delay_zero** (*bool, default True*) – Pin the mean of the source-delay block and of the receiver-delay block to zero, independently. Two separate rows are added to the augmented system: one constraining `sum(d_src) = 0` and one constraining `sum(d_rec) = 0`. This resolves the global shift gauge (`s -> s + c`, `d_src -> d_src - c * offset_mean` etc.) and the inter-block swap gauge (`d_src -> d_src + c`, `d_rec -> d_rec - c`) that together leave the problem rank-deficient otherwise.
- **delay_l2** (*float*) – Weak L2 prior on the two delay blocks, pulling them towards zero. Useful when the true delays are expected to be small (zero mean, low variance): even after `mean_delay_zero` fixes the gauges, a weak L2 prior helps condition the system when the delay structure is subtle.
- **travel_times** (*ndarray*)
- **offsets** (*ndarray*)
- **src_depth_idx** (*ndarray*)
- **rec_depth_idx** (*ndarray*)
- **n_depth** (*int*)
- **depth_axis** (*ndarray | None*)

Returns
Including marginal posterior standard errors for each block.

Return type
InterceptTimeResult

3.5 fwap.dispersion

Dipole flexural processing: dispersion estimation and dispersive STC.

Implements the dipole-sonic algorithms of Part 3 of the book. Because the borehole flexural mode is dispersive, ordinary STC biases shear slowness high; this module provides three remedies: narrow-band STC (`narrow_band_stc()`), per-frequency phase-slowness estimation (`phase_slowness_from_f_k()`, `phase_slowness_matrix_pencil()`), and dispersion-corrected STC (`dispersive_stc()`).

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 3. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Kimball, C. V. (1998). Shear slowness measurement by dispersive processing of the borehole flexural mode. *Geophysics* 63(2), 337-344.
- Ekstrom, M. P. (1995). Dispersion estimation from borehole acoustic arrays using a modified matrix pencil algorithm. *29th Asilomar Conference on Signals, Systems and Computers*, 449-453.
- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes*, Chapter 4. CRC Press.
- Schmitt, D. P. (1988). Shear-wave logging in elastic formations. *Journal of the Acoustical Society of America* 84(6), 2230-2244.

Functions

`bandpass`(data, dt, f_lo, f_hi[, order])	Zero-phase Butterworth band-pass along the time axis.
`classify_flexural_anisotropy`(curve_a, curve_b, *)	Discriminate stress-induced vs intrinsic flexural anisotropy from two dispersion curves.
`dispersive_pseudo_rayleigh_stc`(data, dt, ...)	Dispersive STC for the pseudo-Rayleigh / guided trapped mode.
`dispersive_stc`(data, dt, offsets, ...[, ...])	Dispersive STC in the spirit of Kimball (1998).
`narrow_band_stc`(data, dt, offsets[, f_lo, f_hi])	Band-pass the gather, then run STC -> lower-bias shear slowness.
`phase_slowness_from_f_k`(data, dt, offsets[, ...])	Estimate phase slowness vs frequency by weighted LS phase-vs-offset.
`phase_slowness_matrix_pencil`(data, dt, offsets)	Single-mode phase slowness via a matrix-pencil / ESPRIT-style estimator at each frequency.
`shear_slowness_from_dispersion`(curve[, ...])	Quality-weighted mean of `s(f)` in the low-frequency asymptote band (Kimball, 1998, eq.

Classes

`DispersionCurve`(freq, slowness, quality)	Phase-slowness dispersion curve.
`FlexuralDispersionDiagnosis`(classification, ...)	Output of `classify_flexural_anisotropy()`.

class `fwap.dispersion.`**`DispersionCurve`**(*freq*, *slowness*, *quality*)

Bases: `object`

Phase-slowness dispersion curve.

Parameters

- **freq** (*ndarray*)

- **slowness** (*ndarray*)
- **quality** (*ndarray*)

freq

Frequencies at which the curve is evaluated (Hz).

Type
ndarray, shape (n_f,)

slowness

Estimated phase slowness (s/m) at each frequency. `NaN` / `0` where the estimator was undefined (typically below the source low-frequency cutoff).

Type
ndarray, shape (n_f,)

quality

Fit-quality weight in `[0, 1]` – the product of a phase-fit residual score and a band-normalised amplitude weight. Consumers of this curve (*shear_slowness_from_dispersion()* and plotting code) should mask or weight by this value.

Type
ndarray, shape (n_f,)

freq: ndarray

quality: ndarray

slowness: ndarray

class fwap.dispersion.**FlexuralDispersionDiagnosis**(*classification*, *delta_low*, *delta_high*, *crossover_frequency*, *reasons*)

Bases: `object`

Output of *classify_flexural_anisotropy()*.

Parameters

- **classification** (*str*)
- **delta_low** (*float*)
- **delta_high** (*float*)
- **crossover_frequency** (*float | None*)
- **reasons** (*tuple[str, ...]*)

classification

One of `"isotropic"`, `"intrinsic"`, `"stress_induced"`, `"ambiguous"`.

Type
str

delta_low

Mean $s_b(f) - s_a(f)$ over the low-frequency band (s/m). `NaN` if no quality-passing samples land in the band.

Type
float

delta_high

Same over the high-frequency band.

Type
float

crossover_frequency

Frequency (Hz) of the **first** zero-crossing of $s_b(f) - s_a(f)$ inside the bracket `[f_low_band[0], f_high_band[1]]`. `None` when no crossover is detected (intrinsic / isotropic / ambiguous cases) or when no sign flip is observed inside the bracket.

Type
float or None

reasons

Human-readable description of how the classification was reached. Useful for QC / display.

Type
tuple of str

classification: str

crossover_frequency: float | None

delta_high: float

delta_low: float

reasons: tuple[str, ...]

`fwap.dispersion.`**bandpass**(*data*, *dt*, *f_lo*, *f_hi*, *order=4*)

Zero-phase Butterworth band-pass along the time axis.

Parameters

- **data** (*ndarray*) – Input gather; filtering is applied along `axis=-1`.
- **dt** (*float*) – Sampling interval (s).
- **f_lo** (*float*) – Lower and upper corner frequencies (Hz). Values are clipped to the open interval `(0, Nyquist)` before designing the filter.
- **f_hi** (*float*) – Lower and upper corner frequencies (Hz). Values are clipped to the open interval `(0, Nyquist)` before designing the filter.
- **order** (*int, default 4*) – Filter order per pass. The effective order is doubled by the zero-phase forward-backward application (`sosfiltfilt`).

Returns
Filtered data, same shape as `data`.

Return type
ndarray

`fwap.dispersion.`**classify_flexural_anisotropy**(*curve_a*, *curve_b*, ***, *quality_threshold=0.5*, *min_anisotropy=5e-06*, *f_low_band=(1000.0, 2500.0)*, *f_high_band=(4000.0, 8000.0)*)

Discriminate stress-induced vs intrinsic flexural anisotropy from two dispersion curves.

Following Sinha & Kostek (1996, *Geophysics* 61(6), 1899-1907) and Tang & Cheng (2004, sect. 5.3), the two cross-dipole flexural dispersion curves of a stress-anisotropic formation **cross over** in frequency: the low-frequency flexural mode samples ~1-2 wavelengths radially into the formation – primarily the far-field rock fabric – while

the high-frequency mode samples ~0.1-0.5 wavelengths radially – primarily the near-wellbore stress concentration. When the bulk fabric is isotropic but the borehole is loaded by an anisotropic far-field stress, the two regions see *different* anisotropy, so $\Delta s(f) = s_b(f) - s_a(f)$ changes sign between the bands. Intrinsic anisotropy (aligned fractures, layered shale fabric) shows no such crossover – the same direction is slow at every frequency.

3.5.1 Classification logic

Compute Δs_{low} and Δs_{high} as the means of $\Delta s(f)$ over `f_low_band` and `f_high_band` respectively, restricted to bins where both curves have `quality >= quality_threshold` and a finite slowness. Then:

- both bands < min_anisotropy in magnitude → `"isotropic"`;
- both bands above threshold with the **same** sign → `"intrinsic"`;
- both bands above threshold with **opposite** signs → `"stress_induced"` (and the crossover frequency is reported);
- any other combination (one band missing, only one band anisotropic) → `"ambiguous"`.

param curve_a
The two flexural dispersion curves – typically obtained by running *`phase_slowness_from_f_k()`* on the two rotated cross-dipole components (XX' and YY' in the Alford-rotated frame). The two curves must share the same `freq` axis to within a small relative tolerance.

type curve_a
DispersionCurve

param curve_b
The two flexural dispersion curves – typically obtained by running *`phase_slowness_from_f_k()`* on the two rotated cross-dipole components (XX' and YY' in the Alford-rotated frame). The two curves must share the same `freq` axis to within a small relative tolerance.

type curve_b
DispersionCurve

param quality_threshold
Per-bin quality floor on each curve; bins below this on either curve are excluded from the band averages.

type quality_threshold
float, default 0.5

param min_anisotropy
Magnitude of Δs below which a band is treated as isotropic. The default ~5 us/m corresponds to ~1.5 us/ft.

type min_anisotropy
float, default 5e-6 s/m

param f_low_band
Low-frequency averaging band (samples deep into the formation).

type f_low_band
(float, float), default (1000, 2500) Hz

param f_high_band
High-frequency averaging band (samples near the borehole wall).

type f_high_band
(float, float), default (4000, 8000) Hz

returns
Classification plus the diagnostic numbers used to reach it.

rtype
FlexuralDispersionDiagnosis

raises ValueError
If the two curves have mismatched `freq` axes, or the bands are mis-ordered / overlap.

See also

`phase_slowness_from_f_k`
Produces the input dispersion curves from raw cross-dipole sonic data.

`fwap.anisotropy.alford_rotation`
Cross-dipole rotation that defines the fast / slow components from the raw XX/XY/YX/YY tensor.

References

- Sinha, B. K., & Kostek, S. (1996). Stress-induced azimuthal anisotropy in borehole flexural waves. *Geophysics* 61(6), 1899-1907.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, Section 5.3 (stress-induced vs intrinsic anisotropy discrimination).

Parameters

- **`curve_a`** (`DispersionCurve`)
- **`curve_b`** (`DispersionCurve`)
- **`quality_threshold`** (*`float`*)
- **`min_anisotropy`** (*`float`*)
- **`f_low_band`** (*`tuple[float, float]`*)
- **`f_high_band`** (*`tuple[float, float]`*)

Return type
FlexuralDispersionDiagnosis

`fwap.dispersion.`**`dispersive_pseudo_rayleigh_stc`**(*data*, *dt*, *offsets*, ***, *v_fluid=1500.0*, *a_borehole=0.1*, *shear_slowness_range=(0.00013, 0.0005)*, *n_slowness=91*, *f_range=(3000.0, 12000.0)*, *window_length=0.001*, *time_step=4*, *min_energy_fraction=1e-08*)

Dispersive STC for the pseudo-Rayleigh / guided trapped mode.

Direct analogue of *`dispersive_stc()`* applied to the pseudo-Rayleigh dispersion law of *`fwap.synthetic.pseudo_rayleigh_dispersion()`*. For each trial formation shear slowness `s_shear`, the expected phase slowness

$$s_{\text{phase}}(f; s_{\text{shear}}) \;=\; s_{\text{shear}} \;+\; (s_{\text{fluid}} - s_{\text{shear}})\,\frac{(f/f_c)^2}{1+(f/f_c)^2},$$

with $s_{\text{fluid}} = 1/V_{\text{fluid}}$ and $f_c = V_s/(2\pi a)$, is used to back-project every frequency bin before the windowed semblance. A true pseudo-Rayleigh arrival collapses to zero relative delay at the correct `s_shear`, removing the high-frequency bias that plagues plain STC for guided modes – the same trick *`dispersive_stc()`* uses for the dipole flexural mode.

The output `slowness` axis holds the **formation shear** slowness (i.e., the low-frequency cutoff of the pseudo-Rayleigh branch, equal to $1/V_s$), not the per-frequency phase slowness. Pick the maximum-coherence row of the surface to recover formation Vs from a pseudo-Rayleigh-dominated gather.

3.5.2 Existence constraint

Pseudo-Rayleigh exists only in **fast formations** ($V_s > V_{\text{fluid}}$). The trial shear-slowness range is therefore required to satisfy `shear_slowness_range[1] < 1 / v_fluid`; the wrapper raises `ValueError` for ranges that would step into the slow-formation regime where the dispersion law is undefined.

param data
Real-valued multichannel gather.

type data
ndarray, shape (n_rec, n_samples)

param dt
Sampling interval (s).

type dt
float

param offsets
Source-to-receiver offsets (m).

type offsets
ndarray, shape (n_rec,)

param v_fluid
Borehole-fluid acoustic velocity (m/s). Sets the high- frequency asymptote of the pseudo-Rayleigh phase slowness.

type v_fluid
float, default 1500.0

param a_borehole
Borehole radius (m). Sets the cutoff frequency $f_c = V_s/(2\pi a)$ of the dispersion law.

type a_borehole
float, default 0.1

param shear_slowness_range
Lower and upper trial formation shear slowness (s/m). Must be strictly positive, ordered, and bounded above by `1 / v_fluid`.

type shear_slowness_range
(float, float), default (130 us/m, 500 us/m)

param n_slowness
Number of trial shear-slowness points.

type n_slowness
int, default 91

param f_range
Inclusive frequency band over which the dispersion correction is applied. The pseudo-Rayleigh branch only exists above its low-frequency cutoff f_c; for typical sonic geometry (a = 0.1 m, Vs ~

2-5 km/s) f_c falls in the 3-8 kHz band, motivating the higher-frequency default relative to `dispersive_stc()` (which targets the low-frequency flexural mode).

type f_range
(float, float), default (3000, 12000) Hz

param window_length
STC time-window length (s). Pseudo-Rayleigh wavetrains are typically more impulsive than the flexural mode, so a 1 ms window suffices to bracket the dominant lobe at the default frequency band.

type window_length
float, default 1e-3

param time_step
Stride of window starts in samples.

type time_step
int, default 4

param min_energy_fraction
Energy floor relative to gather RMS; below this the coherence cell is reported as `NaN`.

type min_energy_fraction
float, default 1e-8

returns
`slowness` is the formation shear-slowness axis; a true pseudo-Rayleigh arrival peaks at the correct $1/V_s$ on this axis (the whole point of the transform). All other fields follow `fwap.coherence.STCResult` conventions.

rtype
STCResult

raises ValueError
If `shear_slowness_range[1] >= 1 / v_fluid` (the trial range would leave the pseudo-Rayleigh existence domain), if the range is mis-ordered, or if any bound is non-positive.

See also

fwap.dispersion.dispersive_stc
The dipole-flexural-mode equivalent. The two routines share the same back-projection / semblance machinery; only the per-mode dispersion law differs.

fwap.synthetic.pseudo_rayleigh_dispersion
The phenomenological dispersion law used here.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **v_fluid** (*float*)
- **a_borehole** (*float*)
- **shear_slowness_range** (*tuple[float, float]*)

- **n_slowness** (*int*)
- **f_range** (*tuple[float, float]*)
- **window_length** (*float*)
- **time_step** (*int*)
- **min_energy_fraction** (*float*)

Return type
STCResult

`fwap.dispersion.`**`dispersive_stc`**(*data*, *dt*, *offsets*, *dispersion_family*, *shear_slowness_range=(0.00015, 0.0006)*, *n_slowness=91*, *f_range=(500.0, 6000.0)*, *window_length=0.0015*, *time_step=4*, *min_energy_fraction=1e-08*)

Dispersive STC in the spirit of Kimball (1998).

For each trial shear slowness `s_shear`, the expected phase slowness `s_phase(f) = dispersion_family(s_shear)(f)` is computed and used to back-project every frequency bin before windowed semblance. A true flexural arrival is collapsed to zero relative delay at the correct `s_shear`, removing the high-frequency bias that plagues ordinary STC on dispersive modes.

Parameters

- **dispersion_family** (*Callable[[float], Callable[[ndarray], ndarray]]*) – Callable that maps a candidate shear slowness to a phase- slowness function `s_phase(f)`. The returned callable **must accept a NumPy array** of in-band frequencies and return a same-shape array of slownesses (s/m). For the phenomenological flexural model in this module:

```
lambda s: dipole_flexural_dispersion(vs=1/s, a_borehole=...)
```

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **shear_slowness_range** (*tuple[float, float]*)
- **n_slowness** (*int*)
- **f_range** (*tuple[float, float]*)
- **window_length** (*float*)
- **time_step** (*int*)
- **min_energy_fraction** (*float*)

Returns
`slowness` holds the *shear* slowness axis (not the phase slowness – the whole point of the transform).

Return type
STCResult

References

Kimball, C. V. (1998). Shear slowness measurement by dispersive processing of the borehole flexural mode. *Geophysics* 63(2), 337-344, eqs. 5-10.

`fwap.dispersion.`**`narrow_band_stc`**(*data*, *dt*, *offsets*, *f_lo=1500.0*, *f_hi=4000.0*, ***stc_kwargs*)

Band-pass the gather, then run STC -> lower-bias shear slowness.

Convenience wrapper that applies `bandpass()` to `data` and forwards the result (together with `dt` and `offsets`) to `fwap.coherence.stc()`. Keyword arguments in `stc_kwargs` are passed through unchanged to `stc()`.

Parameters

- **`data`** (*`ndarray, shape (n_rec, n_samples)`*)
- **`dt`** (*`float`*) – Sampling interval (s).
- **`offsets`** (*`ndarray, shape (n_rec,)`*) – Source-to-receiver offsets (m).
- **`f_lo`** (*`float`*) – Band-pass corner frequencies (Hz); see `bandpass()`.
- **`f_hi`** (*`float`*) – Band-pass corner frequencies (Hz); see `bandpass()`.
- **`**stc_kwargs`** – Forwarded to `fwap.coherence.stc()` (e.g. `slowness_range`, `n_slowness`, `window_length`, `time_step`).

Return type
STCResult

`fwap.dispersion.`**`phase_slowness_from_f_k`**(*data*, *dt*, *offsets*, *f_range=(500.0, 8000.0)*, *method='frequency_unwrap'*)

Estimate phase slowness vs frequency by weighted LS phase-vs-offset.

3.5.3 `method`

`"frequency_unwrap"` (default)
Cross-correlate each trace against the reference (first) trace in the frequency domain, then unwrap the cross-spectrum phase along the **frequency** axis per trace. Because the cross spectrum of trace 0 with itself is identically real, the unwrap is anchored; trace-to-trace 2*pi ambiguities are eliminated.

`"spatial_unwrap"`
Unwrap across receivers at each frequency. Fails when the total phase swing across the aperture exceeds pi, i.e. above roughly `1 / (2 * aperture * s)` Hz; prefer `"frequency_unwrap"` unless replicating a specific reference output.

3.5.4 Validity band

The frequency-domain unwrap is itself valid provided `|dphi/df * df| < pi`, i.e. the arrival time is less than half the record length. That holds for typical sonic data.

Parameters

- **`data`** (*`ndarray`*)
- **`dt`** (*`float`*)
- **`offsets`** (*`ndarray`*)
- **`f_range`** (*`tuple[float, float]`*)
- **`method`** (*`Literal['frequency_unwrap', 'spatial_unwrap']`*)

Return type
DispersionCurve

`fwap.dispersion.`**`phase_slowness_matrix_pencil`**(*data*, *dt*, *offsets*, *f_range=(500.0, 8000.0)*)

Single-mode phase slowness via a matrix-pencil / ESPRIT-style estimator at each frequency.

At a given `f` the spatial samples `X_i = A exp(2*pi*i*f*s*x_i)` form a geometric progression along a uniform array. Stacking two shifted windows and taking the generalised eigenvalue yields `exp(2*pi*i*f*s*dx)`; the slowness follows. Bypasses phase unwrapping entirely (Ekstroem, 1995; related to Prony and MUSIC).

3.5.5 Caveats

Requires a uniform receiver spacing; assumes a single dominant spatial mode per frequency. For a two-mode problem one should use a true matrix-pencil algorithm with pencil parameter `L > 1`.

Parameters

- **`data`** (*`ndarray`*)
- **`dt`** (*`float`*)
- **`offsets`** (*`ndarray`*)
- **`f_range`** (*`tuple[float, float]`*)

Return type
DispersionCurve

`fwap.dispersion.`**`shear_slowness_from_dispersion`**(*curve*, *f_lo=500.0*, *f_hi=2500.0*, *quality_threshold=0.8*)

Quality-weighted mean of `s(f)` in the low-frequency asymptote band (Kimball, 1998, eq. 14).

If no dispersion points in `[f_lo, f_hi]` meet `quality_threshold` the call falls back to the unweighted set of finite points in the same band and emits a `logging.warning` so the caller can see that their quality gate was dropped.

Parameters

- **`curve`** (`DispersionCurve`)
- **`f_lo`** (*`float`*)
- **`f_hi`** (*`float`*)
- **`quality_threshold`** (*`float`*)

Return type
float

3.6 fwap.dip

Dip and azimuth estimation from a borehole ring array.

Implements Part 4 of the book ("Dip measurement based on acoustic data"). A ring array sees a dipping bed as an azimuthally cosine- varying arrival; maximising stack coherence over (dip, azimuth) after removing that cosine yields the structural dip and dip direction at each depth.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 4. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Hsu, K., & Esmersoy, C. (1992). Parametric estimation of phase and group slownesses from sonic logging waveforms. *Geophysics* 57(8), 978-985 (azimuthal coherence framework).

Functions

`estimate_dip`(data, dt, axial_offsets, ...[, ...])	Estimate (dip, azimuth) maximising de-tilted array coherence.
`synthesize_azimuthal_arrival`([n_rec, ...])	Synthetic azimuthal ring-array arrival with a dipping-bed signature.

Classes

`AzimuthalGather`(data, dt, axial_offsets, ...)	Output of `synthesize_azimuthal_arrival()`.
`DipResult`(dip, azimuth, coherence, surface, ...)	Output of `estimate_dip()`.

class `fwap.dip.`**`AzimuthalGather`**(*data*, *dt*, *axial_offsets*, *azimuths*, *tool_radius*, *slowness*)

Bases: `NamedTuple`

Output of `synthesize_azimuthal_arrival()`.

A `NamedTuple` so that existing tuple-unpacking call sites keep working while new code can access fields by name.

3.6.1 Fields

data
[ndarray, shape (n_rec, n_samples)] Synthetic azimuthal gather.

dt
[float] Sampling interval (s).

axial_offsets
[ndarray, shape (n_rec,)] All zeros – the ring-array model places every receiver on a common transverse ring.

azimuths
[ndarray, shape (n_rec,)] Receiver azimuths (rad), uniformly spaced in `[0, 2*pi)`.

tool_radius
[float] Ring radius (m).

slowness
[float] Formation slowness of the arrival (s/m).

`axial_offsets: ndarray`
Alias for field number 2

`azimuths: ndarray`
Alias for field number 3

`data: ndarray`
Alias for field number 0

`dt: float`
Alias for field number 1

`slowness: float`
Alias for field number 5

tool_radius: float

Alias for field number 4

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **axial_offsets** (*ndarray*)
- **azimuths** (*ndarray*)
- **tool_radius** (*float*)
- **slowness** (*float*)

class fwap.dip.**DipResult**(*dip*, *azimuth*, *coherence*, *surface*, *dip_axis*, *azimuth_axis*, *refined=False*)

Bases: `object`

Output of `estimate_dip()`.

Parameters

- **dip** (*float*)
- **azimuth** (*float*)
- **coherence** (*float*)
- **surface** (*ndarray*)
- **dip_axis** (*ndarray*)
- **azimuth_axis** (*ndarray*)
- **refined** (*bool*)

dip

Estimated bed dip (rad), `dip in [0, pi/2]`.

Type
float

azimuth

Estimated dip azimuth (rad), folded to `(-pi, pi]`. Measured from the reference direction of the `azimuths` array passed to `estimate_dip()`.

Type
float

coherence

Stack coherence at the recovered `(dip, azimuth)`; in `[0, 1]`, 1 = perfectly aligned after de-tilt.

Type
float

surface

Coherence evaluated on the full coarse grid.

Type
ndarray, shape (n_dip, n_az)

dip_axis

Trial dip angles (rad) corresponding to the rows of `surface`.

Type
ndarray, shape (n_dip,)

azimuth_axis

Trial azimuth angles (rad, in `[-pi, pi)`) corresponding to the columns of `surface`.

Type
ndarray, shape (n_az,)

refined

`True` if the returned `(dip, azimuth)` came from the Nelder-Mead polish; `False` if it is the raw grid maximum.

Type
bool

azimuth: float

azimuth_axis: ndarray

coherence: float

dip: float

dip_axis: ndarray

refined: bool = False

surface: ndarray

`fwap.dip.`**estimate_dip**(*data*, *dt*, *axial_offsets*, *azimuths*, *tool_radius*, *slowness*, *dip_range=(0.0, np.float64(1.0471975511965976))*, *n_dip=31*, *n_az=36*, *refine=True*)

Estimate (dip, azimuth) maximising de-tilted array coherence.

Parameters

- **data** (*ndarray, shape (n_rec, n_samples)*) – Azimuthal ring-array gather at a single depth.
- **dt** (*float*) – Sampling interval (s).
- **axial_offsets** (*ndarray, shape (n_rec,)*) – **Ignored.** The ring-array dip model (Part 4 of Mari et al., 1994) treats all receivers as lying on a common transverse ring; only the azimuthal spread and the tool radius enter the cosine-of-azimuth moveout expected for a dipping bed. The parameter is retained so that callers feeding the `synthesize_azimuthal_arrival` tuple do not need to reshape their call sites, and so that a future mixed axial/ring array can reuse the same signature.
- **azimuths** (*ndarray, shape (n_rec,)*) – Receiver azimuth (rad) around the tool, measured from an arbitrary reference.
- **tool_radius** (*float*) – Radius (m) of the receiver ring.
- **slowness** (*float*) – Formation slowness (s/m) of the arrival whose azimuthal moveout is being analysed.
- **dip_range** (*(float, float)*) – Lower and upper dip angles to scan (rad).
- **n_dip** (*int*) – Resolution of the coarse grid over (dip, azimuth).

- **n_az** (*int*) – Resolution of the coarse grid over (dip, azimuth).
- **refine** (*bool*) – If `True`, polish the grid maximum with Nelder-Mead.
- **Strategy**
- **--------**
- **``(alpha** (*1. Coarse grid search over*) –
- **phi)``.**
- **refine=True** (*2. If*) – maximum. The coherence surface near the true maximum is smooth and unimodal, so a simplex method converges quickly.
- **grid** (*Nelder-Mead polish starting at the*) – maximum. The coherence surface near the true maximum is smooth and unimodal, so a simplex method converges quickly.

Return type
DipResult

`fwap.dip.`**synthesize_azimuthal_arrival**(*n_rec=8*, *n_samples=1024*, *dt=2e-05*, *tool_radius=0.08*, *slowness=0.00025*, *dip=np.float64(0.5235987755982988)*, *azimuth=np.float64(0.7853981633974483)*, *f0=8000.0*, *noise=0.02*, *seed=0*)

Synthetic azimuthal ring-array arrival with a dipping-bed signature.

Builds a single-mode gather in which receiver `i` sees an arrival at `t0 + slowness * tool_radius * sin(dip) * cos(az_i - azimuth)` – the cosine-of-azimuth moveout that *`estimate_dip()`* inverts for.

Parameters

- **n_rec** (*int, default 8*) – Number of receivers on the ring.
- **n_samples** (*int, default 1024*) – Samples per trace.
- **dt** (*float, default 2.0e-5*) – Sampling interval (s).
- **tool_radius** (*float, default 0.08*) – Ring radius (m).
- **slowness** (*float, default 1/4000*) – Formation slowness of the arrival (s/m).
- **dip** (*float, default deg2rad(30)*) – True bed dip (rad).
- **azimuth** (*float, default deg2rad(45)*) – True dip azimuth (rad).
- **f0** (*float, default 8000*) – Ricker peak frequency (Hz).
- **noise** (*float, default 0.02*) – Gaussian-noise RMS relative to the gather RMS.
- **seed** (*int, default 0*) – Seed for the noise RNG.

Returns
Named 6-tuple with fields `data`, `dt`, `axial_offsets`, `azimuths`, `tool_radius`, `slowness`. Tuple-unpacking at the call site continues to work: `data, dt, ax_off, az, a, slow = synthesize_azimuthal_arrival()`.

Return type
AzimuthalGather

3.7 fwap.anisotropy

Cross-dipole Alford rotation for shear-wave azimuthal anisotropy.

> **Note**
>
> Cross-dipole Alford rotation is **not** treated in Mari et al. (1994). This module is an extension that complements the dipole-flexural processing of `fwap.dispersion`: a four-component (XX, XY, YX, YY) shear measurement is rotated into the (fast, slow) shear frame so that each component carries one polarization.

References

- Alford, R. M. (1986). Shear data in the presence of azimuthal anisotropy. *56th SEG Annual International Meeting, Expanded Abstracts*, 476-479.
- Esmersoy, C., Koster, K., Williams, M., Boyd, A., & Kane, M. (1994). Dipole shear anisotropy logging. *64th SEG Annual International Meeting, Expanded Abstracts*, 1139-1142.

Functions

`alford_rotation`(xx, xy, yx, yy)	Cross-dipole Alford rotation: find the rotation angle that minimises cross-component energy.
`alford_rotation_from_tensor`(tensor)	Cross-dipole Alford rotation from a packed `(2, 2, n_samples)` tensor.
`stoneley_horizontal_shear_modulus`(...)	Horizontal shear modulus C_{66} from low-frequency Stoneley slowness (White 1983 tube-wave formula).
`stress_anisotropy_from_alford`(alford, dt)	Re-label an `AlfordResult` in petrophysical terms.
`thomsen_gamma`(c44, c66)	Thomsen (1986) shear-anisotropy parameter from C44 and C66.
`thomsen_gamma_from_logs`(slowness_dipole, ...)	Thomsen γ from combined dipole + Stoneley sonic logs.

Classes

`AlfordResult`(angle, fast, slow, ...)	Output of `alford_rotation()`.
`StressAnisotropyEstimate`(...)	Petrophysical labelling of a cross-dipole Alford rotation.
`ThomsenGammaResult`(c44, c66, gamma)	Output of `thomsen_gamma_from_logs()`.

class `fwap.anisotropy.`**`AlfordResult`**(*angle*, *fast*, *slow*, *cross_energy_ratio*)

Bases: `object`

Output of `alford_rotation()`.

Parameters

- **`angle`** (*`float`*)
- **`fast`** (*`ndarray`*)
- **`slow`** (*`ndarray`*)
- **`cross_energy_ratio`** (*`float`*)

angle

Fast-shear azimuth measured from the X-dipole firing direction (radians, in `(-pi/2, pi/2]`).

Type
float

fast, slow

Rotated fast-shear and slow-shear diagonal components.

Type
ndarray, shape (*n_samples*,)

cross_energy_ratio

`(xy' + yx') energy / total energy` after rotation; zero means perfect diagonalisation.

Type
float

angle: float

cross_energy_ratio: float

fast: ndarray

slow: ndarray

class `fwap.anisotropy.`**StressAnisotropyEstimate**(*max_horizontal_stress_azimuth*, *min_horizontal_stress_azimuth*, *splitting_time_delay*, *anisotropy_strength*, *fracture_indicator*, *rotation_quality*, *alford*)

Bases: `object`

Petrophysical labelling of a cross-dipole Alford rotation.

Re-frames the angle and energy-ratio outputs of `alford_rotation()` in terms of the three Workflow-3 quantities the book (Mari et al. 1994, Part 3) lists for the dipole-sonic deliverable: stress direction, anisotropy strength, and a flexural-fracture indicator. The numerics are the same as in `AlfordResult`; this dataclass just adds the petrophysical interpretation and a couple of derived metrics (splitting-time delay, anisotropy strength as a relative L2 norm) that the raw Alford output does not surface directly.

Parameters

- **max_horizontal_stress_azimuth** (*float*)
- **min_horizontal_stress_azimuth** (*float*)
- **splitting_time_delay** (*float*)
- **anisotropy_strength** (*float*)
- **fracture_indicator** (*float*)
- **rotation_quality** (*float*)
- **alford** (`AlfordResult`)

max_horizontal_stress_azimuth

Fast-shear azimuth, in radians, in `(-pi/2, pi/2]`. Conventionally aligned with the maximum horizontal stress $\sigma_{H,\max}$ for stress-induced anisotropy, with the strike of natural fractures for fracture-induced anisotropy, and with the bedding orientation for intrinsic anisotropy in laminated formations. The mechanism that dominates is *not* identifiable from a single Alford rotation – the user picks the interpretation given image logs, mud-loss data, or other ancillary measurements.

Type
float

min_horizontal_stress_azimuth

Slow-shear azimuth = `max_horizontal_stress_azimuth + pi/2`, folded back into `(-pi/2, pi/2]`.

Type
float

splitting_time_delay

Time delay (s) between the fast and slow shear arrivals, estimated as the lag of the cross-correlation peak between the rotated `fast` and `slow` waveforms (positive ⇔ slow trails fast, which is the physical case). The classic anisotropy strength metric $\gamma = \Delta t / t_{\text{travel}}$ follows directly from this quantity.

Type
float

anisotropy_strength

Dimensionless `[0, 1]` measure of how much fast and slow shear differ, defined as $\|f - s\| / \sqrt{2\left(\|f\|^2 + \|s\|^2\right)}$. `0` for identical waveforms (isotropic medium); approaches `1` only as $s \to -f$ (sign-flipped pair), and equals $1/\sqrt{2} \approx 0.707$ for orthogonal equal-energy waveforms. Time-shift-blind, so reports anisotropy even on a fast / slow pair that differ only in arrival time.

Type
float

fracture_indicator

Heuristic flexural-wave fracture proxy in `[0, 1]`: rotation_quality × anisotropy_strength. High values require both a clean rotation (so the anisotropy is well-resolved) and a strong fast/slow contrast (so the medium is genuinely anisotropic). Cannot distinguish stress-induced from fracture-induced anisotropy on its own; intended as a *flag* track to be cross-checked against a borehole image log or a Stoneley-permeability indicator before being labelled as fractures.

Type
float

rotation_quality

`1 - cross_energy_ratio` from *`AlfordResult`*. `1` means the rotation perfectly diagonalises the cross-dipole tensor; lower values flag depths where the two-shear Alford model itself fits poorly (e.g. multi-mode interference, off-axis arrivals, low-SNR data).

Type
float

alford

The underlying Alford rotation, kept for callers that need the rotated waveforms (`fast` / `slow`) themselves.

Type
AlfordResult

alford: *AlfordResult*

anisotropy_strength: float

fracture_indicator: float

max_horizontal_stress_azimuth: float

min_horizontal_stress_azimuth: float

rotation_quality: float

splitting_time_delay: float

class fwap.anisotropy.**ThomsenGammaResult**(*c44*, *c66*, *gamma*)

Bases: object

Output of *thomsen_gamma_from_logs()*.

Parameters

- **c44** (*ndarray*)
- **c66** (*ndarray*)
- **gamma** (*ndarray*)

c44

Vertical shear modulus $C_{44} = \rho V_{Sv}^2$ (Pa). Derived from the dipole shear log (vertically-propagating shear with horizontal polarization in a vertical well).

Type
ndarray

c66

Horizontal shear modulus C_{66} (Pa). Derived from the Stoneley low-frequency tube-wave inversion (White 1983; Norris 1990 for the VTI extension).

Type
ndarray

gamma

Thomsen (1986) shear-anisotropy parameter $\gamma = (C_{66} - C_{44})/(2C_{44})$. 0 for an isotropic formation; positive in typical VTI shales (horizontal shear stiffer than vertical shear).

Type
ndarray

c44: ndarray

c66: ndarray

gamma: ndarray

fwap.anisotropy.**alford_rotation**(*xx*, *xy*, *yx*, *yy*)

Cross-dipole Alford rotation: find the rotation angle that minimises cross-component energy.

Given the 2x2 time-domain cross-dipole tensor `[[xx, xy], [yx, yy]]`, rotate to fast/slow shear axes. The closed-form minimiser of the sum of squared off-diagonal components over rotation angle `theta` is

tan(4 theta) = (A + B) / (C - D) = 2 A / (C - D)

where
A = <xx - yy, xy + yx>, B = <xy + yx, xx - yy> (= A for real-valued traces), C = <xx - yy, xx - yy>, D = <xy + yx, xy + yx>.

The simplification `A + B = 2 A` is exact for real-valued cross-dipole components; for the general (complex) case keep `(A + B) / (C - D)`.

In practice both `theta` and `theta + pi/2` are stationary points – we pick the one that puts the *slower* shear on the `y'` axis (by integrated arrival time) for the conventional labelling; the caller can swap if the other convention is needed.

Parameters

- **xx** (*ndarray*) – The four components of the cross-dipole tensor at a single depth. All four arrays must have the same shape. Typically one-dimensional (`(n_samples,)`), but any shape is accepted; time-integrated inner products are taken over the flattened array.
- **xy** (*ndarray*) – The four components of the cross-dipole tensor at a single depth. All four arrays must have the same shape. Typically one-dimensional (`(n_samples,)`), but any shape is accepted; time-integrated inner products are taken over the flattened array.
- **yx** (*ndarray*) – The four components of the cross-dipole tensor at a single depth. All four arrays must have the same shape. Typically one-dimensional (`(n_samples,)`), but any shape is accepted; time-integrated inner products are taken over the flattened array.
- **yy** (*ndarray*) – The four components of the cross-dipole tensor at a single depth. All four arrays must have the same shape. Typically one-dimensional (`(n_samples,)`), but any shape is accepted; time-integrated inner products are taken over the flattened array.

Returns

See the `AlfordResult` dataclass for field-by-field descriptions.

Return type

AlfordResult

References

Alford, R. M. (1986). Shear data in the presence of azimuthal anisotropy. *56th SEG Annual Meeting*, Expanded Abstracts, 476-479. Esmersoy, C., et al. (1994). Dipole shear anisotropy logging. *64th SEG Annual Meeting*, Expanded Abstracts, 1139-1142.

`fwap.anisotropy.`**`alford_rotation_from_tensor`**(*tensor*)

Cross-dipole Alford rotation from a packed `(2, 2, n_samples)` tensor.

Thin adapter over `alford_rotation()` for callers that already carry the cross-dipole measurement as a single array with the two dipole-source / two dipole-receiver axes as the leading two dimensions.

Parameters

tensor (ndarray, shape `(2, 2, n_samples)` (or any broadcastable) – shape `(2, 2, ...)`) `tensor[0, 0]` is XX (X-source -> X-receiver), `tensor[0, 1]` is XY, `tensor[1, 0]` is YX, `tensor[1, 1]` is YY.

Return type

AlfordResult

Raises

ValueError – If `tensor.shape[:2] != (2, 2)`.

`fwap.anisotropy.`**`stoneley_horizontal_shear_modulus`**(*slowness_stoneley*, *, *rho_fluid*, *v_fluid*)

Horizontal shear modulus C_{66} from low-frequency Stoneley slowness (White 1983 tube-wave formula).

For a fluid-filled borehole through a (possibly transversely- isotropic) elastic formation, the low-frequency Stoneley phase slowness S_{ST} satisfies (White 1983, eq. 5.42; Norris 1990 generalises the result to VTI media):

$$S_{ST}^2 \;=\; S_f^2 \;+\; \frac{\rho_f}{C_{66}},$$

where $S_f = 1/V_f$ is the borehole-fluid slowness and ρ_f is the fluid density. The relevant shear modulus is C_{66} (the horizontal shear modulus); for an isotropic formation $C_{66} = \mu = \rho V_S^2$. Solving for C_{66}:

$$C_{66} \;=\; \frac{\rho_f}{S_{ST}^2 - S_f^2}.$$

3.7.1 Assumptions

- Low-frequency limit (tube wave well below the dipole-flexural and pseudo-Rayleigh cutoff frequencies). Above the cutoffs Tang-Cheng (2004) eqs. 5.19-5.22 give the corrected, dispersive form.
- Inviscid borehole fluid; centred tool; circular borehole cross- section. Eccentricity and tool-mode effects fold into a multiplicative correction that fwap does not currently apply.
- Hard formation (formation shear impedance >> fluid impedance). Mudcake / casing layers are ignored; for a cased hole the multilayered radial form (Tang & Cheng 2004 §2) is required.

param slowness_stoneley
Per-depth Stoneley-wave slowness (s/m). Must be strictly greater than `1 / v_fluid` everywhere – the Stoneley wave is always slower than the unconfined fluid wave because the formation loads it.

type slowness_stoneley
ndarray or float

param rho_fluid
Borehole-fluid density (kg/m^3). Brine ~ 1000-1100; oil ~ 800-900; gas/foam << 1000.

type rho_fluid
float

param v_fluid
Borehole-fluid acoustic velocity (m/s). Brine ~ 1500; oil ~ 1300; gas << 1000.

type v_fluid
float

returns
C_{66} (Pa), broadcast to the shape of `slowness_stoneley`.

rtype
ndarray

raises ValueError
If `rho_fluid` or `v_fluid` is non-positive, or if any `slowness_stoneley` value is at or below the fluid slowness (the inversion is undefined there).

References

- White, J. E. (1983). *Underground Sound: Application of Seismic Waves.* Elsevier (sect. 5.5: tube waves).
- Norris, A. N. (1990). The speed of a tube wave. *J. Acoust. Soc. Am.* 87(1), 414-417.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, sect. 5.4 (Stoneley-wave inversion for C66 in VTI formations).

Parameters

- **`slowness_stoneley`** (*`ndarray`*)
- **`rho_fluid`** (*`float`*)
- **`v_fluid`** (*`float`*)

Return type
ndarray

`fwap.anisotropy.`**`stress_anisotropy_from_alford`**(*alford*, *dt*)

Re-label an `AlfordResult` in petrophysical terms.

Computes the splitting-time delay and the relative-L2 anisotropy strength from the rotated `fast` / `slow` waveforms, packages them with the orthogonal stress azimuths and a fracture-indicator heuristic, and returns the result as a `StressAnisotropyEstimate`. See that dataclass's docstring for the per-field semantics and caveats; the short version is that `max_horizontal_stress_azimuth` is the Alford fast-shear angle re-labelled with the conventional stress-direction interpretation.

Parameters

- **alford** (`AlfordResult`) – Output of `alford_rotation()` (or `alford_rotation_from_tensor()`) at one depth.
- **dt** (*float*) – Time sampling interval (s) of the rotated traces. Used only for the splitting-time-delay computation.

Return type
StressAnisotropyEstimate

`fwap.anisotropy.`**`thomsen_gamma`**(*c44*, *c66*)

Thomsen (1986) shear-anisotropy parameter from C44 and C66.

$$\gamma \;=\; \frac{C_{66} - C_{44}}{2\,C_{44}}.$$

For a vertically-symmetric (VTI) medium γ measures the difference between horizontally- and vertically-polarised shear stiffness; `0` for an isotropic formation, positive for typical layered shales.

Parameters

- **c44** (*scalar or ndarray*) – Vertical and horizontal shear moduli (Pa). Both must be strictly positive everywhere.
- **c66** (*scalar or ndarray*) – Vertical and horizontal shear moduli (Pa). Both must be strictly positive everywhere.

Returns
Thomsen γ, broadcast to the common shape.

Return type
ndarray

Raises
ValueError – If either modulus is non-positive anywhere.

References

Thomsen, L. (1986). Weak elastic anisotropy. *Geophysics* 51(10), 1954-1966.

`fwap.anisotropy.`**`thomsen_gamma_from_logs`**(*slowness_dipole*, *slowness_stoneley*, *rho*, *, *rho_fluid*, *v_fluid*)

Thomsen γ from combined dipole + Stoneley sonic logs.

Combines two complementary sonic measurements that together fix both off-diagonal shear moduli of a VTI formation:

- **Dipole shear log** – the low-frequency dipole flexural mode travels at the formation vertical shear speed $V_{Sv} = \sqrt{C_{44}/\rho}$, so

$$C_{44} \;=\; \rho \,/\, S_{S,\text{dipole}}^{2}.$$

- **Stoneley tube wave** – inverted via the White (1983) / Norris (1990) low-frequency formula in `stoneley_horizontal_shear_modulus()` to give the horizontal shear modulus C_{66}.

Their ratio gives Thomsen γ per `thomsen_gamma()`:

$$\gamma = \frac{C_{66} - C_{44}}{2\,C_{44}}.$$

This is the standard sonic-only VTI shear-anisotropy estimator that Tang & Cheng (2004), sect. 5.4 list as a Workflow-3 deliverable for the dipole-sonic + Stoneley combination. It does *not* recover the P-wave Thomsen parameters ϵ, δ – those need additional measurements (cross-well or VSP) since a vertical-well sonic record samples only the vertical P-wave slowness.

Parameters

- **slowness_dipole** (*ndarray*) – Per-depth dipole shear slowness (s/m). Typically `fwap.picker.ModePick.slowness` for the `"S"` mode gathered across depths, or the `"DTS"` column of a LAS log written via `fwap.picker.track_to_log_curves()` (after converting from us/ft back to s/m).
- **slowness_stoneley** (*ndarray*) – Per-depth Stoneley-wave slowness (s/m). Typically the `"Stoneley"` mode, low-frequency band.
- **rho** (*ndarray*) – Per-depth formation bulk density (kg/m^3) from the bulk- density log (typically the `RHOB` curve).
- **rho_fluid** (*float*) – Borehole-fluid density (kg/m^3); see `stoneley_horizontal_shear_modulus()`.
- **v_fluid** (*float*) – Borehole-fluid acoustic velocity (m/s).

Returns

`c44` (Pa), `c66` (Pa), and `gamma` (dimensionless), all per-depth and broadcast to the common input shape.

Return type

ThomsenGammaResult

Raises

ValueError – Same conditions as `stoneley_horizontal_shear_modulus()` and `thomsen_gamma()`, plus rejection of non-positive slownesses or densities.

3.8 fwap.attenuation

Attenuation (quality factor Q) estimation from array-sonic gathers.

Provides two complementary estimators: the centroid-frequency-shift method (`centroid_frequency_shift_Q()`) and the spectral-ratio / log-slope method (`spectral_ratio_Q()`).

> **Note**
>
> Q estimation is **not** covered in Mari et al. (1994); this module is an extension included for completeness. It belongs to the same broader "borehole as small-scale seismic experiment" philosophy that the book champions, but the algorithms come from the surface-seismic literature.

References

- Quan, Y., & Harris, J. M. (1997). Seismic attenuation tomography using the frequency shift method. *Geophysics* 62(3), 895-905.
- Bath, M. (1974). *Spectral Analysis in Geophysics.* Developments in Solid Earth Geophysics 7, Elsevier (spectral-ratio Q estimator).

Functions

`centroid_frequency_shift_Q`(data, dt, ...[, ...])	Attenuation (`Q`) from centroid-frequency shift along the array.
`spectral_ratio_Q`(data, dt, offsets, slowness)	Attenuation from the classical spectral-ratio (log-slope) method.

Classes

`AttenuationResult`(q, q_sigma, method[, ...])	Output of `centroid_frequency_shift_Q()` / `spectral_ratio_Q()`.

class `fwap.attenuation.`**`AttenuationResult`**(*q*, *q_sigma*, *method*, *diagnostic=<factory>*)

Bases: `object`

Output of `centroid_frequency_shift_Q()` / `spectral_ratio_Q()`.

Parameters

- **q** (*float*)
- **q_sigma** (*float*)
- **method** (*Literal['centroid', 'spectral_ratio']*)
- **diagnostic** (*dict[str, ndarray]*)

q

Quality factor. Higher `Q` = less attenuation.

Type
float

q_sigma

1-sigma uncertainty on `Q` from the regression.

Type
float

method

`"centroid"` or `"spectral_ratio"`.

Type
str

diagnostic

Fitted parameters and intermediate quantities useful for QC.

Type
dict

diagnostic: dict[str, ndarray]

method: Literal['centroid', 'spectral_ratio']

q: float

q_sigma: float

`fwap.attenuation.`**`centroid_frequency_shift_Q`**(*data*, *dt*, *offsets*, *slowness*, *window_length=0.0005*, *f_range=(2000.0, 30000.0)*, *pick_intercept=0.0*)

Attenuation (`Q`) from centroid-frequency shift along the array.

For a Gaussian source spectrum with centroid `fc0` and variance `sigma_f**2`, the centroid at travel time `t` decays linearly with `t` under constant-`Q` attenuation:

fc(t) = fc0 - pi * sigma_f**2 * t / Q

so `Q = -pi * sigma_f**2 / slope` where the slope comes from a weighted linear fit of `fc` against travel time across receivers.

3.8.1 Assumptions

The closed form above requires (a) a (locally) Gaussian source spectrum whose variance is approximately constant across the array and (b) a constant-`Q` attenuation model over the band of interest. Deviations from either – e.g., a source with structured side-lobes, or frequency-dependent `Q` – are the dominant source of systematic error. See Quan & Harris (1997), Section 3.

param pick_intercept
Absolute record time at which the first-breaking wave passes the `offset = 0` reference. Only shifts the x-axis of the QC plot in `diagnostic['t']`; the fitted slope and derived `Q` depend solely on travel time differences across the array.

type pick_intercept
float, default 0.0

References

Quan, Y., & Harris, J. M. (1997). Seismic attenuation tomography using the frequency shift method. *Geophysics* 62(3), 895-905.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray*)
- **slowness** (*float*)
- **window_length** (*float*)
- **f_range** (*tuple[float, float]*)
- **pick_intercept** (*float*)

Return type
AttenuationResult

`fwap.attenuation.`**`spectral_ratio_Q`**(*data*, *dt*, *offsets*, *slowness*, *window_length=0.0005*, *f_range=(3000.0, 20000.0)*, *reference=0*, *pick_intercept=0.0*)

Attenuation from the classical spectral-ratio (log-slope) method.

Under constant-`Q` attenuation the log amplitude ratio between two traces at travel times `t1` and `t2` is linear in frequency with slope `-pi*(t2 - t1)/Q`:

log(**|A2(f)|** / **|A1(f)|**) = -pi*(t2-t1)/Q * f + const.

A weighted LS fit of `log_ratio` vs `f` across frequency and receivers yields `Q`. This is the Bath (1974) estimator.

Parameters

- **`data`** (*`ndarray, shape (n_rec, n_samples)`*) – Array-sonic gather.
- **`dt`** (*`float`*) – Sampling interval (s).
- **`offsets`** (*`ndarray, shape (n_rec,)`*) – Source-to-receiver offsets (m).
- **`slowness`** (*`float`*) – Formation slowness of the analysed arrival (s/m); used to compute travel-time differences across the array.
- **`window_length`** (*`float, default 5e-4`*) – Length of the Hann-tapered window (s) used for the local spectrum at each receiver.
- **`f_range`** (*`(float, float), default (3000, 20000)`*) – Inclusive frequency band (Hz) over which the log-ratio fit is performed.
- **`reference`** (*`int, default 0`*) – Index of the receiver used as the denominator `A_1` in the log ratio. The fit combines all other receivers against this reference.
- **`pick_intercept`** (*`float, default 0.0`*) – Absolute record time at which the first-breaking wave passes `offset = 0`; affects only the absolute window placement.

Returns

With `method = "spectral_ratio"`. Returns `q = NaN` when no frequency bin has positive amplitude at every receiver.

Return type

AttenuationResult

References

Bath, M. (1974). *Spectral Analysis in Geophysics.* Developments in Solid Earth Geophysics 7, Elsevier.

3.9 fwap.rockphysics

Elastic moduli from sonic velocities.

Closes the loop from the raw-waveform processing in *`fwap.picker`* / *`fwap.dispersion`* (which produce `Vp` and `Vs` logs) to the geomechanical curves a petrophysicist wants next to the other log tracks: bulk modulus, shear modulus, Young's modulus, and Poisson's ratio.

The formulas used here are the standard isotropic small-strain relations; all four quantities follow from `Vp`, `Vs`, and the formation bulk density `rho`.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 1 (workflow from shear logs to elastic moduli). Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Mavko, G., Mukerji, T., & Dvorkin, J. (2009). *The Rock Physics Handbook*, 2nd ed., Section 1.5. Cambridge University Press.

Functions

`elastic_moduli`(vp, vs, rho)	Isotropic elastic moduli from compressional and shear velocity.
`gassmann_fluid_substitution`(k_dry, mu_dry, ...)	Gassmann (1951) fluid substitution.
`hill_average`(moduli, fractions)	Voigt-Reuss-Hill average: arithmetic mean of the Voigt and Reuss bounds.
`hornby_fracture_aperture`(...[, ...])	Fracture aperture from the Hornby et al. (1989) Stoneley reflection-coefficient inversion.
`reuss_average`(moduli, fractions)	Reuss (harmonic, isostress) average of a composite modulus.
`stoneley_amplitude_fracture_indicator`(...)	Dimensionless Stoneley-amplitude fracture / permeability indicator.
`stoneley_permeability_indicator`(...)	Dimensionless Stoneley-wave permeability indicator.
`stoneley_reflection_coefficient`(...)	Stoneley reflection coefficient $\lvert R \rvert$ from incident and reflected pulse amplitudes.
`voigt_average`(moduli, fractions)	Voigt (arithmetic, isostrain) average of a composite modulus.
`vp_vs_ratio`(vp, vs)	Vp/Vs ratio, a common lithology / fluid-content indicator.
`vs_from_stoneley_slow_formation`(...)	Formation shear velocity V_S from low-frequency Stoneley slowness.

Classes

`ElasticModuli`(k, mu, young, poisson, lambda_)	Isotropic elastic moduli derived from Vp, Vs, and density.
`GassmannResult`(k_sat, mu_sat)	Fluid-saturated bulk and shear moduli from Gassmann substitution.

class `fwap.rockphysics.`**`ElasticModuli`**(*k*, *mu*, *young*, *poisson*, *lambda_*)

Bases: `object`

Isotropic elastic moduli derived from Vp, Vs, and density.

Parameters

- **k** (*ndarray*)
- **mu** (*ndarray*)
- **young** (*ndarray*)
- **poisson** (*ndarray*)
- **lambda_** (*ndarray*)

k

Bulk modulus (Pa).

Type
ndarray or float

mu

Shear modulus (Pa).

Type
ndarray or float

young

Young's modulus (Pa).

Type
ndarray or float

poisson

Poisson's ratio (dimensionless, in `(-1, 0.5]`).

Type
ndarray or float

lambda_

First Lame parameter (Pa).

Type
ndarray or float

k: ndarray

lambda_: ndarray

mu: ndarray

poisson: ndarray

young: ndarray

class `fwap.rockphysics.`**GassmannResult**(*k_sat*, *mu_sat*)

Bases: `object`

Fluid-saturated bulk and shear moduli from Gassmann substitution.

Parameters

- **k_sat** (*ndarray*)
- **mu_sat** (*ndarray*)

k_sat

Saturated-rock bulk modulus (Pa).

Type
ndarray or float

mu_sat

Saturated-rock shear modulus (Pa). Equal to `mu_dry` because the Gassmann framework assumes the pore fluid carries no shear stress, so the shear modulus is fluid-insensitive.

Type
ndarray or float

k_sat: ndarray

mu_sat: ndarray

`fwap.rockphysics.`**`elastic_moduli`**(*vp*, *vs*, *rho*)

Isotropic elastic moduli from compressional and shear velocity.

For an isotropic medium of bulk density `rho`, compressional velocity `Vp`, and shear velocity `Vs`:

```
mu      = rho * Vs**2                          # shear modulus
lambda_ = rho * Vp**2 - 2 * mu                 # first Lame
K       = lambda_ + (2 / 3) * mu               # bulk modulus
E       = mu * (3 * lambda_ + 2 * mu) / (lambda_ + mu)
nu      = lambda_ / (2 * (lambda_ + mu))       # Poisson's ratio
```

Inputs may be scalars or arrays; operations broadcast in the usual NumPy way.

Parameters

- **vp** (*scalar or ndarray*) – Compressional wave velocity (m/s). Must be positive.
- **vs** (*scalar or ndarray*) – Shear wave velocity (m/s). Must be positive and satisfy `vs < vp`; otherwise the denominator of Poisson's ratio goes non-physical.
- **rho** (*scalar or ndarray*) – Bulk density (kg/m^3). Must be positive.

Returns

Per-sample moduli, same shape as the broadcast of the inputs. All moduli in Pa; Poisson's ratio is dimensionless.

Return type

ElasticModuli

Raises

ValueError – If any of `vp`, `vs`, `rho` is non-positive, or if `vs >= vp` anywhere (the isotropic rock-physics model requires `Vp > Vs * sqrt(4/3)` for stable positive moduli; we enforce the stricter and more familiar `Vp > Vs`).

`fwap.rockphysics.`**`gassmann_fluid_substitution`**(*k_dry*, *mu_dry*, *k_mineral*, *k_fluid*, *porosity*)

Gassmann (1951) fluid substitution.

Given the drained ("dry-frame") bulk and shear moduli of a porous rock, the bulk modulus of its solid mineral grains, and the bulk modulus of a replacement pore fluid at the rock's porosity, Gassmann gives the closed-form saturated-rock moduli:

$$K_{\text{sat}} \;=\; K_{\text{dry}} \;+\; \frac{\left(1 - K_{\text{dry}}/K_s\right)^2}{\phi/K_f + (1-\phi)/K_s - K_{\text{dry}}/K_s^2}$$

$$\mu_{\text{sat}} \;=\; \mu_{\text{dry}}$$

The shear-modulus invariance follows from the assumption that the pore fluid supports no shear; it is the practical fact that lets the Vs log be read as a rock-frame (lithology) indicator largely independent of fluid content.

Gassmann's equations assume (i) a closed, isotropic, homogeneous pore space with all pores communicating, (ii) the rock frame and pore fluid are in pressure equilibrium at the measurement frequency (no wave-induced local fluid flow), and (iii) the fluid does not chemically alter the frame. Violations of any of these produce the high-frequency / squirt corrections that Biot (1956) and later workers added.

Parameters

- **k_dry** (*scalar or ndarray*) – Dry-frame (drained) bulk modulus (Pa). Must be strictly positive and $\leq K_{\text{mineral}}$.

- **mu_dry** (*scalar or ndarray*) – Dry-frame shear modulus (Pa). Returned unchanged as `mu_sat`.
- **k_mineral** (*scalar or ndarray*) – Bulk modulus of the solid mineral phase (Pa). For quartz this is ~37 GPa, for calcite ~76.8 GPa. Composite mineralogies can be pre-averaged with *hill_average()*.
- **k_fluid** (*scalar or ndarray*) – Bulk modulus of the saturating pore fluid (Pa). Brine is ~2.2 GPa, oil ~1.0 GPa, gas 0.01-0.1 GPa. For multi-phase fluids, Reuss-average the individual fluid moduli first (Wood's law; Mavko et al. 2009 Section 6.1).
- **porosity** (*scalar or ndarray*) – Fractional porosity $\phi \in [0, 1]$.

Returns

`k_sat` (Pa) and `mu_sat` (Pa) broadcast to the common shape of the inputs.

Return type

GassmannResult

Raises

ValueError – If any modulus is non-positive, if `porosity` falls outside `[0, 1]`, or if `k_dry > k_mineral` anywhere (the dry frame cannot be stiffer than the mineral it is made of).

Notes

Limiting cases, verifiable by direct substitution:

- $\phi = 0$ – no pore space, $K_{\text{sat}} = K_s$.
- $K_{\text{dry}} = K_s$ – a solid mineral with no compliant pore space, $K_{\text{sat}} = K_s$.
- $K_f \to 0$ – a "fluid" as soft as vacuum, $K_{\text{sat}} \to K_{\text{dry}}$.

References

- Gassmann, F. (1951). Ueber die Elastizitaet poroeser Medien. *Vierteljahrsschrift der Naturforschenden Gesellschaft in Zuerich* 96, 1-23.
- Biot, M. A. (1956). Theory of propagation of elastic waves in a fluid-saturated porous solid. *J. Acoust. Soc. Am.* 28(2), 168-191 (low-frequency consistency with Gassmann).
- Mavko, G., Mukerji, T., & Dvorkin, J. (2009). *The Rock Physics Handbook*, 2nd ed., Section 6.3. Cambridge University Press.
- Smith, T. M., Sondergeld, C. H., & Rai, C. S. (2003). Gassmann fluid substitutions: A tutorial. *Geophysics* 68(2), 430-440.

`fwap.rockphysics.`**hill_average**(*moduli*, *fractions*)

Voigt-Reuss-Hill average: arithmetic mean of the Voigt and Reuss bounds.

$$M_{\text{VRH}} = \frac{1}{2}(M_V + M_R)$$

Commonly used as a pragmatic "best estimate" for the effective modulus of an isotropic composite when the component geometry is unknown; always lies between the true stiffness bounds, but has no deeper physical justification than "halfway between".

Parameters

- **moduli** (*ndarray, shape (N,)*)
- **fractions** (*ndarray, shape (N,)*) – Volume fractions, must sum to 1.0.

Return type
float

References

- Hill, R. (1952). The elastic behaviour of a crystalline aggregate. *Proceedings of the Physical Society of London A* 65, 349-354.
- Mavko, Mukerji & Dvorkin (2009), Section 4.1.

`fwap.rockphysics.`**`hornby_fracture_aperture`**(*reflection_coefficient*, *frequency_hz*, *stoneley_velocity*, *, *small_amplitude_approx=False*)

Fracture aperture from the Hornby et al. (1989) Stoneley reflection-coefficient inversion.

Hornby, Johnson, Winkler & Plumb (1989, *Geophysics* 54(10), 1274-1288) treat a horizontal open fluid-filled fracture intersecting the borehole as a thin compliant layer. In the rigid-frame, low-frequency, single-fracture limit their reflection coefficient is

$$|R(\omega)| \;=\; \frac{\omega L_0}{\sqrt{V_T^2 + \omega^2 L_0^2}},$$

where L_0 is the fracture aperture (m), $\omega = 2\pi f$ is the angular frequency, and V_T is the Stoneley phase velocity in the unfractured borehole. Inverting for L_0:

$$L_0 \;=\; \frac{V_T\,|R|}{\omega\,\sqrt{1-|R|^2}} \;=\; \frac{V_T\,|R|}{2\pi f\,\sqrt{1-|R|^2}}.$$

For small reflection coefficients the radical can be dropped, giving the small-amplitude approximation

$$L_0 \;\approx\; \frac{V_T\,|R|}{2\pi f},$$

which is exact to better than 5% for $|R| \le 0.3$.

3.9.1 Assumptions

- Single horizontal open fracture intersecting the borehole at a right angle. Inclined fractures pick up a `cos(theta)` factor that is not modelled here.
- Rigid frame (no matrix permeability). Permeable formations attenuate the Stoneley wave through Darcy flow as well as through fracture pumping; separating the two needs the full Hornby et al. (1989) eqs. (5)-(7) or the simplified Biot-Rosenbaum decomposition of Tang, Cheng & Toksoz (1991, *J. Acoust. Soc. Am.* 90(3), 1632-1646). Use this routine on the reflected-wave channel only – where the matrix contribution is small relative to the fracture pumping.
- Low frequency ($\omega L_0 / V_T \lesssim 1$). At higher frequencies the rigid-frame approximation breaks down and the full dispersive form must be used.
- Stoneley wavelength much larger than fracture aperture ($L_0 \ll V_T/f$). For typical sonic frequencies (1-5 kHz) this holds up to centimetre-scale apertures.

param reflection_coefficient
$|R| \in [0, 1)$. Output of `stoneley_reflection_coefficient()`.

type reflection_coefficient
ndarray or float

param frequency_hz
Stoneley dominant frequency at the reflection (Hz). Either a single scalar (one dominant frequency for the whole log) or a per-fracture array. Must be strictly positive.

type frequency_hz
float or ndarray

param stoneley_velocity
Stoneley phase velocity in the unfractured borehole (m/s) near the fracture. The reciprocal of the picked Stoneley slowness in a tight reference zone is the standard estimate; compute as `1.0 / s_Stoneley_ref`. Must be strictly positive.

type stoneley_velocity
float or ndarray

param small_amplitude_approx
If `True`, use $L_0 = V_T|R|/(2\pi f)$ instead of the full Hornby form. Saves the radical evaluation; differs by < 5% for $|R| \leq 0.3$.

type small_amplitude_approx
bool, default False

returns
Fracture aperture L_0 (m). Same shape as the broadcast of the inputs. `+inf` at depths where `|R| = 1` (the inversion saturates); the caller can mask these as catastrophic / non-physical.

rtype
ndarray

raises ValueError
If any reflection coefficient is outside `[0, 1]`, or any frequency / velocity is non-positive.

See also

stoneley_reflection_coefficient

Builds
math:|*R*| from incident / reflected pulse amplitudes.

stoneley_permeability_indicator
Slowness-shift indicator for matrix permeability and fractures.

stoneley_amplitude_fracture_indicator
Amplitude-deficit indicator (transmission-loss form; complementary to this reflection-coefficient inversion).

References

- Hornby, B. E., Johnson, D. L., Winkler, K. W., & Plumb, R. A. (1989). Fracture evaluation using reflected Stoneley-wave arrivals. *Geophysics* 54(10), 1274-1288.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, Section 4.5 (Stoneley reflection and aperture inversion in field practice).

Parameters

- **reflection_coefficient** (*ndarray*)
- **frequency_hz** (*float | ndarray*)
- **stoneley_velocity** (*float | ndarray*)
- **small_amplitude_approx** (*bool*)

Return type
ndarray

`fwap.rockphysics.`**`reuss_average`**(*moduli*, *fractions*)

Reuss (harmonic, isostress) average of a composite modulus.

Gives the *lower* (softer) bound on the effective modulus of a mixture of N isotropic components at arbitrary volume fractions:

$$M_R \;=\; \left(\sum_i \frac{f_i}{M_i}\right)^{-1}$$

The Reuss bound is the exact answer when the components are arranged as layers perpendicular to the loading direction (isostress), and is therefore the worst-case stiffness for any isotropic mixing geometry.

Parameters

- **moduli** (*ndarray, shape (N,)*) – Per-component modulus of the same type (all bulk, or all shear). Units are preserved (Pa in, Pa out).
- **fractions** (*ndarray, shape (N,)*) – Volume fractions, must sum to 1.0 to within 1e-3 and all be non-negative.

Returns
Effective modulus in the same unit as `moduli`.

Return type
float

References

- Reuss, A. (1929). Berechnung der Fliessgrenze von Mischkristallen auf Grund der Plastizitaetsbedingung fuer Einkristalle. *Zeitschrift fuer Angewandte Mathematik und Mechanik* 9, 49-58.
- Mavko, Mukerji & Dvorkin (2009), *The Rock Physics Handbook*, 2nd ed., Section 4.1.

`fwap.rockphysics.`**`stoneley_amplitude_fracture_indicator`**(*amplitude_observed*, *amplitude_reference*)

Dimensionless Stoneley-amplitude fracture / permeability indicator.

Returns the fractional Stoneley amplitude *deficit* relative to a tight (unfractured, low-permeability) reference zone:

$$\beta_{\mathrm{ST}}(d) \;=\; 1 \;-\; \frac{A_{\mathrm{ST,obs}}(d)}{A_{\mathrm{ST,ref}}}.$$

The Stoneley wave attenuates as it crosses a fracture or permeable interval – energy radiates into the formation through Darcy flow in the rock matrix and through fracture-pumping (oscillatory fluid motion in and out of fracture apertures) at fractures intersecting the borehole wall (Tang & Cheng 2004, sect. 5.2; Hornby, Johnson, Winkler & Plumb 1989, *Geophysics* 54(10), 1274-1288). The fractional amplitude deficit therefore flags the same fractures and permeable zones as `stoneley_permeability_indicator()`, but with **complementary noise characteristics**: amplitude attenuation responds primarily to the *loss* of acoustic energy along the wavetrain, while the slowness-shift indicator responds primarily to the dynamic poroelastic delay. Combining the two (e.g. as a coincidence flag, or by averaging after rank- standardisation) is more robust than either alone.

Important: like the slowness-shift companion, this is an **uncalibrated, dimensionless** indicator – a depth-by-depth rank-ordering of fracture / permeability strength, not an absolute permeability or fracture aperture. Conversion to SI quantities needs a calibration tied to the tool's source frequency, borehole-fluid viscosity / bulk modulus, formation Stoneley impedance, and (for fractures) the fracture-aperture distribution; in practice it is tuned against a known interval (image log, core, mudlog) rather than computed from first principles. The closed-form low-frequency expressions are in Hornby et al. (1989), eqs. (3)-(7) (single-fracture reflection / transmission) and Tang & Cheng (2004), sect. 5.2 (matrix- permeability transmission loss).

Parameters

- **amplitude_observed** (*ndarray or float*) – Per-depth Stoneley-wave amplitude. Typically *fwap.picker.ModePick.amplitude* for the "Stoneley" mode gathered across depths, or the "AMPST" column of a log set written via *fwap.picker.track_to_log_curves()*. Must be non-negative.
- **amplitude_reference** (*float or ndarray*) – Tight-reference Stoneley amplitude. Either a single value (a hand-picked tight zone) or a per-depth baseline (e.g. a median-filtered or low-pass-smoothed version of the observed log). Must be strictly positive.

Returns

Dimensionless indicator, typically in [-0.05, 0.9]. Positive values flag permeable / fractured intervals (lower observed amplitude than reference); near-zero values are zones that match the reference. Negative values indicate either an unusual amplification (rare; resonance from thin-bed multiples) or that the reference itself was lower than this depth.

Return type

ndarray

Raises

ValueError – If amplitude_observed is negative anywhere or amplitude_reference is non-positive anywhere.

See also

stoneley_permeability_indicator

The slowness-shift companion. Run both and combine as a coincidence flag for robust fracture / permeability picks.

References

- Hornby, B. E., Johnson, D. L., Winkler, K. W., & Plumb, R. A. (1989). Fracture evaluation using reflected Stoneley-wave arrivals. *Geophysics* 54(10), 1274-1288.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, Section 5.2.
- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 1 (Stoneley amplitude as permeability / fracture proxy). Editions Technip.

fwap.rockphysics.**stoneley_permeability_indicator**(*slowness_observed*, *slowness_reference*)

Dimensionless Stoneley-wave permeability indicator.

Returns the fractional Stoneley-slowness shift relative to a tight (low-permeability) reference zone:

$$\alpha_{\mathrm{ST}}(d) = \frac{s_{\mathrm{ST,obs}}(d)}{s_{\mathrm{ST,ref}}} - 1$$

The Stoneley wave in a fluid-filled borehole loses energy into Darcy flow in a permeable formation, which slows its phase velocity (increases its slowness). At low frequencies the fractional shift scales linearly with formation permeability (Tang & Cheng 2004, Section 5.1), so this indicator rank-orders zones by permeability.

Important: this is an **uncalibrated, dimensionless** indicator, not an absolute permeability in m^2 or mD. Converting to SI permeability needs a calibration constant that depends on the tool's source frequency, borehole fluid viscosity and bulk modulus, and tube-wave phase velocity in the reference zone – quantities that in practice are tuned against a known-permeability interval in the well (core data or a mudlog), not computed from first principles. The simple closed-form Biot low-frequency formula is given in Kostek & Johnson (1992), Section 3, but

its parameters vary enough with local fluid conditions that calibration against a reference zone is the standard workflow.

Parameters

- **slowness_observed** (*ndarray or float*) – Per-depth Stoneley-wave slowness (s/m). Typically *fwap.picker.ModePick.slowness* for the "Stoneley" mode gathered across depths, or the "Stoneley" column of a LAS log read via *fwap.io.read_las()*.
- **slowness_reference** (*float or ndarray*) – Tight-reference Stoneley slowness (s/m). Either a single value (one tight zone) or a per-depth reference baseline (e.g., a low-pass-smoothed version of the observed log).

Returns

Dimensionless indicator, typically in [-0.01, 0.5]. Positive values flag permeable intervals; near-zero values are zones that match the reference. Negative values indicate the reference itself was not tight relative to that depth or that SNR issues have corrupted the pick.

Return type

ndarray

References

- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, Section 5.1.
- Kostek, S., & Johnson, D. L. (1992). The interaction of tube waves with borehole fractures, Part I: Numerical models. *Geophysics* 57(6), 784-795.
- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 1 (Stoneley amplitude as permeability / fracture proxy). Editions Technip.

fwap.rockphysics.**stoneley_reflection_coefficient**(*amplitude_incident*, *amplitude_reflected*)

Stoneley reflection coefficient $|R|$ from incident and reflected pulse amplitudes.

Returns

$$|R| \;=\; |A_{\text{reflected}}| \,/\, |A_{\text{incident}}|,$$

clipped to [0, 1] – numerical drift in the up/down-going decomposition can push noisy estimates a few percent above 1, but a physical reflection coefficient is bounded by unity.

Prerequisite: the caller has separated the up-going (reflected) and down-going (incident) Stoneley arrivals at each fracture candidate. That decomposition is non-trivial and depends on the array geometry: f-k filtering (*fwap.wavesep.fk_filter()*) or tau-p filtering (*fwap.wavesep.tau_p_filter()*) at the Stoneley apparent slowness gives the two propagation directions; their peak-to-peak amplitudes near each picked fracture depth are the inputs here.

Parameters

- **amplitude_incident** (*ndarray or float*) – Incident (down-going) and reflected (up-going) Stoneley pulse amplitudes near each fracture candidate. Same shape; the function broadcasts. Sign is irrelevant – the function takes absolute values.
- **amplitude_reflected** (*ndarray or float*) – Incident (down-going) and reflected (up-going) Stoneley pulse amplitudes near each fracture candidate. Same shape; the function broadcasts. Sign is irrelevant – the function takes absolute values.

Returns

$|R| \in [0, 1]$. 1 is full reflection (impossible for a real fracture; flag for QC); 0 is a non-reflecting depth.

Return type
ndarray

Raises
ValueError – If any incident amplitude is non-positive (the ratio is undefined).

`fwap.rockphysics.`**`voigt_average`**(*moduli*, *fractions*)

Voigt (arithmetic, isostrain) average of a composite modulus.

Gives the *upper* (stiffer) bound on the effective modulus:

$$M_V = \sum_i f_i M_i$$

Exact when the components are arranged as layers parallel to the loading direction (isostrain); the best-case stiffness for any isotropic mixing geometry.

Parameters

- **moduli** (*ndarray, shape (N,)*)
- **fractions** (*ndarray, shape (N,)*) – Volume fractions, must sum to 1.0.

Return type
float

References

- Voigt, W. (1889). Ueber die Beziehung zwischen den beiden Elastizitaetskonstanten isotroper Koerper. *Annalen der Physik und Chemie* 38, 573-587.
- Mavko, Mukerji & Dvorkin (2009), Section 4.1.

`fwap.rockphysics.`**`vp_vs_ratio`**(*vp*, *vs*)

Vp/Vs ratio, a common lithology / fluid-content indicator.

Typical values: ~1.5 for gas sands, ~1.7-1.8 for clean water sands and oil sands, >1.9 for shales and carbonates.

Parameters

- **vp** (*ndarray*)
- **vs** (*ndarray*)

Return type
ndarray

`fwap.rockphysics.`**`vs_from_stoneley_slow_formation`**(*slowness_stoneley*, *rho*, *, *rho_fluid*, *v_fluid*)

Formation shear velocity V_S from low-frequency Stoneley slowness.

The primary sonic-only V_S estimator for **slow formations** ($V_S < V_{\text{fluid}}$), where the formation has no critically-refracted S head wave to pick on a monopole gather (Paillet & Cheng 1991, Ch. 3) and the pseudo- Rayleigh / guided trapped mode does not exist (`fwap.synthetic.pseudo_rayleigh_dispersion()` raises in this regime). The Stoneley wave's low-frequency phase velocity carries the formation shear-modulus information through the classical White (1983) tube-wave formula

$$S_{\text{ST}}^2 = \frac{1}{V_f^2} + \frac{\rho_f}{\mu},$$

where $\mu = \rho V_S^2$ is the formation shear modulus. Inverting for V_S:

$$V_S = \sqrt{\frac{\rho_f}{\rho\,(S_{\text{ST}}^2 - 1/V_f^2)}}.$$

The formula is identical to the `fwap.anisotropy.stoneley_horizontal_shear_modulus()` expression after dividing by ρ and taking the square root; the difference is interpretation. For an isotropic formation $C_{66} = \mu$ and this function returns V_S directly. For a VTI formation use `fwap.anisotropy.stoneley_horizontal_shear_modulus()` to get C_{66} (the horizontal shear modulus, *not* equal to ρV_{Sv}^2 in general).

3.9.2 Assumptions

- Low-frequency limit (Stoneley pulse well below the dipole- flexural and pseudo-Rayleigh cutoffs). At higher frequencies the Stoneley wave is dispersive in slow formations and a full cylindrical-mode solver is needed (Paillet & Cheng 1991, Ch. 4). The slowness here should therefore be the picked Stoneley slowness in a low-pass-filtered or low-frequency band.
- Inviscid borehole fluid; centred tool; circular borehole cross-section. Tool-eccentricity and mudcake corrections (Tang & Cheng 2004, sect. 5.2) are not applied.
- Isotropic formation. For VTI shales the inverted modulus is C_{66}, not μ; the function will then systematically under- or over-estimate the *vertical* V_{Sv} depending on the sign of Thomsen γ. Use `fwap.anisotropy.thomsen_gamma_from_logs()` when an independent dipole shear log is available to detect this case.

param slowness_stoneley
Per-depth low-frequency Stoneley slowness (s/m). Must be strictly greater than `1 / v_fluid` everywhere – the Stoneley wave is always slower than the unconfined fluid wave because the formation loads it.

type slowness_stoneley
ndarray or float

param rho
Per-depth formation bulk density (kg/m^3) from the bulk- density log (typically the `RHOB` curve).

type rho
ndarray or float

param rho_fluid
Borehole-fluid density (kg/m^3). Brine ~ 1000-1100; oil ~ 800-900; gas / foam << 1000.

type rho_fluid
float

param v_fluid
Borehole-fluid acoustic velocity (m/s). Brine ~ 1500; oil ~ 1300; gas << 1000.

type v_fluid
float

returns
Formation shear velocity V_S (m/s), broadcast to the common shape of `slowness_stoneley` and `rho`.

rtype
ndarray

raises ValueError
If any input is non-positive, or if any Stoneley slowness is at or below the fluid slowness `1 / v_fluid` (the inversion is undefined there).

See also

fwap.anisotropy.stoneley_horizontal_shear_modulus
Same physics in the VTI shale case; returns C_{66}.

fwap.anisotropy.thomsen_gamma_from_logs
When a dipole shear log is also available, combines both to flag VTI anisotropy and give the Thomsen γ directly.

References

- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes*, Chapter 3. CRC Press (Stoneley low-f velocity as slow-formation Vs estimator).
- White, J. E. (1983). *Underground Sound: Application of Seismic Waves.* Elsevier, Section 5.5 (tube-wave formula).
- Norris, A. N. (1990). The speed of a tube wave. *J. Acoust. Soc. Am.* 87(1), 414-417.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, Section 5.2.

Parameters

- **slowness_stoneley** (*ndarray*)
- **rho** (*ndarray*)
- **rho_fluid** (*float*)
- **v_fluid** (*float*)

Return type
ndarray

3.10 fwap.geomechanics

Geomechanics indices derived from elastic moduli.

Closes the workflow gap between the elastic-moduli output of `fwap.rockphysics.elastic_moduli()` and the Workflow-3 deliverables listed in Mari et al. (1994), Part 3: the petrophysical / completion- engineering indices that drive **sanding prediction**, **hydraulic- fracture design**, and **fracture-reservoir characterisation**.

The functions in this module are deliberately thin: each takes raw arrays (Young's modulus in Pa, Poisson's ratio, etc.) and applies a single well-cited formula, with the parameter defaults pinned to the original publications. A one-call wrapper `geomechanics_indices()` operates on a `fwap.rockphysics.ElasticModuli` and returns the full `GeomechanicsIndices` bundle.

3.10.1 Caveats

Sonic-derived Young's modulus is the **dynamic** modulus; published correlations for unconfined compressive strength, brittleness windows and similar geomechanics quantities were calibrated against **static** core measurements. The two differ by a lithology- and stress-dependent factor (Mavko, Mukerji & Dvorkin 2009, sect. 5.5; Eissa & Kazi 1988). A dynamic-to-static correction should be applied upstream of this module when absolute numbers matter; the indices here remain useful as relative depth-by-depth rankings without it.

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*, Part 3 (dipole-sonic reservoir characterisation as input for HF design / sanding prediction). Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Rickman, R., Mullen, M. J., Petre, J. E., Grieser, W. V., & Kundert, D. (2008). A practical use of shale petrophysics for stimulation design optimization: All shale plays are not clones of the Barnett Shale. *SPE Annual Technical Conference and Exhibition*, SPE 115258.
- Lacy, L. L. (1997). Dynamic rock mechanics testing for optimized fracture designs. *SPE Annual Technical Conference and Exhibition*, SPE 38716.
- Eaton, B. A. (1969). Fracture gradient prediction and its application in oilfield operations. *J. Petroleum Technology* 21(10), 1353-1360.
- Bratli, R. K., & Risnes, R. (1981). Stability and failure of sand arches. *Society of Petroleum Engineers Journal* 21(2), 236-248.
- Mavko, G., Mukerji, T., & Dvorkin, J. (2009). *The Rock Physics Handbook*, 2nd ed., Chapter 5 (static vs dynamic moduli). Cambridge University Press.

Functions

`brittleness_index_rickman`(young_pa, poisson, *)	Rickman brittleness index from Young's modulus and Poisson's ratio.
`closure_stress`(poisson, sigma_v_pa, *[, ...])	Minimum horizontal (closure) stress under a uniaxial-strain assumption.
`fracability_index`(young_pa, poisson, *[, ...])	Fracability index from Young's modulus and Poisson's ratio.
`geomechanics_indices`(moduli, *[, ...])	One-call geomechanics layer on top of `ElasticModuli`.
`overburden_stress`(depth, density, *[, ...])	Overburden stress $\sigma_v(z)$ by trapezoidal integration of a density log.
`sand_stability_indicator`(shear_pa, *[, ...])	Boolean sand-stability flag from the formation shear modulus.
`unconfined_compressive_strength`(young_pa, *)	Empirical UCS estimate from Young's modulus.

Classes

`GeomechanicsIndices`(brittleness, ...[, ...])	Per-sample geomechanics indices derived from elastic moduli.

class `fwap.geomechanics.`**`GeomechanicsIndices`**(*brittleness*, *fracability*, *ucs*, *sand_stability*, *closure_stress=None*)

Bases: `object`

Per-sample geomechanics indices derived from elastic moduli.

Output of `geomechanics_indices()`. All arrays are aligned on the same depth axis as the input `ElasticModuli`; closure stress is `None` when the caller did not supply an overburden profile.

Parameters

- **`brittleness`** (*ndarray*)
- **`fracability`** (*ndarray*)

- **ucs** (*ndarray*)
- **sand_stability** (*ndarray*)
- **closure_stress** (*ndarray | None*)

brittleness

Rickman brittleness index in `[0, 1]`.

Type
ndarray

fracability

Fracability index (alias of brittleness for sonic-only inputs; see *fracability_index()*).

Type
ndarray

ucs

Estimated unconfined compressive strength (Pa).

Type
ndarray

sand_stability

`True` where the shear modulus exceeds the sanding threshold (formation is stable).

Type
ndarray of bool

closure_stress

Minimum horizontal (closure) stress (Pa). `None` when `sigma_v_pa` was not passed to *geomechanics_indices()*.

Type
ndarray or None

brittleness: ndarray

closure_stress: ndarray | None = None

fracability: ndarray

sand_stability: ndarray

ucs: ndarray

`fwap.geomechanics.`**brittleness_index_rickman**(*young_pa*, *poisson*, *, *e_min_pa=10000000000.0*, *e_max_pa=80000000000.0*, *nu_min=0.15*, *nu_max=0.4*)

Rickman brittleness index from Young's modulus and Poisson's ratio.

Per Rickman et al. (2008, SPE 115258), the brittleness index is the average of two normalised, clipped attributes:

$$\mathrm{BI} \;=\; \tfrac{1}{2}\,(\hat{E} + \hat{\nu}),$$

$$\hat{E} \;=\; \mathrm{clip}\!\left(\frac{E - E_{\min}}{E_{\max} - E_{\min}},\; 0,\; 1\right),$$

$$\hat{\nu} \;=\; \mathrm{clip}\!\left(\frac{\nu_{\max} - \nu}{\nu_{\max} - \nu_{\min}},\; 0,\; 1\right).$$

High Young's modulus and low Poisson's ratio map to high brittleness; the result is in `[0, 1]`.

Parameters

- **young_pa** (*scalar or ndarray*) – Young's modulus (Pa).
- **poisson** (*scalar or ndarray*) – Poisson's ratio (dimensionless).
- **e_min_pa** (*float*) – Normalisation bounds for Young's modulus (Pa). Defaults are `RICKMAN_E_MIN_PA` and `RICKMAN_E_MAX_PA`, matching Rickman et al. (2008) Table 1 (1-8 Mpsi).
- **e_max_pa** (*float*) – Normalisation bounds for Young's modulus (Pa). Defaults are `RICKMAN_E_MIN_PA` and `RICKMAN_E_MAX_PA`, matching Rickman et al. (2008) Table 1 (1-8 Mpsi).
- **nu_min** (*float*) – Normalisation bounds for Poisson's ratio. Defaults are `RICKMAN_NU_MIN` and `RICKMAN_NU_MAX` (0.15-0.40).
- **nu_max** (*float*) – Normalisation bounds for Poisson's ratio. Defaults are `RICKMAN_NU_MIN` and `RICKMAN_NU_MAX` (0.15-0.40).

Returns
Brittleness index in `[0, 1]`, broadcast to the common shape of the inputs.

Return type
ndarray

Raises
ValueError – If `e_max_pa <= e_min_pa` or `nu_max <= nu_min`.

`fwap.geomechanics.`**closure_stress**(*poisson*, *sigma_v_pa*, *, *pore_pressure_pa=0.0*, *biot_alpha=1.0*)

Minimum horizontal (closure) stress under a uniaxial-strain assumption.

Eaton (1969) gives the closure stress in a tectonically relaxed basin as

$$\sigma_h - \alpha P_p = \frac{\nu}{1-\nu}(\sigma_v - \alpha P_p),$$

i.e. the effective horizontal stress is a Poisson-fraction of the effective vertical stress. Solving for the absolute closure stress σ_h:

$$\sigma_h = \frac{\nu}{1-\nu}(\sigma_v - \alpha P_p) + \alpha P_p.$$

This is the standard first-order closure-stress estimate used to seed hydraulic-fracture design (Mavko et al. 2009, sect. 8.7); in tectonically active basins the result is a lower bound and a fault-and-stress correction is required.

Parameters

- **poisson** (*scalar or ndarray*) – Poisson's ratio at each depth (dimensionless). Must satisfy `0 <= poisson < 1`. The lower bound rules out auxetic materials, which are physically valid but produce negative effective horizontal stresses under this formula and are not the use case the Eaton model targets. The upper bound is the formula's removable singularity at $\nu = 1$.
- **sigma_v_pa** (*scalar or ndarray*) – Vertical (overburden) stress at each depth (Pa). Use *`overburden_stress()`* to compute this from a density log, or pass an externally measured value.
- **pore_pressure_pa** (*scalar or ndarray, default 0.0*) – Pore pressure at each depth (Pa). The formula is calibrated in effective-stress terms, so a non-zero pore pressure is needed whenever the formation is over- or under-pressured relative to hydrostatic. Default 0.0 collapses the formula to the dry-rock case.
- **biot_alpha** (*float, default 1.0*) – Biot poro-elastic coefficient. `1.0` (default) is the textbook upper bound for a soft frame; tight rocks may be as low as 0.7-0.8. Carried as a scalar because the constant is dominated by lithology rather than depth-by-depth variability.

Returns
Closure stress (Pa), broadcast to the common shape of the inputs.

Return type
ndarray

Raises
ValueError – If `poisson >= 1` or `poisson < 0` anywhere.

`fwap.geomechanics.`**`fracability_index`**(*young_pa*, *poisson*, ***, *e_min_pa=10000000000.0*, *e_max_pa=80000000000.0*, *nu_min=0.15*, *nu_max=0.4*)

Fracability index from Young's modulus and Poisson's ratio.

For sonic-only inputs the fracability index used in the hydraulic- fracture-design literature reduces to the Rickman brittleness index (Rickman et al. 2008): a stiff, low-Poisson formation fractures more readily and supports a propped fracture better than a ductile, high-Poisson one. Other published fracability formulations layer in mineralogy, in-situ stress contrast or fracture toughness; without those auxiliary inputs the brittleness index is the standard sonic-derived proxy.

Parameters and return value identical to `brittleness_index_rickman()`; this function is provided as a named alias so the call site documents whether the index is being used as a *brittleness* (rock-property) measure or a *fracability* (completion-design) measure.

Parameters

- **young_pa** (*ndarray*)
- **poisson** (*ndarray*)
- **e_min_pa** (*float*)
- **e_max_pa** (*float*)
- **nu_min** (*float*)
- **nu_max** (*float*)

Return type
ndarray

`fwap.geomechanics.`**`geomechanics_indices`**(*moduli*, ***, *sigma_v_pa=None*, *pore_pressure_pa=0.0*, *biot_alpha=1.0*, *ucs_model='lacy_sandstone'*, *sand_threshold_pa=5000000000.0*, *e_min_pa=10000000000.0*, *e_max_pa=80000000000.0*, *nu_min=0.15*, *nu_max=0.4*)

One-call geomechanics layer on top of `ElasticModuli`.

Computes the four sonic-derivable Workflow-3 deliverables – brittleness / fracability, sand-stability flag, UCS, and (when overburden is supplied) closure stress – in a single pass and returns them as a `GeomechanicsIndices` bundle.

Parameters

- **moduli** (`ElasticModuli`) – Output of `fwap.rockphysics.elastic_moduli()` (or any equivalent dataclass instance).
- **sigma_v_pa** (*ndarray, optional*) – Vertical (overburden) stress at each depth (Pa). When omitted the closure-stress field on the result is `None`; all other indices are independent of overburden.
- **pore_pressure_pa** (*scalar or ndarray, default 0.0*) – Pore pressure at each depth (Pa). Forwarded to `closure_stress()`.

- **biot_alpha** (*float, default 1.0*) – Biot coefficient. Forwarded to *closure_stress()*.
- **ucs_model** (str, default "lacy_sandstone") – Empirical UCS correlation. Forwarded to *unconfined_compressive_strength()*.
- **sand_threshold_pa** (*float, default 5 GPa*) – Shear-modulus threshold for the sand-stability flag.
- **e_min_pa** (*float*) – Rickman normalisation bounds. Defaults match *brittleness_index_rickman()*.
- **e_max_pa** (*float*) – Rickman normalisation bounds. Defaults match *brittleness_index_rickman()*.
- **nu_min** (*float*) – Rickman normalisation bounds. Defaults match *brittleness_index_rickman()*.
- **nu_max** (*float*) – Rickman normalisation bounds. Defaults match *brittleness_index_rickman()*.

Return type
GeomechanicsIndices

`fwap.geomechanics.`**`overburden_stress`**(*depth*, *density*, *, *surface_value_pa=0.0*, *g=9.80665*)

Overburden stress $\sigma_v(z)$ by trapezoidal integration of a density log.

$$\sigma_v(z) \;=\; \sigma_v(z_0) \;+\; g \int_{z_0}^{z} \rho(z')\, dz'.$$

Trapezoidal integration is exact for piecewise-linear density interpolation; for typical 0.1-0.5 m sample spacing the error is well below the density-log measurement uncertainty.

Parameters

- **depth** (*ndarray, shape (n_depth,)*) – Depth (m), strictly increasing.
- **density** (*ndarray, shape (n_depth,)*) – Bulk density (kg/m^3) at each depth. Must be non-negative.
- **surface_value_pa** (*float, default 0.0*) – Boundary condition $\sigma_v(z_0)$. Use a non-zero value to seed the integration with the overburden above the first logged sample (e.g. obtained from a regional density-depth model).
- **g** (*float, default 9.80665*) – Standard gravity (m/s^2).

Returns
Overburden stress (Pa) at each depth.

Return type
ndarray, shape (n_depth,)

Raises
ValueError – If `depth` is not strictly increasing, if `density` is negative, or if the two arrays have different length.

`fwap.geomechanics.`**`sand_stability_indicator`**(*shear_pa*, *, *threshold_pa=5000000000.0*)

Boolean sand-stability flag from the formation shear modulus.

Bratli & Risnes (1981) – and the practical Schlumberger field guideline that grew out of it – treats a shear modulus below ~5 GPa as a strong indicator that grain-arch failure (and therefore sand production) is plausible at typical drawdowns. The default `SAND_STABILITY_SHEAR_THRESHOLD_PA` encodes that 5 GPa rule of thumb.

The flag returned here is a soft binary indicator – callers that want a smooth ranking can take the shear modulus itself and threshold downstream.

Parameters

- **shear_pa** (*scalar or ndarray*) – Shear modulus (Pa).
- **threshold_pa** (*float, default 5e9*) – Threshold below which the formation is flagged sand-prone.

Returns

`True` where the formation is **stable** (shear modulus at or above the threshold), `False` where it is sand-prone. Boundary convention: `mu == threshold_pa` is treated as stable. `True` for sand-prone is *not* the convention here so the flag composes naturally with other "is OK" gates.

Return type

ndarray of bool

`fwap.geomechanics.`**`unconfined_compressive_strength`**(*young_pa*, *, *model='lacy_sandstone'*)

Empirical UCS estimate from Young's modulus.

3.10.2 Available models

`"lacy_sandstone"` (default)

Lacy (1997, SPE 38716) sandstone correlation, in the form compiled by Chang et al. (2006, *J. Petr. Sci. Eng.* 51, eq. 7):

$$\mathrm{UCS}\,[\mathrm{MPa}] \;=\; 0.278\,E^2 \;+\; 2.458\,E,$$

with E in GPa. Inputs and outputs are converted to Pa internally so the API stays SI.

3.10.3 Static vs dynamic

The Lacy correlation was fit on **static** core-derived Young's moduli, while sonic-log-derived `young_pa` is the **dynamic** modulus, which is generally larger by a factor of 1.5 to 3 for porous rocks (Mavko et al. 2009, sect. 5.5). Without a dynamic-to- static correction the returned UCS is an upper bound; the depth-by-depth ranking is still informative.

param young_pa

Young's modulus (Pa). Must be non-negative.

type young_pa

scalar or ndarray

param model

Empirical model to use. New models may be added in future versions; existing model names are preserved.

type model

`"lacy_sandstone"`, default `"lacy_sandstone"`

returns

UCS in Pa, broadcast to the shape of `young_pa`.

rtype

ndarray

raises ValueError

If `model` is unknown or `young_pa` is negative.

Parameters

- **young_pa** (*ndarray*)
- **model** (*Literal['lacy_sandstone']*)

Return type
ndarray

3.11 fwap.cylindrical

Elastic surface-wave speeds and cylindrical-mode dispersion models.

Part 3 of Mari et al. (1994) treats the borehole flexural mode with a phenomenological dispersion law in *fwap.synthetic* (dipole_flexural_dispersion()). That model's high-frequency asymptote is 1.25 * shear_slowness, a round number chosen for convenience rather than physics. This module provides a Poisson-ratio-dependent replacement via the Rayleigh surface-wave speed, which is the correct high-frequency limit for a vacuum-loaded free surface.

For a fluid-filled borehole the true high-frequency limit is the Scholte interface-wave speed at the fluid-solid boundary, which is a few percent below the Rayleigh speed; the fluid-loading correction depends on the density ratio and is usually small (Paillet & Cheng, 1991, Ch. 4). A full cylindrical-Biot modal determinant solver (Schmitt, 1988) is future work – fwap deliberately ships only models whose assumptions are explicit and whose published limits match to within percent-level precision.

3.11.1 Functions

- *rayleigh_speed()*
- *flexural_dispersion_physical()*

References

- Rayleigh, Lord (1885). On waves propagating along the plane surface of an elastic solid. *Proceedings of the London Mathematical Society* 17, 4-11.
- Viktorov, I. A. (1967). *Rayleigh and Lamb Waves: Physical Theory and Applications.* Plenum (closed-form Rayleigh approximation).
- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes*, Chapter 4. CRC Press (cylindrical-mode theory).
- Schmitt, D. P. (1988). Shear-wave logging in elastic formations. *Journal of the Acoustical Society of America* 84(6), 2230-2244.

Functions

flexural_dispersion_physical(vp, vs[, ...])	Dipole flexural dispersion with a Rayleigh-speed high-frequency asymptote.
rayleigh_speed(vp, vs)	Rayleigh surface-wave speed for a vacuum-loaded elastic half-space.

fwap.cylindrical.**flexural_dispersion_physical**(*vp*, *vs*, *a_borehole=0.1*)

Dipole flexural dispersion with a Rayleigh-speed high-frequency asymptote.

Returns a callable s(f) that interpolates between the low-frequency limit (formation shear slowness 1/vs) and the high-frequency limit (the Rayleigh slowness 1/V_R, computed by *rayleigh_speed()* from vp and vs) with

the two-parameter form

$$s(f) \;=\; s_{\text{low}} + (s_{\text{high}} - s_{\text{low}})\,\frac{x^2}{1+x^2}$$

where $x = f/f_c$ and $f_c = V_s/(2\pi a)$ is the geometric cut-off frequency. This is the same shape as the phenomenological `fwap.synthetic.dipole_flexural_dispersion()` but with a physically-grounded `s_high` that depends on the Poisson's ratio instead of the round-number `1.25 / vs`.

The interpolation form matches the smoothed-step model in Mari et al. (1994), Part 3, eq. (3.18); the Rayleigh asymptote and Scholte correction are derived from the cylindrical-mode dispersion relations of Paillet & Cheng (1991), Ch. 4, eqs. (4.27) and (4.31).

3.11.2 Caveats

The Rayleigh speed is the vacuum-loaded free-surface limit. For a fluid-filled borehole the correct asymptote is the *Scholte* speed at the fluid-solid interface, which is a few percent lower. The fluid-loading correction depends on the fluid-to-solid density ratio and the fluid speed; for typical brine-filled boreholes it reduces the high-frequency slowness by roughly `1 - 3 %` relative to the Rayleigh limit.

For shear-slowness estimation accurate to better than a few percent in the high-frequency regime, the full cylindrical-Biot modal determinant (Schmitt, 1988; Paillet & Cheng, 1991) should be solved – that remains future work in `fwap`.

param vp
Compressional wave speed of the formation (m/s).

type vp
float

param vs
Shear wave speed of the formation (m/s).

type vs
float

param a_borehole
Borehole radius (m).

type a_borehole
float, default 0.1

returns
Array-in / array-out mapping of frequency (Hz) to phase slowness (s/m).

rtype
Callable[[ndarray], ndarray]

See also

fwap.synthetic.dipole_flexural_dispersion
The phenomenological 1.25 factor model. Use this one when you want a Poisson-grounded asymptote.

Parameters

- **vp** (*float*)
- **vs** (*float*)
- **a_borehole** (*float*)

Return type
Callable[[*ndarray*], *ndarray*]

`fwap.cylindrical.rayleigh_speed(vp, vs)`

Rayleigh surface-wave speed for a vacuum-loaded elastic half-space.

Solves the classical Rayleigh equation

$$(2-\xi)^2 \;=\; 4\sqrt{(1-\xi\cdot(V_s/V_p)^2)\;(1-\xi)}$$

for $\xi = (V_R/V_s)^2 \in (0,1)$ by bracketing the non-trivial root with `scipy.optimize.brentq()`. Returns $V_R = V_s\sqrt{\xi}$.

Typical values: for Poisson's ratio $\nu = 0.25$ (`Vp/Vs = sqrt(3)`) the ratio `V_R / V_s` is approximately 0.9194; for $\nu = 0.33$ it is approximately 0.9325.

Parameters

- **vp** (*float*) – Compressional wave speed (m/s). Must satisfy `vp > vs`.
- **vs** (*float*) – Shear wave speed (m/s). Must be positive.

Returns
Rayleigh wave speed (m/s), always strictly less than `vs`.

Return type
float

Raises
ValueError – If `vs <= 0` or `vp <= vs` (physically invalid for an isotropic solid).

References

- Rayleigh (1885), Proc. London Math. Soc. 17, 4-11.

3.12 fwap.lwd

Logging-While-Drilling (LWD) acoustic processing – phenomenological layer.

LWD acoustic tools record waveforms while a steel drill collar is still inside the borehole. The collar supports its own propagating modes (flexural, longitudinal, quadrupole) that contaminate the formation arrivals – Tang & Cheng (2004), sect. 2.4-2.5 emphasise this as the defining processing problem of the LWD era.

This module provides a **phenomenological** layer for that contamination:

- *`lwd_collar_mode()`* returns a pre-configured `Mode` representing the collar arrival as a Gabor-windowed wave-train at a fixed apparent slowness. This is *not* a true cylindrical-Biot collar-mode forward solver – it captures the dominant contamination signature in slowness-time pickers (a strong, narrow-band arrival at the collar slowness) without modelling the collar's full dispersion.
- *`synthesize_lwd_gather()`* is a convenience wrapper around *`fwap.synthetic.synthesize_gather()`* that adds the collar arrival on top of a list of formation modes.
- *`notch_slowness_band()`* removes a slowness band from a multichannel gather using a tau-p forward / mask / inverse round- trip. Its main use case is collar-mode rejection: notch the band around the known collar slowness and re-run the picker / dispersion estimator on the cleaned record.

For a quantitative LWD inversion (e.g. recovering shear slowness from the LWD quadrupole flexural mode in a slow formation) a true layered- cylindrical-Biot solver over (collar, mud annulus, formation) is required. That is research-grade work and outside the scope of fwap today; the phenomenological layer here is enough to test the processing chain on collar-contaminated synthetics.

References

- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods.* Elsevier, sect. 2.4-2.5 (LWD modes; quadrupole source as the practical solution to collar-mode contamination).
- Aron, J., Chang, S. K., Codazzi, D., Dworak, R., Hsu, K., Lau, T., Minerbo, G., & Yogeswaren, E. (1994). Real-time sonic logging while drilling in hard and soft rocks. *SEG Technical Program Expanded Abstracts*, 13, 1-4.
- Kinoshita, T., Endo, T., Iwasaki, S., Saito, S., Mori, K., Inoue, T., Yoshikawa, R., & Hiraga, A. (2008). Real-time deep-shear sonic-while- drilling in fast and slow formations. *SPWLA 49th Annual Logging Symposium*.

Functions

`lwd_collar_mode`(*[, apparent_slowness, f0, ...])	Pre-configured `Mode` representing the LWD steel-collar arrival.
`lwd_quadrupole_priors`()	Per-mode prior windows tuned for an LWD quadrupole-stacked record.
`notch_slowness_band`(data, dt, offsets, ...)	Notch out an apparent-slowness band from a multichannel gather.
`quadrupole_stack`(data, azimuths, *[, ...])	Receiver-side m=2 (quadrupole) projection of a ring-array record.
`synthesize_lwd_gather`(geom, formation_modes, *)	Build a multichannel LWD gather: formation arrivals + a planted steel-collar wavetrain.
`synthesize_quadrupole_lwd_gather`(*[, n_rec, ...])	Synthetic quadrupole-source LWD ring-array gather.

Classes

`QuadrupoleRingGather`(data, dt, ...)	Output of `synthesize_quadrupole_lwd_gather()`.

class `fwap.lwd.`**`QuadrupoleRingGather`**(*data*, *dt*, *axial_offsets*, *azimuths*, *tool_radius*, *source_azimuth*)

Bases: `object`

Output of `synthesize_quadrupole_lwd_gather()`.

Mirrors `fwap.dip.AzimuthalGather` (the dip-measurement ring array) but with a single axial offset and a quadrupole (m=2) azimuthal pattern.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **axial_offsets** (*ndarray*)
- **azimuths** (*ndarray*)
- **tool_radius** (*float*)
- **source_azimuth** (*float*)

data

Per-receiver waveforms. Receivers lie on a transverse ring at the same axial offset; per-receiver amplitude carries the $\cos(2(\theta_i - \phi_{\text{src}}))$ quadrupole modulation.

Type

ndarray, shape (*n_rec*, *n_samples*)

dt

Sampling interval (s).

Type
float

axial_offsets

Source-to-receiver axial offset (m). All entries equal for a ring array.

Type
ndarray, shape (*n_rec*,)

azimuths

Per-receiver azimuth (rad), uniformly spaced in $[0, 2\pi)$.

Type
ndarray, shape (*n_rec*,)

tool_radius

Ring radius (m).

Type
float

source_azimuth

Quadrupole-source orientation (rad). The cos-pattern axis.

Type
float

axial_offsets: ndarray

azimuths: ndarray

data: ndarray

dt: float

source_azimuth: float

tool_radius: float

`fwap.lwd.`**`lwd_collar_mode`**(*, *apparent_slowness=0.00030303030303030303*, *f0=12000.0*, *sigma=0.00015*, *amplitude=1.5*, *intercept=0.0*)

Pre-configured `Mode` representing the LWD steel-collar arrival.

Phenomenological model: a non-dispersive Gabor wavetrain at a fixed apparent slowness, narrow-band around `f0`. The collar mode is in reality weakly dispersive and modally complex (longitudinal + flexural + screw modes overlap on a monopole record), but for processing-chain testing the dominant contamination signature on a slowness-time-coherence map is a single strong narrow-band peak at the collar slowness; that is what this `Mode` produces.

Use the returned `Mode` either alongside formation modes in a call to *`fwap.synthetic.synthesize_gather()`*, or via the convenience wrapper *`synthesize_lwd_gather()`*.

Parameters

- **`apparent_slowness`** (*`float, default 1/3300 (~92 us/ft)`*) – Collar apparent slowness (s/m). The LWD steel-collar longitudinal/flexural mode is typically observed in the 80-130 us/ft band; 92 us/ft is the published median across recent generations of tools.
- **`f0`** (*`float, default 12 kHz`*) – Source-signature centre frequency (Hz). Current-generation LWD acoustic tools sit in the 8-15 kHz band.

- **sigma** (*float, default 1.5e-4 s*) – Gabor envelope width (s). The narrow envelope captures the finite-duration character of the collar wavetrain compared to a broadband formation P or S impulse.
- **amplitude** (*float, default 1.5*) – Per-trace amplitude multiplier. The default places the collar arrival at roughly the same level as the strongest formation mode, the contamination regime where rejection processing is most useful.
- **intercept** (*float, default 0.0*) – Absolute time offset (s) at which the collar wavetrain appears at zero offset; carry the same value as the formation modes for a coherent gather.

Returns
Ready to drop into the `modes` list of *fwap.synthetic.synthesize_gather()* or *synthesize_lwd_gather()*.

Return type
Mode

`fwap.lwd.`**`lwd_quadrupole_priors`**`()`

Per-mode prior windows tuned for an LWD quadrupole-stacked record.

The deliverable from a quadrupole LWD measurement is the formation **screw / shear** slowness; the dominant contamination is the steel-collar quadrupole mode at higher slowness (typically 110-180 us/ft per Tang & Cheng 2004 sect. 2.5). The two priors below cover that pair.

Time ordering: the collar quadrupole is faster than typical formation-shear slownesses in the slow-formation use case (V_S < V_collar_quad), so it arrives first; `CollarQuadrupole` is therefore picked first (`order=0`) with `FormationShear` after it (`order=1`). On fast-formation records that ordering can be reversed; pass a custom priors dict in that regime.

Returns
Mode -> prior-window dict, suitable to pass as the `priors` argument of *fwap.picker.pick_modes()* / `track_modes()`. Mode names use the LWD-specific labels (`"CollarQuadrupole"`, `"FormationShear"`) rather than the monopole P/S/Stoneley set so the user does not confuse a quadrupole-tool log with a monopole-tool log.

Return type
dict[str, dict[str, float]]

`fwap.lwd.`**`notch_slowness_band`**`(`*data, dt, offsets, slow_min, slow_max, *, n_slowness=181, slowness_pad_factor=1.5, taper_width=0.1*`)`

Notch out an apparent-slowness band from a multichannel gather.

Inverse of *fwap.wavesep.tau_p_filter()*: forward-transforms the gather to (tau, p), zeroes the slowness band `[slow_min, slow_max]` (with cosine-tapered edges), and projects the residual back to (t, x) via the tau-p adjoint. The result is the input gather with energy at apparent slownesses inside the notch band attenuated.

Primary use case: **LWD collar-mode rejection.** When the LWD steel-collar arrival sits at a known apparent slowness (typically 80-130 us/ft per `DEFAULT_COLLAR_SLOWNESS_S_PER_M`), notching that band before running the picker / dispersion estimator removes the dominant collar contamination from the record while preserving the formation arrivals at slownesses outside the band.

Notes

- Implementation: forward-transform to (tau, p), keep **only** the notched band via a cosine-tapered band-PASS mask, project that back to (t, x) via the tau-p adjoint, and **subtract** from the original data. This subtraction route preserves any signal at slownesses *outside* the tau-p grid (e.g. Stoneley arrivals at slownesses well above the collar band), which a naive band-stop mask + adjoint round-trip would destroy because the adjoint cannot reconstruct out-of-grid energy.

- Uses the tau-p adjoint rather than the LSQR inverse; the inverse does not commute with masking and would amplify the band rather than reconstruct it.
- Amplitudes inside the band are attenuated by ~50 % (the slant-stack point-spread function is non-unitary). Deeper rejection would need a multi-pass implementation or a different filter family.
- The notch is **band-stop**, not "ideal brick-wall"; choose `taper_width` according to how clean a separation is needed.

Parameters

- **data** (*ndarray, shape (n_rec, n_samples)*) – Multichannel gather (real-valued).
- **dt** (*float*) – Time sampling interval (s).
- **offsets** (*ndarray, shape (n_rec,)*) – Source-to-receiver offsets (m). Need not be uniform.
- **slow_min** (*float*) – Notched apparent-slowness band edges (s/m). Must satisfy `0 < slow_min < slow_max`.
- **slow_max** (*float*) – Notched apparent-slowness band edges (s/m). Must satisfy `0 < slow_min < slow_max`.
- **n_slowness** (*int, default 181*) – Number of slowness samples in the tau-p grid.
- **slowness_pad_factor** (*float, default 1.5*) – Extra slowness coverage outside the notch, as a multiple of the notch width. Carried over from `tau_p_filter()` for consistent grid behaviour.
- **taper_width** (*float, default 0.1*) – Half-cosine taper width on each notch edge, expressed as a fraction of the notch width. `0.0` gives a hard rectangular notch.

Returns
Gather with the notched band attenuated.

Return type
ndarray, shape (*n_rec*, *n_samples*)

Raises
ValueError – If `slow_min` / `slow_max` are mis-ordered or non- positive.

See also

fwap.wavesep.tau_p_filter
Band-pass companion (keep a slowness band, drop everything else).

`fwap.lwd.`**quadrupole_stack**(*data*, *azimuths*, *, *source_azimuth=0.0*)

Receiver-side m=2 (quadrupole) projection of a ring-array record.

Given a ring-array record `data[i, j]` with receiver azimuths `azimuths[i]`, the stack

$$x_q(t) \;=\; \sum_i \cos(2(\theta_i - \phi_{\text{src}}))\; x_i(t)$$

isolates the m=2 azimuthal component of the field. The orthogonal m=0 (Stoneley / monopole) and m=1 (flexural / dipole) patterns sum to (approximately) zero through the $\cos(2\theta)$ projection on a uniformly-spaced ring – this is the operational rejection of monopole and dipole contamination that a quadrupole receiver geometry provides (Tang & Cheng 2004, sect. 2.5.3).

Returns the single stacked trace; downstream processing (picker, dispersion estimator, attenuation logger) treats it like any other single-trace recording.

Parameters

- **data** (*ndarray, shape (n_rec, n_samples)*) – Ring-array waveforms. Typically *QuadrupoleRingGather.data*.
- **azimuths** (*ndarray, shape (n_rec,)*) – Per-receiver azimuth (rad).
- **source_azimuth** (*float, default 0.0 rad*) – Quadrupole-source orientation (rad). Pass the value used in *synthesize_quadrupole_lwd_gather()* (or the value recovered from a cross-dipole calibration on real data).

Returns
Quadrupole-stacked trace.

Return type
ndarray, shape (*n_samples*,)

Raises
ValueError – If `data` is not 2-D, or if `azimuths` length does not match `data.shape[0]`.

`fwap.lwd.`**`synthesize_lwd_gather`**(*geom*, *formation_modes*, ***, *collar_amplitude=1.5*, *collar_slowness=0.00030303030303030303*, *collar_f0=12000.0*, *collar_intercept=0.0*, *collar_sigma=0.00015*, *noise=0.02*, *seed=None*)

Build a multichannel LWD gather: formation arrivals + a planted steel-collar wavetrain.

Convenience wrapper around *fwap.synthetic.synthesize_gather()* that adds an *lwd_collar_mode()* to the supplied formation modes before synthesis. Lets users exercise the picker / wave- separation / dispersion-estimation pipeline against a synthetic LWD record without hand-building the collar `Mode`.

Parameters

- **geom** (`ArrayGeometry`) – Tool geometry (offsets, sampling) – see *fwap.synthetic.ArrayGeometry*.
- **formation_modes** (*sequence of* `Mode`) – Formation arrivals to synthesise (e.g. the output of *fwap.synthetic.monopole_formation_modes()*). The collar arrival is appended to this list.
- **collar_amplitude** (*float*)
- **collar_slowness** (*float*)
- **collar_f0** (*float*)
- **collar_intercept** (*float*)
- **collar_sigma** (*float*)
- **noise** (*float*)
- **seed** (*int | None*)

Return type
ndarray

:param : :param collar_sigma: Parameters of the planted collar `Mode`. See

lwd_collar_mode() for the conventions.

Parameters

- **noise** (*float, default 0.02*) – Per-trace Gaussian noise standard deviation as a fraction of the noise-free RMS, identical to `fwap.synthetic.synthesize_gather()`.
- **seed** (*int or None*) – Seed for the noise RNG. Pass `int` for reproducible synthetics (test gathers) and `None` for non-deterministic noise (visual demos).
- **geom** (`ArrayGeometry`)
- **formation_modes** (*Sequence[Mode]*)
- **collar_amplitude** (*float*)
- **collar_slowness** (*float*)
- **collar_f0** (*float*)
- **collar_intercept** (*float*)
- **collar_sigma** (*float*)

Returns
Common-source LWD gather, real-valued, `float64`.

Return type
ndarray, shape (*n_rec*, *n_samples*)

`fwap.lwd.`**`synthesize_quadrupole_lwd_gather`**(*, *n_rec=8*, *n_samples=2048*, *dt=1e-05*, *tool_offset=3.0*, *tool_radius=0.1*, *formation_slowness=0.0004*, *formation_f0=6000.0*, *formation_amplitude=1.0*, *formation_intercept=0.0*, *source_azimuth=0.0*, *include_collar=True*, *collar_slowness=0.00030303030303030303*, *collar_f0=12000.0*, *collar_amplitude=1.0*, *collar_sigma=0.00015*, *noise=0.02*, *seed=None*)

Synthetic quadrupole-source LWD ring-array gather.

Builds a ring of `n_rec` receivers at a single axial offset `tool_offset` and a tool radius `tool_radius`. A quadrupole source at azimuth `source_azimuth` excites the formation **screw / quadrupole-flexural** mode at `formation_slowness`, plus (optionally) the LWD steel-collar quadrupole mode at `collar_slowness`. Both modes carry the $\cos(2(\theta_i - \phi_{\text{src}}))$ azimuthal pattern that is the distinguishing signature of an m=2 source / receiver geometry.

Tang & Cheng (2004), sect. 2.5 emphasises this geometry as the practical solution to LWD collar-mode contamination: the quadrupole collar mode is dispersive and well-separated from the formation flexural / screw modes, so a quadrupole source + quadrupole receiver projection (via `quadrupole_stack()`) gives a substantially cleaner shear measurement than the monopole or dipole equivalent in a slow formation.

Parameters

- **n_rec** (*int, default 8*) – Number of azimuthal receivers (must be >= 4 to resolve the m=2 pattern by Nyquist).
- **n_samples** (*int, default 2048*)
- **dt** (*float, default 1e-5 s*)
- **tool_offset** (*float, default 3.0 m*) – Source-to-receiver axial offset.
- **tool_radius** (*float, default 0.1 m*) – Ring radius (typical LWD tool).
- **formation_slowness** (*float, default 1/2500 s/m*) – Formation screw / quadrupole-flexural slowness (= formation shear slowness in the low-frequency limit; Tang & Cheng 2004 sect. 2.5.4).

- **formation_f0** (*float*) – Formation-mode source-signature centre frequency, per-trace amplitude (before quadrupole modulation), and zero-offset arrival time.
- **formation_amplitude** (*float*) – Formation-mode source-signature centre frequency, per-trace amplitude (before quadrupole modulation), and zero-offset arrival time.
- **formation_intercept** (*float*) – Formation-mode source-signature centre frequency, per-trace amplitude (before quadrupole modulation), and zero-offset arrival time.
- **source_azimuth** (*float, default 0.0 rad*) – Quadrupole-source orientation. Per-receiver amplitudes carry a $\cos(2(\theta_i - \phi_{\mathrm{src}}))$ modulation.
- **include_collar** (*bool, default True*) – Plant the steel-collar quadrupole arrival on top of the formation mode.
- **collar_slowness** (*float*) – Steel-collar quadrupole parameters; defaults match *lwd_collar_mode()*.
- **collar_f0** (*float*) – Steel-collar quadrupole parameters; defaults match *lwd_collar_mode()*.
- **collar_amplitude** (*float*) – Steel-collar quadrupole parameters; defaults match *lwd_collar_mode()*.
- **collar_sigma** (*float*) – Steel-collar quadrupole parameters; defaults match *lwd_collar_mode()*.
- **noise** (*float, default 0.02*)
- **seed** (*int or None*) – Per-trace Gaussian noise + RNG seed; same conventions as *fwap.synthetic.synthesize_gather()*.

Returns
`data`, `azimuths` and the geometry that *quadrupole_stack()* consumes.

Return type
QuadrupoleRingGather

Raises
ValueError – If `n_rec < 4` (the m=2 pattern needs at least four samples per cycle by Nyquist).

3.13 fwap.synthetic

Synthetic multi-receiver borehole gathers.

Provides the canonical P / S / Stoneley monopole gather and the phenomenological dipole flexural dispersion law used by the demos and unit tests. The default *ArrayGeometry* matches the original Schlumberger Array Sonic specification (8 receivers, 10 ft source-to- first-receiver, 6 in spacing) used throughout Mari et al. (1994).

References

- Mari, J.-L., Coppens, F., Gavin, P., & Wicquart, E. (1994). *Full Waveform Acoustic Data Processing*. Editions Technip, Paris. ISBN 978-2-7108-0664-6.
- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes*, Chapter 4. CRC Press (cylindrical-mode dispersion).
- Schmitt, D. P. (1988). Shear-wave logging in elastic formations. *Journal of the Acoustical Society of America* 84(6), 2230-2244.

Functions

dipole_flexural_dispersion(vs[, a_borehole])	Phenomenological flexural-mode dispersion `s(f)`.
gabor(t, f0, t0, sigma)	Gabor (Morlet-like) wavelet useful for narrow-band modes.
monopole_formation_modes([vp, vs, ...])	Canonical P / S / Stoneley mode list for a monopole sonic tool.
pseudo_rayleigh_dispersion(vs[, v_fluid, ...])	Phenomenological pseudo-Rayleigh dispersion `s(f)`.
ricker(t, f0[, t0])	Zero-phase Ricker wavelet, peak frequency `f0` (Hz).
synthesize_gather(geom, modes[, noise, seed])	Build a common-source multi-receiver gather `(n_rec, n_samples)`.

Classes

ArrayGeometry([n_rec, tr_offset, dr, dt, ...])	Axial multi-receiver borehole array geometry.
Mode(name, slowness, f0[, amplitude, ...])	A single wave mode (P-head, S-head, Stoneley, flexural, ...).

class `fwap.synthetic.`**`ArrayGeometry`**(*n_rec=8*, *tr_offset=3.0*, *dr=0.1524*, *dt=1e-05*, *n_samples=2048*)

Bases: `object`

Axial multi-receiver borehole array geometry.

The geometry is semantically immutable: changing `n_rec`, `dr`, etc. on a constructed instance would invalidate the cached *`offsets`* / *`t`* arrays, which is almost never what the caller intended. `offsets` and `t` are therefore computed on first access and cached via `functools.cached_property` so repeated use inside a processing loop incurs no recomputation cost.

Parameters

- **`n_rec`** (*`int`*)
- **`tr_offset`** (*`float`*)
- **`dr`** (*`float`*)
- **`dt`** (*`float`*)
- **`n_samples`** (*`int`*)

`n_rec`

Number of receivers.

Type
int

`tr_offset`

Transmitter-to-first-receiver offset (m).

Type
float

`dr`

Inter-receiver spacing (m).

Type
float

dt

Sampling interval (s).

Type
float

n_samples

Samples per trace.

Type
int

dr: float = 0.1524

dt: float = 1e-05

classmethod from_imperial(*n_rec=8*, *tr_offset_ft=10.0*, *dr_in=6.0*, *dt=1e-05*, *n_samples=2048*)

Convenience constructor using imperial units (ft, in).

Matches the original Schlumberger Array Sonic specification (8 receivers, 10 ft source-to-first-receiver, 6 in spacing).

Parameters

- **n_rec** (*int*)
- **tr_offset_ft** (*float*)
- **dr_in** (*float*)
- **dt** (*float*)
- **n_samples** (*int*)

Return type
ArrayGeometry

n_rec: int = 8

n_samples: int = 2048

property offsets: ndarray

Source-to-receiver offsets (m), shape `(n_rec,)`.

classmethod schlumberger_array_sonic(*dt=1e-05*, *n_samples=2048*)

Canonical Schlumberger Array Sonic geometry.

8 receivers, 10 ft (3.048 m) source-to-first-receiver offset, 6 in (0.1524 m) inter-receiver spacing – the reference tool geometry throughout Mari et al. (1994). The dataclass default constructor already encodes these numbers in metric; this factory documents the intent at the call site.

Parameters

- **dt** (*float*)
- **n_samples** (*int*)

Return type
ArrayGeometry

property t: ndarray

Time axis (s), shape `(n_samples,)`.

tr_offset: float = 3.0

class `fwap.synthetic.`**Mode**(*name*, *slowness*, *f0*, *amplitude=1.0*, *intercept=0.0*, *dispersion=None*, *src_delay=0.0*, *rec_delay=0.0*, *wavelet='ricker'*, *sigma=0.0002*)

Bases: `object`

A single wave mode (P-head, S-head, Stoneley, flexural, …).

If `dispersion` is provided (callable `f -> slowness(f)`), the arrival is synthesised in the frequency domain with that dispersion law; otherwise it is a non-dispersive wavelet arriving at `intercept + offset * slowness + (src_delay + rec_delay)`.

Notes

The `dispersion` callable is called with a NumPy array of positive frequencies and must return a same-shape array of phase slownesses (s/m). A purely scalar callable will fail at runtime; wrap it with `np.vectorize` at the call site if needed. The built-in `dipole_flexural_dispersion()` already satisfies the array contract.

Parameters

- **name** (*str*)
- **slowness** (*float*)
- **f0** (*float*)
- **amplitude** (*float*)
- **intercept** (*float*)
- **dispersion** (*Callable[[ndarray], ndarray] | None*)
- **src_delay** (*float*)
- **rec_delay** (*float*)
- **wavelet** (*str*)
- **sigma** (*float*)

amplitude: float = 1.0

dispersion: Callable[[ndarray], ndarray] | None = None

f0: float

intercept: float = 0.0

name: str

rec_delay: float = 0.0

sigma: float = 0.0002

slowness: float

src_delay: float = 0.0

wavelet: str = 'ricker'

`fwap.synthetic.`**`dipole_flexural_dispersion`**(*vs*, *a_borehole=0.1*)

Phenomenological flexural-mode dispersion `s(f)`.

Low-frequency limit equals the true shear slowness (`1/vs`), high- frequency limit is 25% above. This is *not* a full cylindrical Biot model; for production use one should solve the 3x3 modal determinant in Hankel/Bessel functions (Paillet & Cheng, 1991, chap. 4; Schmitt, 1988, *JASA* 84(6), 2230-2244).

The returned callable accepts a NumPy array of frequencies and returns a same-shape array of slownesses.

Parameters

- **vs** (*float*)
- **a_borehole** (*float*)

Return type
Callable[[*ndarray*], *ndarray*]

`fwap.synthetic.`**`gabor`**(*t*, *f0*, *t0*, *sigma*)

Gabor (Morlet-like) wavelet useful for narrow-band modes.

Parameters

- **t** (*ndarray*)
- **f0** (*float*)
- **t0** (*float*)
- **sigma** (*float*)

Return type
ndarray

`fwap.synthetic.`**`monopole_formation_modes`**(*vp=4000.0*, *vs=2300.0*, *v_stoneley=1400.0*, *f_p=15000.0*, *f_s=10000.0*, *f_st=3000.0*, *p_amp=1.0*, *s_amp=1.5*, *st_amp=2.0*, ***, *v_fluid=1500.0*, *a_borehole=0.1*, *f_pr=None*, *pr_amp=1.5*)

Canonical P / S / Stoneley mode list for a monopole sonic tool.

When `f_pr` is given (any positive frequency, in Hz), a fourth pseudo-Rayleigh / guided arrival is appended at the band-centre phase slowness predicted by `pseudo_rayleigh_dispersion()` – i.e. as a non-dispersive Ricker so that the test gather is pickable by ordinary slowness-time coherence. The mode only exists in fast formations (`vs > v_fluid`); the dispersion factory raises if that condition is violated.

Callers who want a *dispersive* pseudo-Rayleigh wavetrain (so plain STC sees a smeared peak that requires `narrow_band_stc()` or `dispersive_stc()` to pick cleanly) should construct the fourth `Mode` by hand, passing `dispersion=pseudo_rayleigh_dispersion(vs, v_fluid, a_borehole)` explicitly.

Parameters

- **vp** (*float*)
- **vs** (*float*)
- **v_stoneley** (*float*)
- **f_p** (*float*)
- **f_s** (*float*)
- **f_st** (*float*)
- **p_amp** (*float*)
- **s_amp** (*float*)

- **st_amp** (*float*)
- **v_fluid** (*float*)
- **a_borehole** (*float*)
- **f_pr** (*float | None*)
- **pr_amp** (*float*)

Return type
list[Mode]

`fwap.synthetic.`**`pseudo_rayleigh_dispersion`**(*vs*, *v_fluid=1500.0*, *a_borehole=0.1*)

Phenomenological pseudo-Rayleigh dispersion `s(f)`.

Pseudo-Rayleigh is a guided trapped mode in the fluid-filled borehole that exists only in *fast* formations (`vs > v_fluid`). At its low-frequency cutoff its phase velocity equals the formation shear velocity (so phase slowness = `1/vs`); at high frequency its phase velocity asymptotes to the fluid velocity (phase slowness = `1/v_fluid`). The cutoff frequency scale is set by the borehole radius via `fc = vs / (2 pi a_borehole)`.

Parameters

- **vs** (*float*) – Formation shear velocity (m/s).
- **v_fluid** (*float, default 1500.0*) – Borehole-fluid acoustic velocity (m/s).
- **a_borehole** (*float, default 0.1*) – Borehole radius (m).

Returns

- *Callable that accepts a NumPy array of frequencies and returns a*
- *same-shape array of phase slownesses (s/m).*

Raises
ValueError – If `vs <= v_fluid` (slow formation – pseudo-Rayleigh does not exist).

Return type
Callable[[*ndarray*], *ndarray*]

Notes

This phenomenological law has the same rational-Lorentzian shape as `dipole_flexural_dispersion()` – a smooth interpolation between the cutoff and the high-frequency asymptote – and is *not* a full cylindrical Biot model. For quantitative work see Paillet & Cheng (1991) chapter 4 or Schmitt (1988) for the 3x3 Hankel/Bessel modal determinant.

`fwap.synthetic.`**`ricker`**(*t*, *f0*, *t0=0.0*)

Zero-phase Ricker wavelet, peak frequency `f0` (Hz).

Parameters

- **t** (*ndarray*)
- **f0** (*float*)
- **t0** (*float*)

Return type
ndarray

`fwap.synthetic.`**`synthesize_gather`**(*geom*, *modes*, *noise=0.02*, *seed=None*)

Build a common-source multi-receiver gather `(n_rec, n_samples)`.

Parameters

- **geom** (`ArrayGeometry`) – Tool geometry (offsets, sampling).
- **modes** (*sequence of* `Mode`) – Modes to synthesise; their slownesses, intercepts, and wavelets are summed coherently before noise is added.
- **noise** (*float, default 0.02*) – Per-trace Gaussian noise standard deviation, expressed as a fraction of the noise-free RMS of the synthesised gather. Set to 0 for a clean noise-free output.
- **seed** (*int or None, default None*) – Seed for the noise generator (see `numpy.random.Generator`). Pass an integer (e.g. `seed=0`) for reproducible noise – critical for unit tests and any property-based or regression check. Pass `None` for fresh non-deterministic noise on every call; use that for visual demos where exact byte-for- byte reproducibility is not required.

Returns

Common-source gather, real-valued, `float64`.

Return type

ndarray, shape `(n_rec, n_samples)`

3.14 fwap.io

I/O wrappers for the formats sonic data arrives in.

- *`read_las()`* / *`write_las()`* – LAS (Log ASCII Standard), the most common exchange format for *log curves* (the Vp/Vs/Stoneley outputs of the processing chain). Wraps lasio.
- *`read_dlis()`* / *`write_dlis()`* – DLIS (Digital Log Interchange Standard, API RP66 v1), the binary equivalent of LAS used by modern wireline acquisition systems. Read uses dlisio; write uses dliswriter.
- *`read_segy()`* – SEG-Y is the standard format for *waveform* dumps (the inputs to the processing chain). Wraps segyio.

All four underlying libraries (`lasio`, `dlisio`, `dliswriter`, `segyio`) are core fwap dependencies, so the LAS / DLIS / SEG-Y readers and writers are always available.

References

- Canadian Well Logging Society (1991). *Log ASCII Standard (LAS) Version 2.0.*
- American Petroleum Institute (1991). *Recommended Practices for Exploration and Production Data Digital Interchange (RP66 v1).*
- Society of Exploration Geophysicists (2017). *SEG Y rev 2.0 Data Exchange Format.*
- Mari, J.-L., & Vergniault, C. (2018). *Well Seismic Surveying and Acoustic Logging*, Chapter 1 (log formats). EDP Open.

Functions

`read_dlis`(path, *[, logical_file_index, ...])	Read one frame of a DLIS (RP66 v1) file into a *`DlisCurves`*.
`read_las`(path)	Read a LAS file into a *`LasCurves`* container.

continues on next page

Table 26 – continued from previous page

read_segy(path, *[, offset_header])	Read a SEG-Y file into a *SegyGather*.
write_dlis(path, depth, curves, *[, ...])	Write an fwap-derived log set out as a DLIS (RP66 v1) file.
write_las(path, depth, curves, *[, ...])	Write an fwap-derived log set out as a LAS file.
write_segy(path, data, dt[, offsets, ...])	Write a multichannel gather to a SEG-Y rev 1 file (IEEE float).

Classes

DlisCurves(depth, curves, units, well, step, ...)	A DLIS frame loaded into memory.
LasCurves(depth, curves, units, well, step)	A LAS file loaded into memory.
SegyGather(data, dt, offsets, n_traces, ...)	A SEG-Y file loaded into memory as a single gather.

class `fwap.io.`**`DlisCurves`**(*depth*, *curves*, *units*, *well*, *step*, *frame_name*, *index_type*)

Bases: `object`

A DLIS frame loaded into memory.

Mirrors *LasCurves* so curves can move between LAS and DLIS without the caller having to translate field names.

Parameters

- **depth** (*ndarray*)
- **curves** (*dict[str, ndarray]*)
- **units** (*dict[str, str]*)
- **well** (*dict[str, str]*)
- **step** (*float*)
- **frame_name** (*str*)
- **index_type** (*str*)

depth

The frame's index channel (typically borehole depth, in the unit declared by the channel header – usually metres or feet).

Type
ndarray, shape (n_depth,)

curves

One entry per non-index channel, keyed by channel name. Arrays have the same length as `depth`. `NaN` round-trips natively as the IEEE-754 NaN bit pattern.

Type
dict[str, ndarray]

units

Per-channel units from the DLIS header; empty string when no unit was set.

Type
dict[str, str]

well

Origin metadata, re-keyed to the LAS-2.0 mnemonics (`WELL`, `COMP`, `FLD`, `PROD`, `UWI`) so that the same dict can be passed to *`write_las()`*. Origin fields without a LAS analogue are dropped.

Type
dict[str, str]

step

Frame index spacing (`Frame.spacing` in DLIS); `NaN` if the file does not declare a constant spacing.

Type
float

frame_name

Name of the DLIS frame the data was read from.

Type
str

index_type

DLIS frame index type (e.g. `"BOREHOLE-DEPTH"`, `"VERTICAL-DEPTH"`, `"TIME"`). Empty string when the frame has no declared index type.

Type
str

curves: dict[str, ndarray]

depth: ndarray

frame_name: str

index_type: str

step: float

units: dict[str, str]

well: dict[str, str]

class `fwap.io.`**LasCurves**(*depth*, *curves*, *units*, *well*, *step*)

Bases: `object`

A LAS file loaded into memory.

Parameters

- **depth** (*ndarray*)
- **curves** (*dict[str, ndarray]*)
- **units** (*dict[str, str]*)
- **well** (*dict[str, str]*)
- **step** (*float*)

depth

The depth axis (usually `DEPT` or `DEPTH`), in the units declared by the LAS header (typically metres or feet).

Type
ndarray, shape (n_depth,)

curves

One entry per non-depth curve, keyed by mnemonic (e.g. `"GR"`, `"DT"`, `"RHOB"`). Arrays have the same length as `depth`. `NaN` marks null values in the LAS file.

Type
dict[str, ndarray]

units

Per-curve units from the LAS header; empty string when the unit field was blank.

Type
dict[str, str]

well

The `~Well` section as a flat dict (keys like `"WELL"`, `"COMPANY"`, `"FLD"`, `"SRVC"`).

Type
dict[str, str]

step

Constant sampling step from the LAS header (`STEP`). May be `NaN` when the file does not use a uniform step.

Type
float

curves: dict[str, ndarray]

depth: ndarray

step: float

units: dict[str, str]

well: dict[str, str]

class `fwap.io.`**SegyGather**(*data*, *dt*, *offsets*, *n_traces*, *n_samples*, *textual_header*)

Bases: `object`

A SEG-Y file loaded into memory as a single gather.

Parameters

- **data** (*ndarray*)
- **dt** (*float*)
- **offsets** (*ndarray | None*)
- **n_traces** (*int*)
- **n_samples** (*int*)
- **textual_header** (*str*)

data

Trace data, one row per trace, in the file's original sample order.

Type
ndarray, shape (*n_traces*, *n_samples*)

dt

Sample interval (s). Read from the binary header's `Interval` field, which is stored in microseconds; we convert to seconds.

Type
float

offsets

Source-to-receiver offsets (m) extracted from a trace header field (`offset` by default). `None` if the header field was all zeros, which is common for synthetic files.

Type
ndarray, shape (*n_traces*,) or None

n_traces

Number of traces in the file.

Type
int

n_samples

Samples per trace.

Type
int

textual_header

The 3200-byte EBCDIC textual file header, decoded to ASCII.

Type
str

data: ndarray

dt: float

n_samples: int

n_traces: int

offsets: ndarray | None

textual_header: str

`fwap.io.`**read_dlis**(*path*, *, *logical_file_index=0*, *frame_index=0*)

Read one frame of a DLIS (RP66 v1) file into a `DlisCurves`.

A DLIS file is a container that may hold multiple Logical Files, each of which may hold multiple Frames; `logical_file_index` and `frame_index` select one. The defaults read the first frame of the first logical file, which covers the common single-pass / single-frame case.

All channels are returned as `float64` NumPy arrays. `NaN` is preserved as IEEE-754 NaN. Multi-dimensional channels (e.g. array waveforms) are skipped: only scalar-per-sample curves are loaded – the dataclass mirrors `LasCurves`, which assumes scalar curves.

Parameters

- **path** (*str*) – Filesystem path to a DLIS file.
- **logical_file_index** (*int, default 0*) – Which logical file to read.
- **frame_index** (*int, default 0*) – Which frame inside that logical file to read.

Return type
DlisCurves

Raises
IndexError – If `logical_file_index` or `frame_index` is out of range for the file.

`fwap.io.read_las(path)`

Read a LAS file into a *LasCurves* container.

All curves are returned as `float64` NumPy arrays with null values replaced by `NaN`.

Parameters
path (*str*) – Filesystem path to a LAS file. LAS 2.0 and 3.0 are both supported via `lasio`.

Return type
LasCurves

`fwap.io.read_segy(path, *, offset_header='offset')`

Read a SEG-Y file into a *SegyGather*.

This is the minimal reader needed to feed the *fwap.coherence* / *fwap.dispersion* processing chain from real sonic data. It assumes the file contains one gather (all traces share a source position); callers with multi-gather data should split on `FieldRecord` before reading with this function.

Parameters

- **path** (*str*) – Filesystem path to a SEG-Y rev 1 or rev 2 file.
- **offset_header** (str, default `"offset"`) – Name of the `segyio.TraceField` attribute from which to read per-trace source-to-receiver offsets. Common choices: `"offset"` (standard offset field, bytes 37-40), `"GroupX"` / `"SourceX"` for receiver / source positions. If the chosen header is all zero the returned `offsets` attribute is `None`.

Return type
SegyGather

Raises
AttributeError – If `offset_header` is not a valid `segyio.TraceField` name.

`fwap.io.write_dlis(path, depth, curves, *, depth_unit='m', well_name='', well=None, units=None, frame_name='MAIN', index_type='BOREHOLE-DEPTH', origin_name='FWAP')`

Write an fwap-derived log set out as a DLIS (RP66 v1) file.

Mirror of *write_las()*: `depth` becomes the index channel of a single frame named `frame_name` inside a single logical file, and `curves` are added as non-index channels. Units for the standard fwap mnemonics (`DTP`, `DTS`, `DTST`, `COH*`, `VPVS`, `Q`, `K`, `MU`, `E`, `NU`) are filled in from the same internal table used by *write_las()*.

Parameters

- **path** (*str*) – Output DLIS path.
- **depth** (*ndarray, shape (n_depth,)*) – Depth axis. Becomes the frame's index channel and must match the first dimension of every curve array.
- **curves** (*mapping from str to ndarray*) – `{channel_name: ndarray}`. Each array must have shape `(n_depth,)`; `NaN` is preserved as IEEE-754 NaN.
- **depth_unit** (str, default `"m"`) – Unit for the depth axis.
- **well_name** (*str, default empty*) – Convenience for the `"WELL"` entry, identical to placing `"WELL"` in `well`.

- **well** (*mapping from str to str, optional*) – Origin metadata. Keys use the LAS-2.0 mnemonics (`"WELL"`, `"COMP"`, `"FLD"`, `"PROD"`, `"UWI"`) so the same dict works with `write_las()`. Other keys are silently ignored.
- **units** (*mapping from str to str, optional*) – Per-channel unit override, keyed by channel name. Missing entries fall back to the built-in table and then to empty.
- **frame_name** (str, default `"MAIN"`) – Name of the DLIS frame written.
- **index_type** (str, default `"BOREHOLE-DEPTH"`) – DLIS frame index type. Common values: `"BOREHOLE-DEPTH"`, `"VERTICAL-DEPTH"`, `"TIME"`.
- **origin_name** (str, default `"FWAP"`) – Name of the Origin record. Identifies the producer of the logical file; rarely user-visible.

Raises
ValueError – If any curve has a length different from `depth`.

Return type
None

Notes

DLIS standard units are restricted to a small whitelist (see RP66 v1 Appendix B). Non-canonical units like `us/ft` are accepted by `dliswriter` but trigger an info-level log entry on its `dliswriter.utils.internal.validator_enum` logger; this is suppressed during the write so the file is produced silently.

`fwap.io.`**write_las**(*path*, *depth*, *curves*, *, *depth_unit='M'*, *well_name=''*, *well=None*, *units=None*)

Write an fwap-derived log set out as a LAS file.

Units for the common fwap curves (DTP, DTS, DTST, COH*, VPVS, Q, K, MU, E, NU) are filled in automatically from an internal table. Pass `units` to override or supply units for custom mnemonics.

Parameters

- **path** (*str*) – Output LAS path.
- **depth** (*ndarray, shape (n_depth,)*) – Depth axis. Must match the first dimension of every curve array.
- **curves** (*mapping from str to ndarray*) – `{mnemonic: ndarray}`. Each array must have shape `(n_depth,)`; `NaN` is written as the LAS null value.
- **depth_unit** (str, default `"M"`) – Unit for the depth axis (LAS convention: `"M"` or `"FT"`).
- **well_name** (*str, default empty*) – Value for the LAS `WELL` header entry.
- **well** (*mapping from str to str, optional*) – Additional `~Well`-section entries. Keys must be the standard LAS 2.0 mnemonics (`COMP` for company, `FLD` for field, `SRVC` for service company, `UWI`, etc.). Non-standard mnemonics are rejected by `lasio`.
- **units** (*mapping from str to str, optional*) – Per-curve unit override, keyed by mnemonic. Missing entries fall back to the built-in table and then to empty.

Raises
ValueError – If any curve has a length different from `depth`.

Return type
None

`fwap.io.`**write_segy**(*path*, *data*, *dt*, *offsets=None*, *, *textual_header=None*)

Write a multichannel gather to a SEG-Y rev 1 file (IEEE float).

Mirror of `read_segy()` on the output side. Useful for piping processed synthetics back out as SEG-Y for interchange with other seismic software.

Parameters

- **path** (*str*) – Output file path.
- **data** (*ndarray, shape (n_traces, n_samples)*) – Trace data. Written as IEEE 32-bit float (SEG-Y format code 5).
- **dt** (*float*) – Sample interval (s). Converted to microseconds for the binary header's `Interval` field; values below 1 us are clipped.
- **offsets** (*ndarray, shape (n_traces,), optional*) – Source-to-receiver offsets written into the `offset` trace header field. Coerced to `int` because the SEG-Y header is a 32-bit integer. If `None`, all-zero offsets are written.
- **textual_header** (*str, optional*) – Free-form text for the 3200-byte EBCDIC header. Truncated / padded to 3200 bytes as required by SEG-Y.

Raises

ValueError – If `data` is not 2-D, `offsets` has the wrong length, or `dt` is non-positive.

Return type

None

3.15 fwap.plotting

Plotting helpers for the fwap demos and for interactive use.

Two utilities:

- `wiggle_plot()` – draw a wiggle plot of a multichannel gather (time axis in ms, receivers on the y-axis).
- `save_figure()` – create the output directory if needed and write the figure to `<figdir>/<name>`, with a log line recording where it went.

Both were previously available as `fwap._plotting._wiggle` and `fwap._plotting._savefig`; those underscored names are kept as aliases so third-party scripts importing them continue to work.

Functions

`save_figure`(fig, figdir, name[, show])	Save `fig` to `<figdir>/<name>`, creating the directory if needed, and close or show the figure depending on `show`.
`wiggle_plot`(ax, data, t[, scale, xmax, title])	Draw a wiggle plot of a multichannel gather.

`fwap.plotting.`**`save_figure`**(*fig*, *figdir*, *name*, *show=False*)

Save `fig` to `<figdir>/<name>`, creating the directory if needed, and close or show the figure depending on `show`.

Parameters

- **fig** (*matplotlib.figure.Figure*)
- **figdir** (*str*) – Output directory. Created with `os.makedirs(..., exist_ok=True)`.
- **name** (*str*) – Filename under `figdir`, including extension (typically `.png`).
- **show** (*bool, default False*) – If `True`, call `plt.show()` instead of `plt.close(fig)`.

Return type
None

`fwap.plotting.`**`wiggle_plot`**(*ax*, *data*, *t*, *scale=1.5*, *xmax=None*, *title=''*)

Draw a wiggle plot of a multichannel gather.

Parameters

- **ax** (*matplotlib.axes.Axes*)
- **data** (*ndarray, shape (n_rec, n_samples)*) – Gather to plot. Each receiver is drawn as a trace offset vertically by its receiver index.
- **t** (*ndarray, shape (n_samples,)*) – Time axis (s). Displayed in milliseconds on the x-axis.
- **scale** (*float, default 1.5*) – Amplitude scale: each trace is scaled so that its peak is at most `scale` vertical units.
- **xmax** (*float, optional*) – If given, restrict the x-axis to `[0, xmax * 1e3]` ms.
- **title** (*str, default empty*) – Axes title.

Return type
None

CHAPTER

FOUR

FWAP ROADMAP

A living document of open items that would meaningfully extend fwap beyond the 0.4.0 release, ordered by estimated effort × user-value.

4.1 Released (for reference)

4.1.1 0.4.0

All of these landed on the 0.4.0 PR and are covered by tests:

- Part 1: STC + rule-based picker, Viterbi picker (per-mode and fully-joint 3-mode).
- Part 2: f-k filtering, SVD / Karhunen-Loeve separation.
- Part 3: intercept-time inversion (midpoint + segmented), dipole flexural dispersion (phenomenological + Rayleigh-speed physical limit), delay-to-altered-zone-thickness conversion.
- Part 4: dip / azimuth estimation.
- Extensions: cross-dipole Alford rotation, Q from centroid-shift and spectral-ratio, elastic moduli, Reuss / Voigt / Hill mixing laws, Stoneley permeability indicator, LAS / SEG-Y I/O, `fwap process` CLI.

4.1.2 Since 0.4.0 ([Unreleased])

The post-0.4.0 cycle was a sweep of book-completeness gaps flagged in `ideas/Mari1994.docx`; every algorithm-level item the book names is now in the package:

- **Part 1 picker**: pseudo-Rayleigh / guided-mode picking is now a fourth default mode (`DEFAULT_PRIORS["PseudoRayleigh"]`, `pseudo_rayleigh_dispersion`); per-mode amplitude logs are exposed alongside coherence (`STCResult.amplitude`, `ModePick.amplitude`); the wavelet-shape and onset-polarity expert rules are available as post-pick filters (`onset_polarity`, `wavelet_shape_score`, `filter_picks_by_shape`, `filter_track_by_shape`); cross-mode consistency QC flags depths where Vp/Vs is unphysical or the canonical time ordering is violated (`PickQualityFlags`, `quality_control_picks`, `quality_control_track`).
- **Part 2 wave separation**: tau-p / slant-stack / linear Radon joins f-k and SVD/K-L (`tau_p_forward`, `tau_p_inverse`, `tau_p_adjoint`, `tau_p_filter`).
- **Part 3 altered zone**: the joint (thickness, velocity-contrast) deliverable is now a first-class output (`altered_zone_estimate`, `delay_to_altered_zone_velocity_contrast`, `AlteredZoneEstimate`); the original `delay_to_altered_zone_thickness` stays as the thickness-anchor branch.
- **Workflow 3 dipole-sonic**: a petrophysical labelling layer re-frames the Alford rotation in stress-direction terms (`StressAnisotropyEstimate`, `stress_anisotropy_from_alford`) with max-horizontal-stress azimuth, splitting-time delay, anisotropy strength, and a heuristic fracture indicator.

- **I/O**: DLIS read / write joins LAS and SEG-Y as a core dependency (`read_dlis`, `write_dlis`, `DlisCurves`); all four log-format libraries (`lasio`, `dlisio`, `dliswriter`, `segyio`) are now in the base `dependencies` list, and the optional `[io]` / `[dlis]` / `[segy]` extras are gone.
- **Demos / CLI**: `fwap pseudorayleigh`, `fwap taup`, and `fwap dlis` are wired into the demo registry alongside the existing chapter demos.

A second sweep (after the docx pair `Paillet1991.docx` and `Tang2004.docx` were added to `ideas/`) closed the gaps Tang & Cheng (2004) flag as the post-1994 borehole-acoustic processing literature:

- **Picker → log-curve bridge**: `track_to_log_curves(track) -> (depths, curves)` converts a per-depth pick track from `track_modes` / `viterbi_pick` / `viterbi_pick_joint` into the fixed-length `{mnemonic: ndarray}` dict the LAS / DLIS writers consume directly. Slowness is converted to us/ft (the LAS unit table convention); missing picks become NaN by default with an optional numeric sentinel.
- **Geomechanics layer (`fwap.geomechanics`)**: Rickman 2008 brittleness / fracability index, Eaton 1969 uniaxial-strain closure stress, Lacy 1997 (Chang 2006 form) sandstone UCS, Bratli–Risnes 1981 sand-stability flag, density-log overburden integration, and a one-call `geomechanics_indices(moduli, ...)` bundle returning a `GeomechanicsIndices` dataclass with all four indices (closure stress optional, conditional on a supplied overburden).
- **Dispersive pseudo-Rayleigh STC**: `dispersive_pseudo_rayleigh_stc` is the pseudo-Rayleigh analogue of `dispersive_stc`; same back-projection machinery, only the per-mode dispersion law differs. Enforces the fast-formation existence constraint (`shear_slowness_range[1] < 1 / v_fluid`).
- **Stoneley amplitude fracture indicator**: `stoneley_amplitude_fracture_indicator(A_obs, A_ref)` = `1 - A_obs / A_ref` — companion to the existing `stoneley_permeability_indicator`. Detects the same fractures / permeable zones via energy loss rather than via the poroelastic-delay slowness shift; the two have complementary noise characteristics.
- **Hornby et al. (1989) Stoneley reflection-coefficient fracture- aperture inversion**: `stoneley_reflection_coefficient(...)` builds `|R|`; `hornby_fracture_aperture(R, frequency_hz, V_T, ...)` inverts the low-frequency closed form $|R| = \omega L_0 / \mathrm{sqrt}(V_T^2 + \omega^2 L_0^2)$ for the fracture aperture L_0 (m). Quantitative complement to the two slowness- and amplitude-based indicators.
- **Thomsen-gamma from combined dipole + Stoneley (`fwap.anisotropy`)**: `thomsen_gamma(c44, c66)`, `stoneley_horizontal_shear_modulus(s_ST, rho_fluid, v_fluid)` (White 1983 / Norris 1990 tube-wave inversion), and a one-call `thomsen_gamma_from_logs(s_dipole, s_stoneley, rho, ...)` returning a `ThomsenGammaResult` with C44, C66, gamma per depth.
- **Slow-formation Vs from low-frequency Stoneley (`fwap.rockphysics`)**: `vs_from_stoneley_slow_formation(...)` is the primary sonic-only V_S estimator for the case where the formation has no S head wave on a monopole gather and pseudo-Rayleigh does not exist (V_S < V_fluid; Paillet & Cheng 1991 Ch. 3).
- **Stress-vs-intrinsic anisotropy classifier (`fwap.dispersion`)**: `classify_flexural_anisotropy(curve_a, curve_b)` labels a cross-dipole record as `"isotropic"`, `"intrinsic"`, `"stress_induced"`, or `"ambiguous"` based on whether the slowness difference Δs(f) crosses zero between a low-f band and a high-f band — the Sinha & Kostek 1996 diagnostic that distinguishes far-field rock fabric from borehole-wall stress concentration.
- **LWD phenomenological layer (`fwap.lwd`)**: `lwd_collar_mode`, `synthesize_lwd_gather`, and `notch_slowness_band` (subtract- the-in-band route, preserves out-of-grid signals) deliver the monopole-side collar-rejection workflow; `QuadrupoleRingGather`, `synthesize_quadrupole_lwd_gather`, `quadrupole_stack` and `lwd_quadrupole_priors` deliver the m=2 source / receiver geometry that Tang & Cheng 2004 sect. 2.5 frame as the practical solution to LWD collar contamination. `fwap lwd` runs the worked-example demo. **Not** a layered cylindrical-Biot solver (still flagged as Open item A below).

4.2 Open items

4.2.1 A. Full cylindrical-Biot dispersion solver

Status: fwap currently ships a phenomenological flexural dispersion (`fwap.synthetic.dipole_flexural_dispersion`) and a Rayleigh-speed-asymptote improvement (`fwap.cylindrical.flexural_dispersion_physical`). Neither solves the full 3×3 modal determinant. A production sonic-processing package needs the Schmitt / Paillet–Cheng / Tang solver.

What to build:

Root-find the zeros of the modal determinant `M_n(ω, k) = 0` in complex phase-slowness (axial wavenumber `k` for mode order `n`), then sample the resulting dispersion curve `s(f) = k(ω)/ω` at the caller's frequencies.

Radial wavenumbers (same for every azimuthal order):

```
f_f = sqrt(k² - ω² / V_p_fluid²)        # in the fluid (r < a)
f_p = sqrt(k² - ω² / V_p_solid²)        # P wave in the solid
f_s = sqrt(k² - ω² / V_s_solid²)        # S wave in the solid
```

Each of these is real when the corresponding mode is evanescent at that radius, imaginary when it's propagating; the branch selection is what makes the root-finder non-trivial.

Modal determinants, from Kurkjian & Chang (1986) Section II and Paillet & Cheng (1991) Chapter 4:

- **Monopole (n=0)**: a 2×2 determinant involving only fluid and solid radial terms. Zeros give the Stoneley mode (c < V_p_f) and the pseudo-Rayleigh branch (leaky, c ≈ V_s for high frequency).
- **Dipole (n=1)**: a 3×3 determinant with entries built from modified-Bessel combinations `I_0(f_f a)`, `I_1(f_f a)`, `K_0(f_p a)`, `K_1(f_p a)`, `K_0(f_s a)`, `K_1(f_s a)`, weighted by the Lamé parameters. Zeros give the flexural mode that is currently approximated phenomenologically by :func:`fwap.synthetic.dipole_flexural_dispersion`.

Boundary conditions at `r = a` are the standard elastodynamic set:

1. radial displacement continuity (fluid normal velocity matches solid radial velocity);
2. radial stress continuity (fluid pressure matches solid normal stress);
3. tangential stress vanishes in the fluid (no shear-wave coupling).

Writing these out in terms of the per-region potentials and substituting the Bessel-function solutions gives the 3×3 matrix; the exact layout is in Kurkjian & Chang (1986) equations 8 and 9.

Implementation strategy:

1. Start at a frequency where the answer is known (e.g., low frequency where `s → 1 / V_s` is exact for the flexural mode).
2. March in frequency, using the previous iterate as the initial guess for `scipy.optimize.newton`.
3. Track the sign of `Im(f_s)` to stay on the right branch (propagating vs evanescent shear); flip branches explicitly at the cutoff frequency.
4. For the dipole high-frequency limit, converge to the Scholte speed at the fluid-solid interface (slightly below the Rayleigh speed used by the current phenomenological code).

Public API target:

```
fwap.cylindrical.modal_dispersion(
    vp: float,
    vs: float,
```

(continues on next page)

(continued from previous page)

```
    vp_fluid: float,
    rho_solid: float,
    rho_fluid: float,
    a_borehole: float,
    mode: Literal["flexural", "stoneley", "pseudo_rayleigh"] = "flexural",
) -> Callable[[np.ndarray], np.ndarray]
```

Returns the same callable contract as the existing `dipole_flexural_dispersion`: array of frequencies in, array of phase slownesses out.

Scope: ~500 lines of physics code plus a validation notebook that reproduces the published dispersion curves (Paillet & Cheng 1991 Figure 4.5; Schmitt 1988 Figure 4; Tang & Cheng 2004 Figure 3.4) to within plotting accuracy. Several days of focused work. The hardest piece is robust branch selection across the pseudo-Rayleigh cutoff; start with the Stoneley mode (no cutoff in the band of interest) before attempting dipole flexural.

References:

- Schmitt, D. P. (1988). Shear-wave logging in elastic formations. *J. Acoust. Soc. Am.* 84(6), 2230-2244.
- Paillet, F. L., & Cheng, C. H. (1991). *Acoustic Waves in Boreholes*, Chapter 4. CRC Press.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods*, Chapter 3. Elsevier.
- Kurkjian, A. L., & Chang, S.-K. (1986). Acoustic multipole sources in fluid-filled boreholes. *Geophysics* 51(1), 148-163 (most explicit derivation of the 3×3 dipole determinant).
- Ellefsen, K. J., Cheng, C. H., & Toksoz, M. N. (1991). Applications of perturbation theory to acoustic logging. *J. Geophys. Res.* 96(B1), 537-549 (starting-guess strategy for the dipole root-finder).

4.2.2 B. Quantitative Stoneley permeability (Tang–Cheng–Toksöz 1991)

Status: fwap ships three complementary, *uncalibrated* Stoneley indicators / inversions:

- `stoneley_permeability_indicator` returns the fractional slowness shift relative to a tight reference (matrix permeability + open fractures together);
- `stoneley_amplitude_fracture_indicator` returns the fractional amplitude deficit (transmission-loss form; complementary noise characteristics);
- `hornby_fracture_aperture` inverts the reflected-wave coefficient for an actual fracture aperture in metres (rigid-frame, single- fracture limit).

What is **still missing** is the matrix-permeability inversion that maps the Stoneley slowness shift onto an absolute permeability in m^2 or mD. That is the Tang–Cheng–Toksöz (1991) simplified Biot-Rosenbaum piece outlined below, and remains the open item.

What to build:

The closed-form low-frequency inversion of the Stoneley slowness shift into formation permeability, from Tang–Cheng–Toksöz (1991).

Starting point: the observed slowness shift `α_ST` (dimensionless fractional shift vs a tight reference, already computed by :func:`fwap.rockphysics.stoneley_permeability_indicator`) and a set of Biot / fluid parameters.

Tang–Cheng–Toksöz (1991) simplified Biot-Rosenbaum: at angular frequency `ω` well below the Biot characteristic frequency,

```
α_ST(ω) ≈ 1 / 2 · (K_f / K_ψ) · (1 / (1 − i ω / ω_c))
```

where

- `K_f` : fluid bulk modulus (Pa)
- `K_ψ` : frame bulk modulus of the porous formation (Pa)
- `ω_c` : Biot characteristic angular frequency, `ω_c = η ψ / (κ ρ_f)` for dynamic fluid viscosity `η`, porosity `ψ`, permeability `κ`, fluid density `ρ_f`.

Solving for `κ` given an observed `α_ST(ω)` gives the per-depth permeability. Real and imaginary parts of `α_ST` carry independent permeability information — the real part (amplitude-based) is preferred when SNR allows.

Implementation:

1. Given observed `α_ST` (`1 - i 0` approximation for real slowness shifts), invert the closed form for `ω_c`.
2. Recover `κ = η ψ / (ω_c ρ_f)`.
3. Clip to non-negative values (noise-driven negatives are bounded by zero).

Public API target:

```
fwap.rockphysics.stoneley_permeability_tang_cheng(
    slowness_observed: np.ndarray,
    slowness_reference: np.ndarray | float,
    *,
    frequency: float,
    fluid_bulk_modulus: float,           # K_f in Pa
    fluid_viscosity: float,              # η in Pa·s
    fluid_density: float,                # ρ_f in kg/m³
    porosity: np.ndarray,                # ψ, unitless
    frame_bulk_modulus: np.ndarray,      # K_ψ in Pa
) -> np.ndarray
```

Returns permeability in m^2 (multiply by `9.869e-13` to convert to darcies).

Validation target: reproduce Figure 5.3 of Tang & Cheng (2004) — a synthetic permeable bed (1-2 darcy) sandwiched between tight limestone (0.01-0.1 millidarcy) recovered from the Stoneley slowness shift alone.

Scope: ~60 lines of code plus one validation test against the Tang & Cheng (2004) Figure 5.3 numbers. One focused day with the reference open.

References:

- Tang, X.-M., Cheng, A., & Toksöz, M. N. (1991). Dynamic permeability and borehole Stoneley waves: A simplified Biot-Rosenbaum model. *J. Acoust. Soc. Am.* 90(3), 1632-1646.
- Kostek, S., & Johnson, D. L. (1992). The interaction of tube waves with borehole fractures, Part I: Numerical models. *Geophysics* 57(6), 784-795.
- Tang, X.-M., & Cheng, A. (2004). *Quantitative Borehole Acoustic Methods*, Section 5.1. Elsevier.

4.2.3 C. Fully-joint Viterbi extensions

`fwap.picker.viterbi_pick_joint` already does joint 3-mode Viterbi across depths, `viterbi_posterior_marginals` runs the forward-backward pass for per-mode posterior marginals, and the `soft_time_order` keyword on both turns the strict ordering constraint into a soft penalty – so the only remaining sub-item from the original 0.4.0 roadmap is:

1. **Variable candidate budget**: current implementation caps valid triples per depth at `max_triples_per_depth` (default 2000) and raises on overflow. A smarter top-K pruning per mode (keep the K most coherent candidates per mode before triple enumeration) would handle pathological peak-heavy STC surfaces gracefully without raising.

A natural follow-on now that the picker supports four default modes (P / S / pseudo-Rayleigh / Stoneley) is:

2. **4-mode joint Viterbi**: currently both `viterbi_pick_joint` and `viterbi_posterior_marginals` are hard-coded to the (P, S, Stoneley) triple and subset `DEFAULT_PRIORS` accordingly; full 4-mode picking still falls back to the per-mode sequential `viterbi_pick`. Extending the trellis to quadruples squares its width but is otherwise mechanical.

Scope: each is a few-hours patch. Lower leverage than the algorithms above because the current picker already handles the hard-to-handle cases.

4.2.4 D. Conda-forge recipe

The package is ready for PyPI (0.4.0 builds cleanly, wheels ship `py.typed`). A conda-forge recipe (`meta.yaml` + CI setup) can be submitted to staged-recipes once the first PyPI release is live. Reversible, low-risk; one afternoon's work.

4.2.5 E. `ruff format` sweep

Run `ruff format .` once across the tree as a standalone formatting commit. Reformats ~34 files. Not enabled in pre-commit today because the existing hand-formatted style is consistent but differs from ruff-format's defaults (trailing semicolons, specific indent conventions). After the sweep, add `ruff-format` to the pre-commit hook list so drift is prevented automatically.

4.2.6 F. Real-data test fixtures

Ship an anonymised reference dataset (single LAS + single SEG-Y pulled from a public well) so the test suite includes a genuine real-data integration test, not just synthetics. Would need permission for redistribution; the USGS open-file datasets are likely candidates.

4.3 Non-goals

These have come up in reviews and been deliberately deferred:

- **GUI / plotting app**. `fwap.plotting` exposes `wiggle_plot` and `save_figure` for use in notebooks and scripts. A dedicated GUI is out of scope; integrate with Jupyter or your own plotting stack.
- **Production multi-well log management**. `fwap.io.read_las` / `write_las` are single-file helpers. A database / catalog layer belongs in a separate package.
- **Time-frequency analysis beyond the STC surface**. Wavelet transforms, short-time Fourier, spectrogram picking – all useful, all out of scope for a reference implementation of the 1994 book.

CHAPTER

FIVE

CHANGELOG

Below is a snapshot of `CHANGELOG.md`; see the file itself for the most up-to-date entries.

```
# Changelog

All notable changes to this project are documented here. The format
loosely follows [Keep a Changelog](https://keepachangelog.com/), and
the project uses [Semantic Versioning](https://semver.org/).

## [Unreleased]

### Added
- **LWD (logging-while-drilling) phenomenological layer**
  (`fwap.lwd`). Models the steel-drill-collar contamination Tang &
  Cheng (2004) sect. 2.4-2.5 frame as the defining processing
  problem of the LWD era, plus the two practical responses:
  collar-band notching and quadrupole-source / receiver geometry.
  Public surface: `lwd_collar_mode(...)` returns a pre-configured
  Gabor `Mode` at the published 80-130 us/ft collar band;
  `synthesize_lwd_gather(...)` plants the collar on top of the
  formation modes; `notch_slowness_band(...)` is a slowness-band-
  stop filter via tau-p forward + cosine-tapered band-pass mask +
  adjoint **+ subtract-the-in-band** (the subtraction route
  preserves signals at slownesses outside the tau-p grid, e.g.
  Stoneley at ~217 us/ft survives a notch at ~92 us/ft); on the
  quadrupole side, `QuadrupoleRingGather`,
  `synthesize_quadrupole_lwd_gather(...)` builds a ring of n_rec
  >= 4 receivers with a `cos(2(theta - phi))` source pattern and
  `quadrupole_stack(data, azimuths, ...)` projects the ring onto
  m=2, rejecting m=0 / m=1 patterns by orthogonality;
  `lwd_quadrupole_priors()` returns a tool-aware picker priors
  dict. `fwap.demos.demo_lwd` and `fwap lwd` CLI provide a
  worked-example end-to-end. **Not** a layered cylindrical-Biot
  solver -- that is research-grade work and remains future work.
- **Stress-vs-intrinsic anisotropy classifier from a fast / slow
  flexural dispersion-curve crossover.** Sinha & Kostek (1996)
  showed that the two cross-dipole flexural dispersion curves of a
  stress-anisotropic formation cross over in frequency: the
  low-frequency mode samples the far-field rock fabric and the
  high-frequency mode samples the near-wellbore stress
  concentration, so a Δs(f) sign flip between the bands flags
```

(continues on next page)

(continued from previous page)

```
  stress-induced anisotropy (intrinsic anisotropy shows no such
  crossover). New `classify_flexural_anisotropy(curve_a, curve_b,
  ...)` returns a `FlexuralDispersionDiagnosis` with a
  classification in `{"isotropic", "intrinsic", "stress_induced",
  "ambiguous"}`, the per-band Δs averages, the interpolated
  crossover frequency (when present, restricted to the bracket
  spanning the two band means), and a tuple of human-readable
  reasons for QC.
- **Slow-formation Vs from low-frequency Stoneley slowness**
  (`fwap.rockphysics.vs_from_stoneley_slow_formation`). Inverts the
  White (1983) tube-wave formula `S_ST^2 = 1/V_f^2 + rho_f / mu`
  for the formation shear velocity. Primary sonic-only Vs
  estimator for slow formations (V_S < V_fluid) where the
  formation has no critically-refracted S head wave on a monopole
  gather (Paillet & Cheng 1991, Ch. 3) and pseudo-Rayleigh does
  not exist. Same physics as
  `stoneley_horizontal_shear_modulus` (which returns C_66 for
  VTI), divided by rho and square-rooted; the difference is
  interpretation.
- **Hornby et al. (1989) Stoneley reflection-coefficient fracture-
  aperture inversion.** Quantitative complement to the existing
  Stoneley indicators. New `stoneley_reflection_coefficient(A_inc,
  A_refl)` builds `|R|` from incident / reflected pulse
  amplitudes; new `hornby_fracture_aperture(R, frequency_hz,
  V_T, ...)` inverts the rigid-frame, low-frequency, single-
  fracture closed form
  `|R(omega)| = omega L_0 / sqrt(V_T^2 + omega^2 L_0^2)` for the
  fracture aperture L_0 (m). Saturates at +inf for `|R| -> 1`; an
  optional small-amplitude approximation `L_0 ~ V_T |R| / omega`
  is < 5 % off for |R| <= 0.3.
- **Stoneley amplitude fracture indicator**
  (`fwap.rockphysics.stoneley_amplitude_fracture_indicator`).
  Companion to the existing slowness-shift permeability indicator.
  Returns `1 - A_obs / A_ref` -- the fractional Stoneley amplitude
  deficit relative to a tight reference. Detects the same
  fractures / permeable zones via the loss of acoustic energy
  rather than via the dynamic-poroelastic delay; complementary
  noise characteristics, so a coincidence flag is more robust than
  either indicator alone.
- **Dispersive STC for the pseudo-Rayleigh / guided trapped mode**
  (`fwap.dispersion.dispersive_pseudo_rayleigh_stc`). Direct
  pseudo-Rayleigh analogue of `dispersive_stc`: scans formation
  shear slowness, applies the per-frequency phase-slowness
  correction from `pseudo_rayleigh_dispersion`, returns an
  `STCResult` whose slowness axis is the formation `1 / V_S`.
  Removes the high-frequency bias that plain STC produces on
  guided arrivals. Enforces the fast-formation existence
  constraint (`shear_slowness_range[1] < 1 / v_fluid`).
- **Geomechanics indices on top of `ElasticModuli`** (new module
  `fwap.geomechanics`). Closes the gap between the elastic-moduli
  output of `fwap.rockphysics.elastic_moduli` and the
  Workflow-3 deliverables Mari et al. (1994) Part 3 lists --
```

(continues on next page)

(continued from previous page)

```
  *sanding prediction* and *hydraulic-fracture design*. Public
  surface: `brittleness_index_rickman(E, nu, ...)` (Rickman et al.
  2008 BI in `[0, 1]`); `fracability_index(...)` (alias of BI for
  HF-design call sites); `closure_stress(nu, sigma_v, P_p,
  alpha)` (Eaton 1969 uniaxial-strain closure stress, with
  validation rejecting both `nu >= 1` singularity and `nu < 0`
  auxetic regime); `unconfined_compressive_strength(E,
  model='lacy_sandstone')` (Lacy 1997 / Chang et al. 2006);
  `sand_stability_indicator(mu, threshold)` (Bratli & Risnes 1981
  / 5 GPa shear-modulus rule); `overburden_stress(z, rho)`
  (trapezoidal density-log integration). One-call wrapper
  `geomechanics_indices(moduli, ...)` returns a
  `GeomechanicsIndices` dataclass with `brittleness`,
  `fracability`, `ucs`, `sand_stability` and (when
  `sigma_v_pa` is supplied) `closure_stress`. Module-level
  constants `RICKMAN_E_MIN_PA`, `RICKMAN_E_MAX_PA`,
  `RICKMAN_NU_MIN`, `RICKMAN_NU_MAX`,
  `SAND_STABILITY_SHEAR_THRESHOLD_PA` expose the published
  defaults. Six new LAS / DLIS mnemonics in `_FWAP_UNITS`:
  `BRIT`, `FRAC`, `UCS`, `SH`, `SV`, `SAND`.
- **Thomsen-gamma from combined dipole + Stoneley sonic logs**
  (`fwap.anisotropy`). VTI shear-anisotropy parameter
  `gamma = (C_66 - C_44) / (2 C_44)` from two complementary
  measurements: the dipole shear log gives `C_44 = rho * V_Sv^2`,
  and the Stoneley low-frequency tube-wave inversion (White 1983
  / Norris 1990) gives `C_66 = rho_f / (S_ST^2 - 1/V_f^2)`. New
  `stoneley_horizontal_shear_modulus(s_ST, rho_fluid, v_fluid)`,
  `thomsen_gamma(c44, c66)`, and a one-call
  `thomsen_gamma_from_logs(s_dipole, s_stoneley, rho, ...)`
  returning a `ThomsenGammaResult` with `c44`, `c66`, `gamma`.
  New LAS mnemonics: `C44`, `C66`, `GAMMA`.
- **Picker -> log-curve bridge**
  (`fwap.picker.track_to_log_curves`). Converts a
  `Sequence[DepthPicks]` from `track_modes` / `viterbi_pick` /
  `viterbi_pick_joint` into a `(depths, curves)` tuple where
  `curves` is a `{mnemonic: ndarray}` dict suitable to pass
  straight to `write_las` / `write_dlis`. Standard fwap mnemonics
  (`DTP`, `DTS`, `DTST`, `DTPR` / `COHP`, `COHS`, `COHST`,
  `COHPR` / `AMP*` / `TIM*` / `VPVS`); slowness in us/ft;
  missing picks become NaN by default with an optional numeric
  sentinel via `null_value`. `_FWAP_UNITS` extended to carry the
  new mnemonics.

### Fixed
- **WLS in attenuation Q estimators** (`centroid_frequency_shift_Q`
  and `spectral_ratio_Q`): the previous implementations multiplied
  both `A` and `y` by `W = diag(w)` before passing to `lstsq`,
  which makes the solver minimise `sum(w_i^2 * r_i^2)` instead of
  the documented `sum(w_i * r_i^2)`. Switched to `sqrt(W) @ A`,
  `sqrt(W) @ y` so the per-trace weights match the docstring intent
  ("weights = total power") and the residual variance / standard
  error formulas now match the system actually being solved.
```

(continues on next page)

(continued from previous page)

```
- **`read_las` depth-curve detection**: skipped the depth curve
  via mnemonic equality with `las.curves[0].mnemonic`, which was
  fragile when a non-depth curve happened to share the depth
  mnemonic. Now skips by index instead (always first).
- **`viterbi_pick` doc bug**: the comment claimed the
  no-previous-mode sentinel was `+inf`; the code (correctly) uses
  `-inf`. Comment fixed.
- **`anisotropy_strength` docstring**: claimed the metric reaches 1
  for orthogonal waveforms, but actually reaches `1 / sqrt(2)` for
  orthogonal equal-energy waveforms and only saturates at 1 as
  `s -> -f`. Updated formula and verbal description; the metric
  itself was correct.

### Changed
- **`closure_stress` validation tightened** to reject negative
  Poisson's ratio. The Eaton uniaxial-strain model is calibrated
  for the positive-Poisson regime of typical sedimentary rocks;
  negative inputs would produce negative effective horizontal
  stresses. Auxetic materials are out of scope.
- **`classify_flexural_anisotropy`**: band-overlap guard tightened
  from `>` to `>=` (touching bands now rejected); crossover-
  frequency search restricted to the bracket
  `[f_low_band[0], f_high_band[1]]` so a spurious noise zero-
  crossing outside the bands is not reported as the band-to-band
  transition.
- **`track_to_log_curves`**: float-coerce `null_value` at function
  entry (`null_value = float(null_value)`) so passing `None` or
  any non-float type raises `TypeError` cleanly instead of
  slipping through the NaN check and producing object-dtype
  curves.
- **`dispersive_stc`**: rename misleading internal variable
  `tau_of_f` -> `s_of_f`. The variable holds slowness, not a time
  τ; no behaviour change.
- **`sand_stability_indicator`**: docstring now explicit that the
  `mu == threshold_pa` boundary is treated as stable.

- **Cross-mode consistency QC for the picker.** Closes the soft
  Workflow-3 gap flagged in the closing paragraph of the docx
  review: the book's QC philosophy is *"log continuity AND
  cross-consistency between modes"*; continuity was already
  enforced inside the Viterbi pickers but no API surfaced the
  cross-consistency layer. New ``quality_control_picks(picks,
  depth=, *, vp_vs_min=1.4, vp_vs_max=2.6, require_time_order=
  True)`` returns a ``PickQualityFlags`` with two checks: the
  Vp/Vs ratio (``s_S / s_P``) is gated against the canonical
  sedimentary-rock band, and the canonical time ordering
  ``t_P <= t_S <= t_PseudoRayleigh <= t_Stoneley`` is verified
  over the modes that were picked. The function only flags --
  callers decide whether to drop, mark, or human-review the
  flagged depths. ``quality_control_track`` is the multi-depth
  analogue. Picks whose Vp or S is missing have ``vp_vs=None``
  and ``vp_vs_in_band=True`` (the gate is skipped, not failed).
```

(continues on next page)

(continued from previous page)

```
- **Stress-direction / flexural-fracture-indicator API on top of the
  cross-dipole Alford rotation.** The book frames the Workflow-3
  dipole-sonic deliverable as "shear anisotropy, mechanical
  properties and fracture indicators from the flexural wave" plus
  "stress-direction estimation"; the numerics already lived inside
  ``alford_rotation`` (the fast-shear angle, the cross-energy
  ratio) but no API surfaced them in those petrophysical terms.
  New ``stress_anisotropy_from_alford(alford, dt)`` returns a
  ``StressAnisotropyEstimate`` carrying ``max_horizontal_stress
  _azimuth`` (= fast-shear angle, with a docstring caveat that the
  conventional stress-direction interpretation depends on whether
  the anisotropy is stress- or fracture-induced),
  ``min_horizontal_stress_azimuth`` (orthogonal, folded into
  ``(-pi/2, pi/2]``), ``splitting_time_delay`` (cross-correlation
  lag of slow vs fast), ``anisotropy_strength`` (relative L2 norm
  in ``[0, 1]``), ``rotation_quality`` (= ``1 - cross_energy_ratio``)
  and a heuristic ``fracture_indicator`` (their product). The
  underlying ``alford`` is kept on the result so callers that need
  the rotated waveforms can still reach them.
- **Wavelet-shape + onset-polarity expert rules in the picker.** The
  book (Mari et al. 1994, Part 1) lists *"expert rules on slowness
  range, **wavelet shape, onset polarity**, coherence across the
  receiver array, depth-to-depth continuity"* as the picker's
  knowledge-based discriminator set; the package only had the
  bolded subset (slowness windows + STC coherence + Viterbi
  continuity). Two opt-in expert rules now run as post-pick gates:
  ``polarity`` (``+1``/``-1``/``0``) checks the sign of the
  stacked window's largest-absolute sample, and ``shape_match_min``
  gates picks against the absolute Pearson correlation between the
  stacked window and a Ricker template at the prior's ``f0``. Both
  are exposed via ``filter_picks_by_shape(picks, data, dt,
  offsets, *, priors)`` and ``filter_track_by_shape(track, datas,
  dt, offsets, *, priors)``; the underlying ``onset_polarity`` and
  ``wavelet_shape_score`` primitives are also public. Default
  priors leave both gates disabled (``polarity=0``,
  ``shape_match_min=0.0``) so existing callers are unaffected.
- **Per-mode amplitude logs in the Workflow-1 picker pipeline.** The
  book (Mari et al. 1994, Part 1) frames the rule-based picker's
  deliverable as *"continuous Vp, Vs and Stoneley slowness curves
  together with per-mode amplitude **and coherence** logs"*; the
  pipeline only carried the coherence half. ``STCResult`` now has an
  ``amplitude`` ``ndarray | None`` field of the same
  ``(n_slowness, n_time)`` shape as ``coherence``, populated by both
  ``coherence.stc`` and ``dispersion.dispersive_stc`` with the RMS
  of the per-trace stack contribution at each cell (a unit-amplitude
  sine on every trace gives ``amplitude = 1/sqrt(2)``).
  ``find_peaks`` returns a 4-column ``[slowness, time, coherence,
  amplitude]`` table when the input STC carries amplitude, and
  ``ModePick`` gained an ``amplitude: float | None`` field that
  ``pick_modes`` / ``track_modes`` / ``viterbi_pick`` /
  ``viterbi_pick_joint`` / ``viterbi_posterior_marginals`` now
  populate from the picked cell. Existing 3-column / amplitude=None
```

(continues on next page)

(continued from previous page)

```
  call sites continue to work unchanged.
- **Altered-zone velocity contrast as a Workflow-2 deliverable.**
  The book (Mari et al. 1994, Part 3) frames the intercept-time
  workflow's altered-zone product as the *(thickness, velocity-
  contrast)* pair, but `fwap.tomography` only had
  `delay_to_altered_zone_thickness` -- which forces the caller to
  supply the altered-zone slowness as an input. The single
  refraction-geometry equation `delay = 2 * h * (s_altered -
  s_virgin)` is one constraint in two unknowns, so the package now
  exposes both directions plus a joint helper:
  `delay_to_altered_zone_velocity_contrast(delay, thickness)` is
  the algebraic dual; `altered_zone_estimate(delay, s_virgin,
  thickness=...)` or `altered_zone_estimate(delay, s_virgin,
  slowness_altered=...)` returns an `AlteredZoneEstimate` dataclass
  carrying thickness, absolute altered slowness, and slowness
  contrast at every depth, with the helper rejecting calls that
  pin both or neither anchor.
- **τ-p (slant-stack / linear Radon) wave separation.** The book
  (Mari et al. 1994, Part 2) lists the τ-p domain alongside f-k as
  a textbook multichannel velocity-filter, but `fwap.wavesep`
  previously offered only `fk_filter` and SVD/K-L. New
  `tau_p_forward`, `tau_p_adjoint`, `tau_p_inverse`, and
  `tau_p_filter` mirror the f-k API: forward stacks a (t, x) gather
  into a (τ, p) panel, the LSQR-style `tau_p_inverse` is a true
  per-frequency pseudoinverse (round-trip identity to ~0.1 % on a
  clean monopole gather), and the convenience `tau_p_filter` does
  forward → cosine-tapered slowness mask → adjoint for band-pass
  separation. Unlike `fk_filter`, τ-p tolerates non-uniform
  receiver spacings. New `demos.demo_tau_p_separation` and CLI
  subcommand `fwap taup` exercise the pipeline on the canonical
  P/S/Stoneley monopole gather.
- **Pseudo-Rayleigh / guided-mode picking.** The book (Mari et al.
  1994, Part 1) lists pseudo-Rayleigh alongside P, S, and Stoneley as
  one of the arrivals the rule-based picker must identify in fast
  formations; it was the only mode in that list missing from the
  package. `fwap.picker.DEFAULT_PRIORS` now carries a
  `"PseudoRayleigh"` entry (130-200 us/ft), and Stoneley's lower
  bound has been tightened from 180 to 200 us/ft so the four
  windows are non-overlapping. New phenomenological dispersion law
  `fwap.synthetic.pseudo_rayleigh_dispersion(vs, v_fluid, a_borehole)`
  matches the formation shear slowness at the low-frequency cutoff
  and asymptotes to the borehole-fluid slowness at high frequency.
  `monopole_formation_modes(...)` gained a `f_pr=` kwarg that
  appends a fourth mode at the band-centre slowness predicted by
  that law. New `demos.demo_pseudo_rayleigh` and CLI subcommand
  `fwap pseudorayleigh` exercise the four-mode pipeline end-to-end.
  `viterbi_pick_joint` and `viterbi_posterior_marginals` remain
  hardcoded to the (P, S, Stoneley) triple (extending the trellis
  to 4 modes squares its width); both now subset the default priors
  for backward compatibility and raise on a 4-mode prior dict.
- **DLIS read / write** — `fwap.io.read_dlis`, `fwap.io.write_dlis`,
  and `DlisCurves` mirror the existing LAS API for the binary RP66 v1
```

(continues on next page)

(continued from previous page)

```
  format. Wraps `dlisio` for reading and `dliswriter` for writing.
  Well metadata is re-keyed to LAS-2.0 mnemonics (`WELL`, `COMP`,
  `FLD`, `PROD`, `UWI`) so the same dict can be passed to either
  writer.

### Changed
- **All log-format libraries are now core dependencies.** `lasio`,
  `dlisio`, `dliswriter`, and `segyio` are folded into the base
  `dependencies` list; the `[io]`, `[dlis]`, and `[segy]` extras
  are gone. The corresponding lazy-import helpers (`_require_lasio`,
  `_require_dlisio`, `_require_dliswriter`, `_require_segyio`) and
  their friendly-error guards have been removed.

## [0.4.0] - 2026-04-22

First formally-versioned release. Promotes the port of the 1994 Mari
et al. algorithms from a prototype into a tested, documented Python
package.

### Added
- **Package infrastructure**
  - Repository renamed `src/` to `fwap/` so the `from fwap.X import Y`
    imports throughout the codebase actually resolve.
  - `pyproject.toml` with runtime dependencies (NumPy, SciPy,
    Matplotlib) and optional `dev` / `docs` / `io` extras.
  - `LICENSE` (MIT).
  - `CITATION.cff` citing both the software and the 1994 book.
  - `README.md` expanded from a title-only placeholder to a
    chapter-to-module map, install + quick-start, companion
    references, and links to tests and docs.
  - `CHANGELOG.md` (this file).
- **Tests** (`tests/`, 83 cases, ~9 s run):
  - One file per algorithm module plus edge cases, sign-convention
    invariants, and demo regression tests that assert on the numerics
    logged by each `demo_*` function.
  - `.github/workflows/ci.yml` runs pytest on Python 3.9 / 3.11 /
    3.12 and smoke-tests `python -m fwap --quiet`.
- **Documentation** (`docs/`):
  - Sphinx skeleton (`conf.py`, `index.rst`, `quickstart.rst`,
    `chapter_map.rst`, `api.rst`, `changelog.rst`) that autogenerates
    an API reference from the docstrings.
  - `.readthedocs.yaml` so the repo can be connected to ReadTheDocs
    without further configuration.
  - CI `docs` job builds the site and uploads the rendered HTML as
    an artifact.
- **New algorithms / modules**:
  - `fwap.rockphysics.elastic_moduli(vp, vs, rho)` -> bulk / shear /
    Young's modulus and Poisson's ratio, closing the loop from
    raw-waveform Vp/Vs to geomechanical curves.
  - `fwap.rockphysics.vp_vs_ratio` (lithology / fluid indicator).
  - `fwap.io.read_las` / `fwap.io.write_las` via the optional
    `lasio` dependency.
```

(continues on next page)

(continued from previous page)

```
- **API extensions**:
  - `fwap.anisotropy.alford_rotation_from_tensor` accepts a packed
    `(2, 2, n_samples)` cross-dipole tensor.
  - `fwap.dip.AzimuthalGather` NamedTuple returned by
    `synthesize_azimuthal_arrival` (tuple unpacking still works).
  - `fwap.synthetic.ArrayGeometry.schlumberger_array_sonic()`
    classmethod documenting the canonical 8/10 ft/6 in reference
    geometry.
  - `fwap.logger` is the shared package logger; every submodule
    imports it from `fwap._common`.
  - `fwap.picker.track_modes` gained `continuity_tol_cap_factor`
    (default 3.0) bounding the effective jump tolerance across long
    runs of missed picks.
- **Performance**: per-frequency inner loops vectorised in
  `dispersive_stc`, `phase_slowness_from_f_k` (both methods),
  `phase_slowness_matrix_pencil`, and `synthesize_gather` (5x-42x
  speedups on the reference benchmarks).
- **Docstrings**: every algorithm module now carries a book reference
  and a chapter map linking back to Mari, Coppens, Gavin & Wicquart
  (1994); every public symbol has Parameters / Returns sections with
  units and array shapes.

### Changed
- `solve_intercept_time` with `mean_delay_zero=True` now emits two
  separate zero-sum constraint rows (one per delay block). The
  previous single joint row left a
  `(d_src, d_rec) -> (d_src + c, d_rec - c)` gauge unresolved.
- `centroid_frequency_shift_Q` / `spectral_ratio_Q`: internal
  variable renamed `t_arr -> t_travel` to match Quan & Harris (1997);
  the docstring now flags the Gaussian-source assumption.
- `shear_slowness_from_dispersion` emits `logging.warning` when it
  has to fall back from the quality-weighted set to the unweighted
  set.
- `_coherence_after_detilt` now calls `fwap.coherence.semblance`
  instead of reimplementing the ratio; the two sites can no longer
  diverge on the semblance definition.
- `dispersion_family` (argument of `dispersive_stc`) and
  `Mode.dispersion` now document an array-in / array-out contract;
  `dipole_flexural_dispersion`'s type hint reflects it.

### Fixed
- Alford rotation docstring formula was off by a factor of 2:
  `tan(4 theta) = (A + B) / (C - D) = 2 A / (C - D)`. The code was
  already correct.
- `fk_filter` sign convention (`S = -k/f`) is now documented in the
  function's docstring rather than only an inline comment.

### Removed
- `fwap.coherence.semblance` no longer takes a `min_energy`
  parameter. The default was effectively dead (it fired only on a
  bit-exact zero sum). Callers needing an energy floor should filter
  windows upstream.
```

(continues on next page)

(continued from previous page)

```
- `fwap.picker.pick_modes` / `track_modes`:
  `selection_rule="earliest"` was carrying a legacy fwap03 picker
  rule with no tests exercising it. Removed; use
  `"max_coherence"` or `"scored"` (the default).
- `fwap.picker.track_modes`: the deprecated `max_slow_jump_per_depth`
  keyword alias was removed. Callers must use `max_slow_jump`; the
  old name now raises `TypeError` with the standard "unexpected
  keyword argument" message.
- `fwap.tomography.solve_intercept_time`: the deprecated
  `smoothing` scalar was removed. Use the explicit per-block
  weights `smooth_s`, `smooth_src`, `smooth_rec` (all default 0.0).
- Internal references to legacy version tags ("fwap01", "fwap02",
  "v1", "v2", "testing_fwap03.py") have been stripped from
  docstrings and user-facing comments. Those were porting notes
  that did not belong in the published API documentation.
```

CHAPTER

SIX

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

f

fwap.anisotropy, 58
fwap.attenuation, 65
fwap.coherence, 11
fwap.cylindrical, 87
fwap.dip, 53
fwap.dispersion, 44
fwap.geomechanics, 80
fwap.io, 102
fwap.lwd, 89
fwap.picker, 14
fwap.plotting, 109
fwap.rockphysics, 68
fwap.synthetic, 96
fwap.tomography, 36
fwap.wavesep, 29

INDEX

A

alford (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60
alford_rotation() (*in module fwap.anisotropy*), 61
alford_rotation_from_tensor() (*in module fwap.anisotropy*), 62
AlfordResult (*class in fwap.anisotropy*), 58
altered_zone_estimate() (*in module fwap.tomography*), 38
AlteredZoneEstimate (*class in fwap.tomography*), 37
amplitude (*fwap.coherence.STCResult attribute*), 13
amplitude (*fwap.picker.ModePick attribute*), 16
amplitude (*fwap.synthetic.Mode attribute*), 99
angle (*fwap.anisotropy.AlfordResult attribute*), 58, 59
anisotropy_strength (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60
apply_moveout() (*in module fwap.wavesep*), 30
ArrayGeometry (*class in fwap.synthetic*), 97
assemble_observations_from_picks() (*in module fwap.tomography*), 39
AttenuationResult (*class in fwap.attenuation*), 66
axial_offsets (*fwap.dip.AzimuthalGather attribute*), 54
axial_offsets (*fwap.lwd.QuadrupoleRingGather attribute*), 91
azimuth (*fwap.dip.DipResult attribute*), 55, 56
azimuth_axis (*fwap.dip.DipResult attribute*), 56
AzimuthalGather (*class in fwap.dip*), 54
azimuths (*fwap.dip.AzimuthalGather attribute*), 54
azimuths (*fwap.lwd.QuadrupoleRingGather attribute*), 91

B

bandpass() (*in module fwap.dispersion*), 46
brittleness (*fwap.geomechanics.GeomechanicsIndices attribute*), 82
brittleness_index_rickman() (*in module fwap.geomechanics*), 82
build_design_matrix() (*in module fwap.tomography*), 40
build_design_matrix_segmented() (*in module fwap.tomography*), 40

C

c44 (*fwap.anisotropy.ThomsenGammaResult attribute*), 61
c66 (*fwap.anisotropy.ThomsenGammaResult attribute*), 61
centroid_frequency_shift_Q() (*in module fwap.attenuation*), 67
classification (*fwap.dispersion.FlexuralDispersionDiagnosis attribute*), 45, 46
classify_flexural_anisotropy() (*in module fwap.dispersion*), 46
closure_stress (*fwap.geomechanics.GeomechanicsIndices attribute*), 82
closure_stress() (*in module fwap.geomechanics*), 83
coherence (*fwap.coherence.STCResult attribute*), 12, 13
coherence (*fwap.dip.DipResult attribute*), 55, 56
coherence (*fwap.picker.ModePick attribute*), 16
coherences (*fwap.picker.PosteriorPick attribute*), 18
cross_energy_ratio (*fwap.anisotropy.AlfordResult attribute*), 59
crossover_frequency (*fwap.dispersion.FlexuralDispersionDiagnosis attribute*), 46
curves (*fwap.io.DlisCurves attribute*), 103, 104
curves (*fwap.io.LasCurves attribute*), 104, 105

D

data (*fwap.dip.AzimuthalGather attribute*), 54
data (*fwap.io.SegyGather attribute*), 105, 106
data (*fwap.lwd.QuadrupoleRingGather attribute*), 90, 91
delay_rec (*fwap.tomography.InterceptTimeResult attribute*), 38
delay_src (*fwap.tomography.InterceptTimeResult attribute*), 38
delay_to_altered_zone_thickness() (*in module fwap.tomography*), 41
delay_to_altered_zone_velocity_contrast() (*in module fwap.tomography*), 42
delta_high (*fwap.dispersion.FlexuralDispersionDiagnosis attribute*), 45, 46
delta_low (*fwap.dispersion.FlexuralDispersionDiagnosis attribute*), 45, 46
depth (*fwap.io.DlisCurves attribute*), 103, 104
depth (*fwap.io.LasCurves attribute*), 104, 105

depth (*fwap.picker.DepthPicks attribute*), 15
depth (*fwap.picker.PickQualityFlags attribute*), 16, 17
DepthPicks (*class in fwap.picker*), 15
depths (*fwap.tomography.InterceptTimeResult attribute*), 38
diagnostic (*fwap.attenuation.AttenuationResult attribute*), 66
dip (*fwap.dip.DipResult attribute*), 55, 56
dip_axis (*fwap.dip.DipResult attribute*), 55, 56
dipole_flexural_dispersion() (*in module fwap.synthetic*), 99
DipResult (*class in fwap.dip*), 55
dispersion (*fwap.synthetic.Mode attribute*), 99
DispersionCurve (*class in fwap.dispersion*), 44
dispersive_pseudo_rayleigh_stc() (*in module fwap.dispersion*), 48
dispersive_stc() (*in module fwap.dispersion*), 51
DlisCurves (*class in fwap.io*), 103
dr (*fwap.synthetic.ArrayGeometry attribute*), 97, 98
dt (*fwap.dip.AzimuthalGather attribute*), 54
dt (*fwap.io.SegyGather attribute*), 105, 106
dt (*fwap.lwd.QuadrupoleRingGather attribute*), 90, 91
dt (*fwap.synthetic.ArrayGeometry attribute*), 97, 98

E

elastic_moduli() (*in module fwap.rockphysics*), 71
ElasticModuli (*class in fwap.rockphysics*), 69
estimate_dip() (*in module fwap.dip*), 56

F

f0 (*fwap.synthetic.Mode attribute*), 99
fast (*fwap.anisotropy.AlfordResult attribute*), 59
filter_picks_by_shape() (*in module fwap.picker*), 18
filter_track_by_shape() (*in module fwap.picker*), 19
find_peaks() (*in module fwap.coherence*), 13
fk_filter() (*in module fwap.wavesep*), 30
fk_forward() (*in module fwap.wavesep*), 31
fk_inverse() (*in module fwap.wavesep*), 31
flagged (*fwap.picker.PickQualityFlags attribute*), 17
flexural_dispersion_physical() (*in module fwap.cylindrical*), 87
FlexuralDispersionDiagnosis (*class in fwap.dispersion*), 45
fracability (*fwap.geomechanics.GeomechanicsIndices attribute*), 82
fracability_index() (*in module fwap.geomechanics*), 84
fracture_indicator (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60
frame_name (*fwap.io.DlisCurves attribute*), 104
freq (*fwap.dispersion.DispersionCurve attribute*), 45
from_imperial() (*fwap.synthetic.ArrayGeometry class method*), 98
fwap.anisotropy
    module, 58
fwap.attenuation
    module, 65
fwap.coherence
    module, 11
fwap.cylindrical
    module, 87
fwap.dip
    module, 53
fwap.dispersion
    module, 44
fwap.geomechanics
    module, 80
fwap.io
    module, 102
fwap.lwd
    module, 89
fwap.picker
    module, 14
fwap.plotting
    module, 109
fwap.rockphysics
    module, 68
fwap.synthetic
    module, 96
fwap.tomography
    module, 36
fwap.wavesep
    module, 29

G

gabor() (*in module fwap.synthetic*), 100
gamma (*fwap.anisotropy.ThomsenGammaResult attribute*), 61
gassmann_fluid_substitution() (*in module fwap.rockphysics*), 71
GassmannResult (*class in fwap.rockphysics*), 70
geomechanics_indices() (*in module fwap.geomechanics*), 84
GeomechanicsIndices (*class in fwap.geomechanics*), 81

H

hill_average() (*in module fwap.rockphysics*), 72
hornby_fracture_aperture() (*in module fwap.rockphysics*), 73

I

index_type (*fwap.io.DlisCurves attribute*), 104
intercept (*fwap.synthetic.Mode attribute*), 99
InterceptTimeResult (*class in fwap.tomography*), 37

K

k (*fwap.rockphysics.ElasticModuli attribute*), 69, 70
k_sat (*fwap.rockphysics.GassmannResult attribute*), 70

L

lambda_ (*fwap.rockphysics.ElasticModuli attribute*), 70
LasCurves (*class in fwap.io*), 104
lwd_collar_mode() (*in module fwap.lwd*), 91
lwd_quadrupole_priors() (*in module fwap.lwd*), 92

M

max_horizontal_stress_azimuth (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 59, 60
method (*fwap.attenuation.AttenuationResult attribute*), 66, 67
method (*fwap.tomography.InterceptTimeResult attribute*), 38
min_horizontal_stress_azimuth (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60
Mode (*class in fwap.synthetic*), 99
ModePick (*class in fwap.picker*), 16
module
    fwap.anisotropy, 58
    fwap.attenuation, 65
    fwap.coherence, 11
    fwap.cylindrical, 87
    fwap.dip, 53
    fwap.dispersion, 44
    fwap.geomechanics, 80
    fwap.io, 102
    fwap.lwd, 89
    fwap.picker, 14
    fwap.plotting, 109
    fwap.rockphysics, 68
    fwap.synthetic, 96
    fwap.tomography, 36
    fwap.wavesep, 29
monopole_formation_modes() (*in module fwap.synthetic*), 100
mu (*fwap.rockphysics.ElasticModuli attribute*), 70
mu_sat (*fwap.rockphysics.GassmannResult attribute*), 70

N

n_rec (*fwap.synthetic.ArrayGeometry attribute*), 97, 98
n_samples (*fwap.io.SegyGather attribute*), 106
n_samples (*fwap.synthetic.ArrayGeometry attribute*), 98
n_traces (*fwap.io.SegyGather attribute*), 106
name (*fwap.picker.ModePick attribute*), 16
name (*fwap.synthetic.Mode attribute*), 99
narrow_band_stc() (*in module fwap.dispersion*), 51
notch_slowness_band() (*in module fwap.lwd*), 92

O

offsets (*fwap.io.SegyGather attribute*), 106
offsets (*fwap.synthetic.ArrayGeometry property*), 98
onset_polarity() (*in module fwap.picker*), 19
overburden_stress() (*in module fwap.geomechanics*), 85

P

p_absent (*fwap.picker.PosteriorPick attribute*), 18
phase_slowness_from_f_k() (*in module fwap.dispersion*), 52
phase_slowness_matrix_pencil() (*in module fwap.dispersion*), 52
pick_modes() (*in module fwap.picker*), 20
PickQualityFlags (*class in fwap.picker*), 16
picks (*fwap.picker.DepthPicks attribute*), 15
poisson (*fwap.rockphysics.ElasticModuli attribute*), 70
PosteriorPick (*class in fwap.picker*), 17
probabilities (*fwap.picker.PosteriorPick attribute*), 18
pseudo_rayleigh_dispersion() (*in module fwap.synthetic*), 101

Q

q (*fwap.attenuation.AttenuationResult attribute*), 66, 67
q_sigma (*fwap.attenuation.AttenuationResult attribute*), 66, 67
quadrupole_stack() (*in module fwap.lwd*), 93
QuadrupoleRingGather (*class in fwap.lwd*), 90
quality (*fwap.dispersion.DispersionCurve attribute*), 45
quality_control_picks() (*in module fwap.picker*), 20
quality_control_track() (*in module fwap.picker*), 21

R

rayleigh_speed() (*in module fwap.cylindrical*), 89
read_dlis() (*in module fwap.io*), 106
read_las() (*in module fwap.io*), 107
read_segy() (*in module fwap.io*), 107
reasons (*fwap.dispersion.FlexuralDispersionDiagnosis attribute*), 46
reasons (*fwap.picker.PickQualityFlags attribute*), 17
rec_delay (*fwap.synthetic.Mode attribute*), 99
refined (*fwap.dip.DipResult attribute*), 56
reuss_average() (*in module fwap.rockphysics*), 75
ricker() (*in module fwap.synthetic*), 101
rms_residual (*fwap.tomography.InterceptTimeResult attribute*), 38
rotation_quality (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60

S

sand_stability (*fwap.geomechanics.GeomechanicsIndices attribute*), 82

sand_stability_indicator() (*in module fwap.geomechanics*), 85
save_figure() (*in module fwap.plotting*), 109
schlumberger_array_sonic() (*fwap.synthetic.ArrayGeometry class method*), 98
SegyGather (*class in fwap.io*), 105
semblance() (*in module fwap.coherence*), 13
sequential_kl_separation() (*in module fwap.wavesep*), 32
shear_slowness_from_dispersion() (*in module fwap.dispersion*), 53
sigma (*fwap.synthetic.Mode attribute*), 99
sigma_delay_rec (*fwap.tomography.InterceptTimeResult attribute*), 38
sigma_delay_src (*fwap.tomography.InterceptTimeResult attribute*), 38
sigma_slowness (*fwap.tomography.InterceptTimeResult attribute*), 38
slow (*fwap.anisotropy.AlfordResult attribute*), 59
slowness (*fwap.coherence.STCResult attribute*), 12, 13
slowness (*fwap.dip.AzimuthalGather attribute*), 54
slowness (*fwap.dispersion.DispersionCurve attribute*), 45
slowness (*fwap.picker.ModePick attribute*), 16
slowness (*fwap.synthetic.Mode attribute*), 99
slowness (*fwap.tomography.InterceptTimeResult attribute*), 38
slowness_altered (*fwap.tomography.AlteredZoneEstimate attribute*), 37
slowness_contrast (*fwap.tomography.AlteredZoneEstimate attribute*), 37
slownesses (*fwap.picker.PosteriorPick attribute*), 18
solve_intercept_time() (*in module fwap.tomography*), 43
source_azimuth (*fwap.lwd.QuadrupoleRingGather attribute*), 91
spectral_ratio_Q() (*in module fwap.attenuation*), 67
splitting_time_delay (*fwap.anisotropy.StressAnisotropyEstimate attribute*), 60, 61
src_delay (*fwap.synthetic.Mode attribute*), 99
stc() (*in module fwap.coherence*), 14
STCResult (*class in fwap.coherence*), 12
step (*fwap.io.DlisCurves attribute*), 104
step (*fwap.io.LasCurves attribute*), 105
stoneley_amplitude_fracture_indicator() (*in module fwap.rockphysics*), 75
stoneley_horizontal_shear_modulus() (*in module fwap.anisotropy*), 62
stoneley_permeability_indicator() (*in module fwap.rockphysics*), 76
stoneley_reflection_coefficient() (*in module fwap.rockphysics*), 77
stress_anisotropy_from_alford() (*in module fwap.anisotropy*), 63
StressAnisotropyEstimate (*class in fwap.anisotropy*), 59
surface (*fwap.dip.DipResult attribute*), 55, 56
svd_project() (*in module fwap.wavesep*), 32
synthesize_azimuthal_arrival() (*in module fwap.dip*), 57
synthesize_gather() (*in module fwap.synthetic*), 101
synthesize_lwd_gather() (*in module fwap.lwd*), 94
synthesize_quadrupole_lwd_gather() (*in module fwap.lwd*), 95

T

t (*fwap.synthetic.ArrayGeometry property*), 98
tau_p_adjoint() (*in module fwap.wavesep*), 32
tau_p_filter() (*in module fwap.wavesep*), 33
tau_p_forward() (*in module fwap.wavesep*), 34
tau_p_inverse() (*in module fwap.wavesep*), 35
textual_header (*fwap.io.SegyGather attribute*), 106
thickness (*fwap.tomography.AlteredZoneEstimate attribute*), 37
thomsen_gamma() (*in module fwap.anisotropy*), 64
thomsen_gamma_from_logs() (*in module fwap.anisotropy*), 64
ThomsenGammaResult (*class in fwap.anisotropy*), 61
time (*fwap.coherence.STCResult attribute*), 12, 13
time (*fwap.picker.ModePick attribute*), 16
time_order_ok (*fwap.picker.PickQualityFlags attribute*), 17
times (*fwap.picker.PosteriorPick attribute*), 18
tool_radius (*fwap.dip.AzimuthalGather attribute*), 54
tool_radius (*fwap.lwd.QuadrupoleRingGather attribute*), 91
tr_offset (*fwap.synthetic.ArrayGeometry attribute*), 97, 98
track_modes() (*in module fwap.picker*), 21
track_to_log_curves() (*in module fwap.picker*), 22

U

ucs (*fwap.geomechanics.GeomechanicsIndices attribute*), 82
unapply_moveout() (*in module fwap.wavesep*), 36
unconfined_compressive_strength() (*in module fwap.geomechanics*), 86
units (*fwap.io.DlisCurves attribute*), 103, 104
units (*fwap.io.LasCurves attribute*), 105

V

viterbi_pick() (*in module fwap.picker*), 24
viterbi_pick_joint() (*in module fwap.picker*), 25
viterbi_posterior_marginals() (*in module fwap.picker*), 27
voigt_average() (*in module fwap.rockphysics*), 78
vp_vs (*fwap.picker.PickQualityFlags attribute*), 16, 17

vp_vs_in_band (*fwap.picker.PickQualityFlags attribute*), 17
vp_vs_ratio() (*in module fwap.rockphysics*), 78
vs_from_stoneley_slow_formation() (*in module fwap.rockphysics*), 78

W

wavelet (*fwap.synthetic.Mode attribute*), 99
wavelet_shape_score() (*in module fwap.picker*), 29
well (*fwap.io.DlisCurves attribute*), 103, 104
well (*fwap.io.LasCurves attribute*), 105
wiggle_plot() (*in module fwap.plotting*), 110
window_length (*fwap.coherence.STCResult attribute*), 12, 13
write_dlis() (*in module fwap.io*), 107
write_las() (*in module fwap.io*), 108
write_segy() (*in module fwap.io*), 108

Y

young (*fwap.rockphysics.ElasticModuli attribute*), 70